calculus-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `calculus_ai999x.about()` for the same answer.

Every capability calculus-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import calculus_ai999x

result = calculus_ai999x.solve('derivative_at_point', f='x^3 - 2x', a=2)
result.show()
# derivative = 10, derivative_expression = 3*x^2 - 2
```

By topic

topic	capabilities
Derivatives	Derivative at a point, Second derivative at a point, Higher derivative at a point, Implicit differentiation, Mean value theorem
Derivatives / concavity	Inflection points
Derivatives / curvature	Curvature of a graph
Derivatives / linearization	Linear approximation
Derivatives / motion	Velocity and acceleration from position
Derivatives / optimization	Absolute extrema on a closed interval, Critical points and their classification
Derivatives / rates of change	Average rate of change
Derivatives / tangent and normal lines	Tangent line, Normal line
Fourier series	Fourier coefficients
Integrals	Definite integral, Derivative of an integral (fundamental theorem)
Integrals / applications	Average value of a function
Integrals / arc length	Arc length of a graph
Integrals / area	Area between two curves, Area enclosed by two curves
Integrals / centroids	Centroid of a region under a curve
Integrals / improper integrals	Improper integral to infinity
Integrals / Riemann sums	Left Riemann sum, Right Riemann sum
Integrals / surface area	Surface area of revolution

topic	capabilities
Integrals / volumes of revolution	Volume of revolution (disk method), Volume of revolution (washer method), Volume of revolution (shell method)
Limits	Limit at a point, Limit at infinity
Multiple integrals	Double integral over a rectangle, Double integral over a region, Triple integral over a box, Surface area of a graph
Multivariable calculus	Partial derivative, Gradient, Directional derivative, Tangent plane
Multivariable calculus / change of variables	Jacobian determinant
Multivariable calculus / optimization	Second derivative test (two variables)
Numerical differentiation	Finite-difference derivative
Numerical integration	Trapezoidal rule, Simpson's rule, Midpoint rule, Gauss-Legendre quadrature
Numerical methods / root finding	Newton's method
Parametric curves	Arc length of a parametric curve, Slope and concavity of a parametric curve
Polar curves	Area in polar coordinates
Series	Sum of an infinite geometric series, Partial sum of a geometric series, Sum of an arithmetic series, Sum of an infinite series, Ratio test
Series / Taylor	Taylor polynomial
Special functions	Gamma function, Beta function, Error function, Regularized incomplete gamma functions, Bessel function of the first kind
Special functions / orthogonal polynomials	Legendre polynomial, Chebyshev polynomial
Special functions / series	Riemann zeta function
Vector calculus	Laplacian, Divergence, Curl, Line integral of a vector field, Line integral with respect to arc length, Green's theorem on a rectangle, Outward flux through a box (divergence theorem)

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
derivative_at_point	Derivative at a point	f, a	derivative, derivative_expression
second_derivative_at_point	Second derivative at a point	f, a	second_derivative, second_derivative_expression
nth_derivative_at_point	Higher derivative at a point	f, n, a	nth_derivative, nth_derivative_expression
tangent_line	Tangent line	f, a	slope, intercept, y0, equation
normal_line	Normal line	f, a	slope, intercept, y0
average_rate_of_change	Average rate of change	f, a, b	average_rate, fa, fb
velocity_acceleration	Velocity and acceleration from position	s, t0	velocity, acceleration, speed, position
linear_approximation	Linear approximation	f, a, x1	approximation, actual, error, slope
curvature	Curvature of a graph	f, a	curvature, radius_of_curvature
definite_integral	Definite integral	f, a, b	integral
average_value	Average value of a function	f, a, b	average_value, integral
area_between_curves	Area between two curves	f, g, a, b	area, net_integral, crossings
area_enclosed_curves	Area enclosed by two curves	f, g	area, x_left, x_right, intersections

id	operation	inputs	outputs
disk_volume	Volume of revolution (disk method)	f, a, b	volume
washer_volume	Volume of revolution (washer method)	f, g, a, b	volume
shell_volume	Volume of revolution (shell method)	f, a, b	volume
arc_length	Arc length of a graph	f, a, b	arc_length
surface_area_revolution	Surface area of revolution	f, a, b	surface_area
centroid_region	Centroid of a region under a curve	f, a, b	area, x_bar, y_bar
improper_integral	Improper integral to infinity	f, a	integral
left_riemann_sum	Left Riemann sum	f, a, b, n	approximation
right_riemann_sum	Right Riemann sum	f, a, b, n	approximation
trapezoid_rule	Trapezoidal rule	f, a, b, n	approximation, exact, error
simpson_rule	Simpson's rule	f, a, b, n	approximation, exact, error
midpoint_rule	Midpoint rule	f, a, b, n	approximation, exact, error
gauss_legendre	Gauss-Legendre quadrature	f, a, b, n	approximation, exact, error
numerical_derivative	Finite-difference derivative	f, a, h, method	approximation, exact, error
newton_method	Newton's method	f, x0, n	x_n, f_x_n, iterates
limit_at_point	Limit at a point	f, a	limit
limit_at_infinity	Limit at infinity	f	limit
implicit_derivative	Implicit differentiation	F, p	dy_dx
extrema_on_interval	Absolute extrema on a closed interval	f, a, b	absolute_max, absolute_min, critical_points
critical_points_classify	Critical points and their classification	f	critical_points, local_maxima, local_minima
inflection_points	Inflection points	f	inflection_points
mean_value_theorem	Mean value theorem	f, a, b	secant_slope, c_values
ftc_derivative	Derivative of an integral (fundamental theorem)	f, a, g, x0	derivative
geometric_series_sum	Sum of an infinite geometric series	a, r	sum
geometric_partial_sum	Partial sum of a geometric series	a, r, n	sum
arithmetic_series_sum	Sum of an arithmetic series	a1, d, n	sum, last_term
series_sum	Sum of an infinite series	a, k	sum
ratio_test	Ratio test	a	ratio_limit, verdict
taylor_polynomial	Taylor polynomial	f, a, n, x1	coefficients, value, function_value, error
partial_derivative	Partial derivative	f, var, p	partial
gradient	Gradient	f, p	gradient, magnitude
directional_derivative	Directional derivative	f, p, u	directional_derivative
tangent_plane	Tangent plane	f, p	z0, fx, fy, c

id	operation	inputs	outputs
hessian_classification	Second derivative test (two variables)	f, p	fxx, fxy, fyy, D, classification
laplacian	Laplacian	f, p	laplacian
divergence	Divergence	P, Q, R, p	divergence
curl	Curl	P, Q, R, p	curl
jacobian_determinant	Jacobian determinant	xu, yv, p	jacobian
double_integral_rectangle	Double integral over a rectangle	f, a, b, c, d	integral
double_integral_region	Double integral over a region	f, a, b, g1, g2	integral
triple_integral_box	Triple integral over a box	f, x0, x1, y0, y1, z0, z1	integral
surface_area_graph	Surface area of a graph	f, a, b, c, d	surface_area
line_integral_vector	Line integral of a vector field	P, Q, xt, yt, t0, t1	line_integral
line_integral_scalar	Line integral with respect to arc length	f, xt, yt, t0, t1	line_integral
greens_theorem	Green's theorem on a rectangle	P, Q, a, b, c, d	circulation, double_integral
divergence_theorem_box	Outward flux through a box (divergence theorem)	P, Q, R, x0, x1, y0, y1, z0, z1	flux
parametric_arc_length	Arc length of a parametric curve	xt, yt, t0, t1	arc_length
parametric_slope	Slope and concavity of a parametric curve	xt, yt, t0	dy_dx, d2y_dx2
polar_area	Area in polar coordinates	r, al, be	area
gamma_function	Gamma function	x	gamma
beta_function	Beta function	a, b	beta
error_function	Error function	x	erf, erfc
regularized_gamma	Regularized incomplete gamma functions	a, x	P, Q
bessel_j	Bessel function of the first kind	n, x	J
riemann_zeta	Riemann zeta function	s	zeta
legendre_polynomial	Legendre polynomial	n, x	P
chebyshev_polynomial	Chebyshev polynomial	n, x	T
fourier_coefficients	Fourier coefficients	f, L, k	a0, a_k, b_k

Derivative at a point, `derivative_at_point`

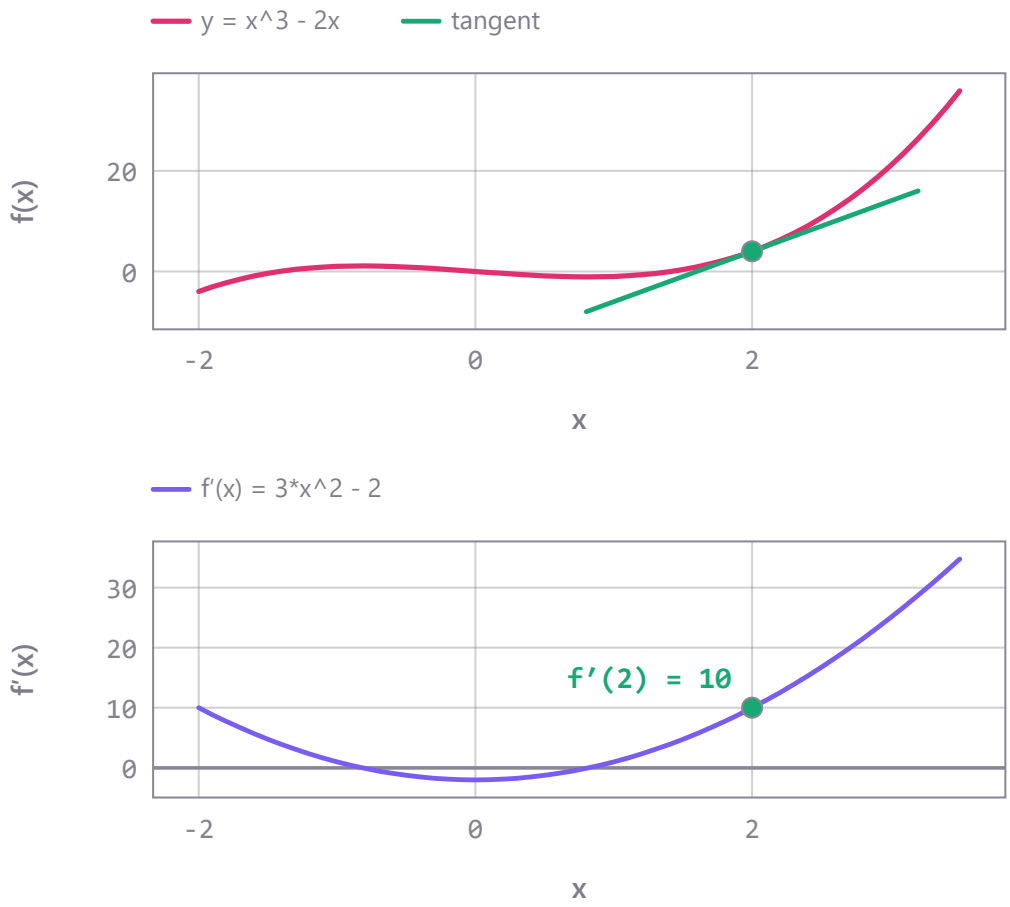
The derivative f'(a) of a function f(x) at $x = a$ (the slope of its graph there).

$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

input	meaning	condition
`f`	the function f(x)	
`a`	the point $x = a$	

output	meaning
`derivative`	f'(a)
`derivative_expression`	f'(x) as an expression

- **Method:** exact symbolic differentiation (sum, product, quotient and chain rules), then evaluation at $x = a$
- **Assumes:** f is defined and differentiable at $x = a$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: y = x^3 - 2x (values computed by the package) with its tangent at x = 2. Bottom: the derivative f′(x) = 3*x^2 - 2 computed by the package at each x; its value at x = 2 is the tangent's slope. Marked: f′(2) = 10.

```
calculus_ai999x.solve('derivative_at_point', f='x^3 - 2x', a=2)
# -> derivative = 10, derivative_expression = 3*x^2 - 2   (SOLVED)
```

```
calculus_ai999x.solve('derivative_at_point', f='sin(3x) + x^2', a=0)
# -> derivative = 3, derivative_expression = 2*x + 3*cos(3*x)   (SOLVED)
```

```
calculus_ai999x.solve('derivative_at_point', f='x^2*exp(x)', a=1)
# -> derivative = 8.15485, derivative_expression = x^2*exp(x) + 2*x*exp(x)   (SOLVED)
```

```
calculus_ai999x.solve('derivative_at_point', f='log(x)', a=4)
# -> derivative = 0.25, derivative_expression = 1/x   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the derivative of f(x) = x^3 - 2x at x = 2.	SOLVED	derivative = 10, derivative_expression = 3*x^2 - 2
What is f'(0) if f(x) = sin(3x) + x^2?	SOLVED	derivative = 3, derivative_expression = 2*x + 3*cos(3*x)
Differentiate x^2*exp(x) and evaluate at x = 1.	SOLVED	derivative = 8.15485, derivative_expression = x^2*exp(x) + 2*x*exp(x)
What is the slope of the curve y = sqrt(x) at x = 9?	SOLVED	derivative = 0.166667, derivative_expression = 1/(2*sqrt(x))
Find the instantaneous rate of change of x^2 + 3x at x = 1.	SOLVED	derivative = 5, derivative_expression = 2*x + 3
Find the derivative of log(x) at x = -1.	INVALID_INPUT	Derivative at a point: the expression is not real there.

Second derivative at a point, `second_derivative_at_point`

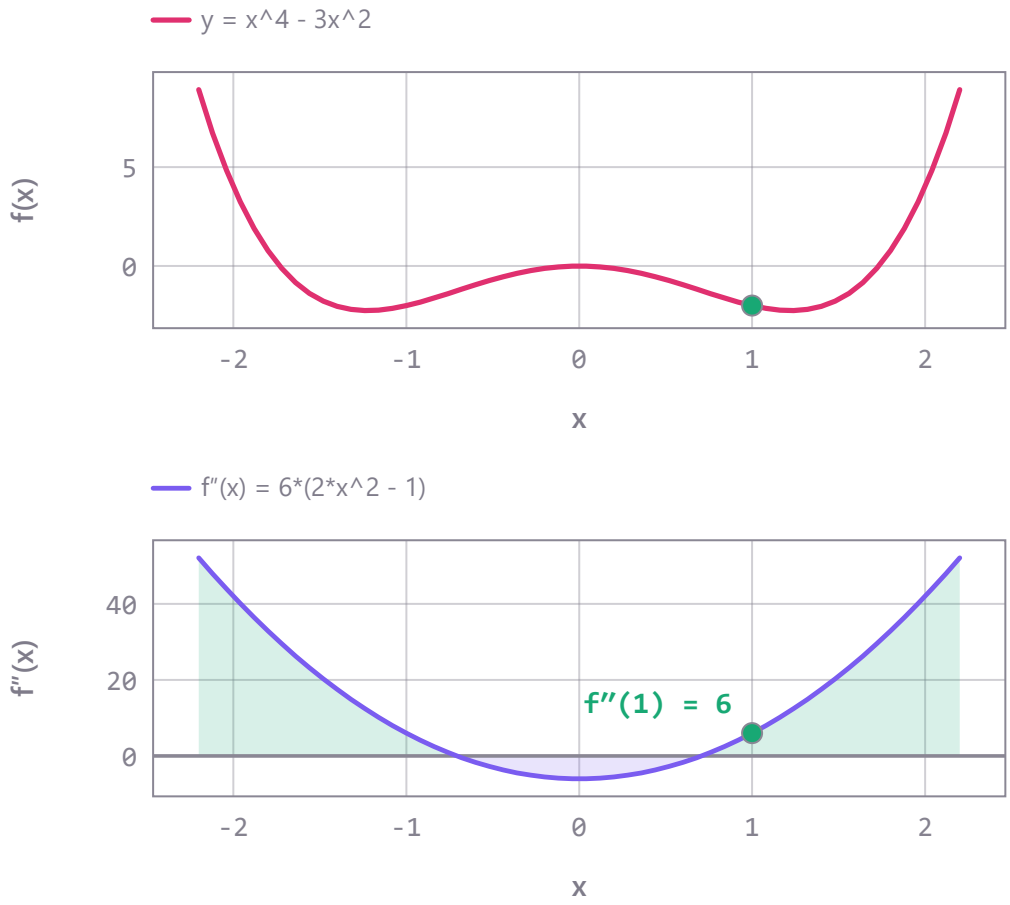
The second derivative f''(a) of f(x) at $x = a$ (concavity: positive means concave up).

$$f''(a) = \frac{d^2 f}{dx^2}\Big|_{x=a}$$

input	meaning	condition
`f`	the function f(x)	
`a`	the point $x = a$	

output	meaning
`second_derivative`	f''(a)
`second_derivative_expression`	f''(x)

- **Method:** exact symbolic differentiation twice, then evaluation at $x = a$
- **Assumes:** f is twice differentiable at $x = a$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: y = x^4 - 3x^2 on [−2.2, 2.2] (values computed by the package). Bottom: the second derivative f″(x) = 6*(2*x^2 - 1) from the package, shaded where it is positive (concave up) and negative (concave down). Marked: f″(1) = 6.

```
calculus_ai999x.solve('second_derivative_at_point', f='x^4 - 3x^2', a=1)
# -> second_derivative = 6, second_derivative_expression = 6*(2*x^2 - 1)   (SOLVED)
```

```
calculus_ai999x.solve('second_derivative_at_point', f='cos(2x)', a=0)
# -> second_derivative = -4, second_derivative_expression = -4*cos(2*x)   (SOLVED)
```

```
calculus_ai999x.solve('second_derivative_at_point', f='exp(2x)', a=0)
# -> second_derivative = 4, second_derivative_expression = 4*exp(2*x)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the second derivative of x^4 - 3x^2 at x = 1.	SOLVED	second_derivative = 6, second_derivative_expression = 6*(2*x^2 - 1)
What is f''(0) if f(x) = cos(2x)?	SOLVED	second_derivative = -4, second_derivative_expression = -4*cos(2*x)
Find the second derivative of sqrt(x) at x = -4.	INVALID_INPUT	Second derivative at a point: the expression is not real there.

Higher derivative at a point, `nth_derivative_at_point`

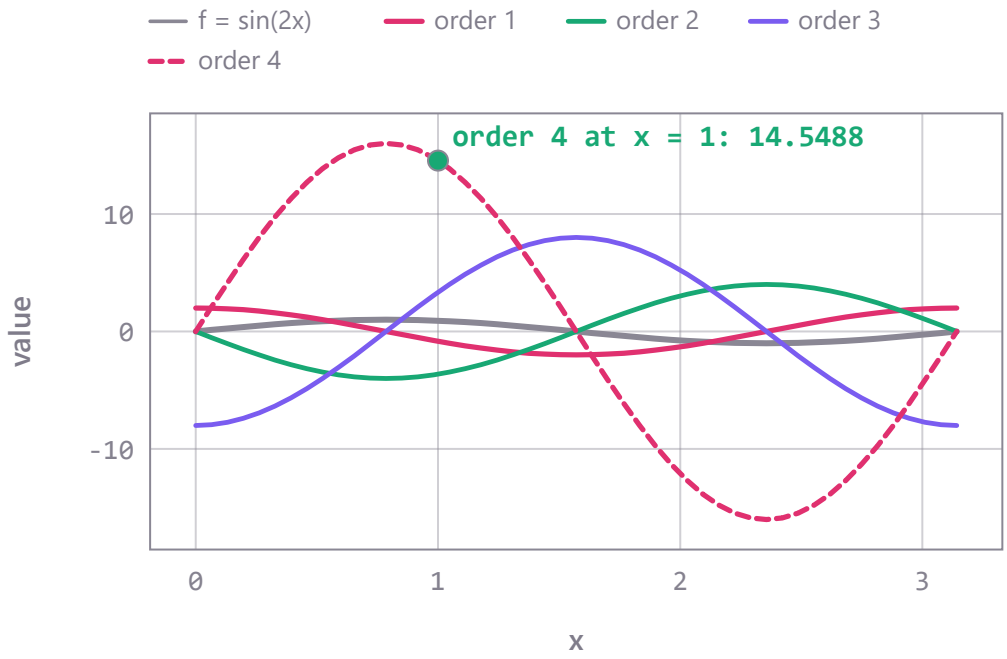
The n-th derivative f^{n}(a) of f(x) at $x = a$.

$$f^{(n)}(a) = \frac{d^n f}{dx^n}\Big|_{x=a}$$

input	meaning	condition
`f`	the function f(x)	
`n`	the order of the derivative	
`a`	the point $x = a$	

output	meaning
`nth_derivative`	f^n(a)
`nth_derivative_expression`	f^n(x)

- **Method:** exact symbolic differentiation n times, then evaluation at $x = a$
- **Assumes:** $1 \le n \le 12$; f is n times differentiable at $x = a$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



f(x) = sin(2x) and its derivatives of order 1 to 4 on [0, π], each computed by the package at 49 points; each differentiation doubles the amplitude. Marked: the order-4 derivative 16*sin(2*x) at x = 1, 14.5488.

```
calculus_ai999x.solve('nth_derivative_at_point', f='sin(2x)', n=4, a=1)
# -> nth_derivative = 14.5488, nth_derivative_expression = 16*sin(2*x)   (SOLVED)
```

```
calculus_ai999x.solve('nth_derivative_at_point', f='x^5', n=3, a=2)
# -> nth_derivative = 240, nth_derivative_expression = 60*x^2   (SOLVED)
```

```
calculus_ai999x.solve('nth_derivative_at_point', f='exp(3x)', n=5, a=0)
# -> nth_derivative = 243, nth_derivative_expression = 243*exp(3*x)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the 4th derivative of sin(2x) at x = 1.	SOLVED	nth_derivative = 14.5488, nth_derivative_expression = 16*sin(2*x)
Find the third derivative of x^5 at x = 2.	SOLVED	nth_derivative = 240, nth_derivative_expression = 60*x^2
What is the 5th derivative of exp(3x) at x = 0?	SOLVED	nth_derivative = 243, nth_derivative_expression = 243*exp(3*x)

Tangent line, `tangent_line`

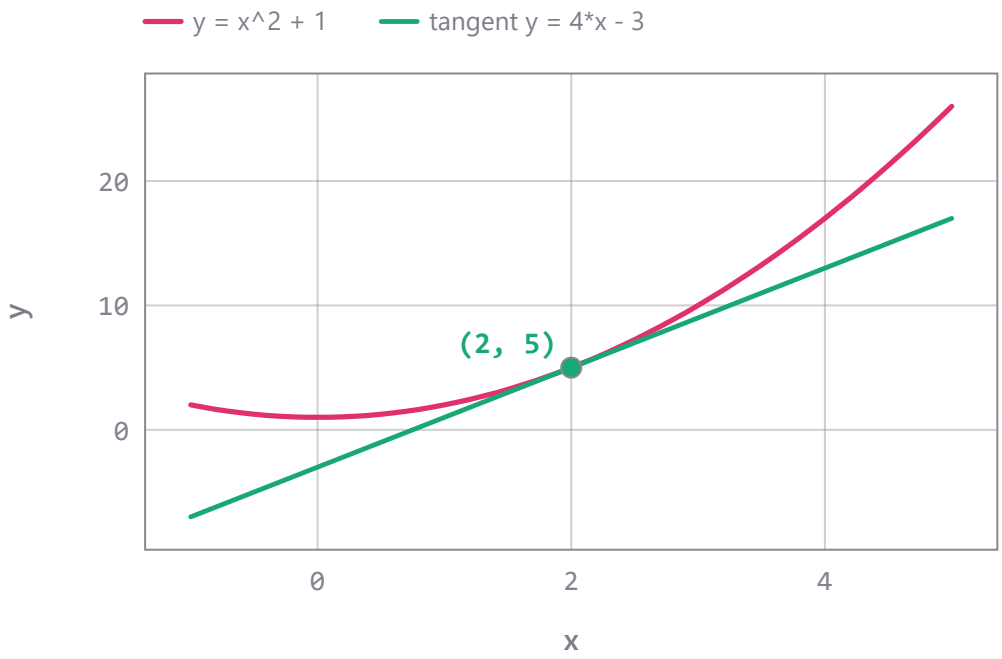
The tangent line $y = mx + b$ to $y = f$(x) at $x = a$: slope $m = f'$(a), intercept $b = f$(a) - a*f'(a).

$$y = f(a) + f'(a)\,(x - a)$$

input	meaning	condition
`f`	the function f(x)	
`a`	the point of tangency $x = a$	

output	meaning
`slope`	slope $m = f'$(a)
`intercept`	y-intercept b
`y0`	f(a)
`equation`	the line as $y = mx + b$

- **Method:** point-slope form through (a, f(a)) with the exact derivative
- **Assumes:** f is differentiable at $x = a$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The graph of y = x^2 + 1 (curve values computed by the package) and its tangent line at x = 2: slope f'(2) = 4, intercept -3. Marked: the point of tangency (2, 5).

```
calculus_ai999x.solve('tangent_line', f='x^2 + 1', a=2)
# -> slope = 4, intercept = -3, y0 = 5, equation = y = 4*x - 3   (SOLVED)
```

```
calculus_ai999x.solve('tangent_line', f='sin(x)', a=0)
# -> slope = 1, intercept = 0, y0 = 0, equation = y = 1*x + 0   (SOLVED)
```

```
calculus_ai999x.solve('tangent_line', f='exp(x)', a=1)
# -> slope = 2.71828, intercept = 0, y0 = 2.71828, equation = y = 2.71828182846*x + 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the equation of the tangent line to y = x^2 + 1 at x = 2.	SOLVED	slope = 4, intercept = -3, y0 = 5, equation = y = 4*x - 3
Find the tangent line to f(x) = sin(x) at x = 0.	SOLVED	slope = 1, intercept = 0, y0 = 0, equation = y = 1*x + 0
What is the slope of the tangent line to y = x^3 at x = -1?	SOLVED	slope = 3, intercept = 2, y0 = -1, equation = y = 3*x + 2

Normal line, `normal_line`

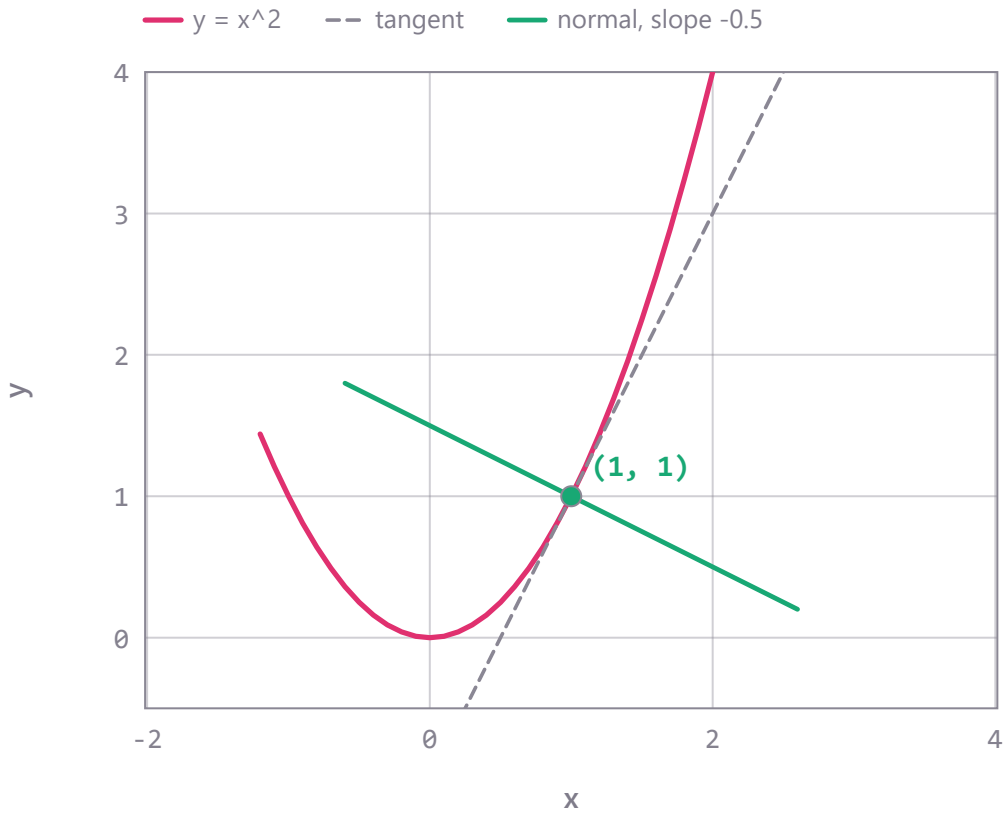
The normal line (perpendicular to the tangent) to $y = f(x)$ at $x = a$: slope -1/f'(a).

$$y = f(a) - \frac{1}{f'(a)}(x - a)$$

input	meaning	condition
`f`	the function f(x)	
`a`	the point $x = a$	

output	meaning
`slope`	slope -1/f'(a)
`intercept`	y-intercept
`y0`	f(a)

- **Method:** perpendicular slope -1/m through (a, f(a))
- **Assumes:** f'$(a) \neq 0$ (otherwise the normal is vertical)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The graph of y = x^2 (values computed by the package) with its normal line at x = 1, drawn with equal scales so the right angle shows: normal slope -0.5, intercept 1.5; the tangent (slope 2) dashed. Marked: (1, 1).

```
calculus_ai999x.solve('normal_line', f='x^2', a=1)
# -> slope = -0.5, intercept = 1.5, y0 = 1   (SOLVED)
```

```
calculus_ai999x.solve('normal_line', f='exp(x)', a=0)
# -> slope = -1, intercept = 1, y0 = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the normal line to y = x^2 at x = 1.	SOLVED	slope = -0.5, intercept = 1.5, y0 = 1
Find the equation of the normal line to f(x) = exp(x) at x = 0.	SOLVED	slope = -1, intercept = 1, y0 = 1
Find the normal line to y = x^2 at x = 0.	INVALID_INPUT	Normal line: the tangent is horizontal there, so the normal line is vertical (x = a) and has no slope.

Average rate of change, `average_rate_of_change`

The average rate of change (f(b) - f(a))/(b - a) of f over [a, b]: the slope of the secant line.

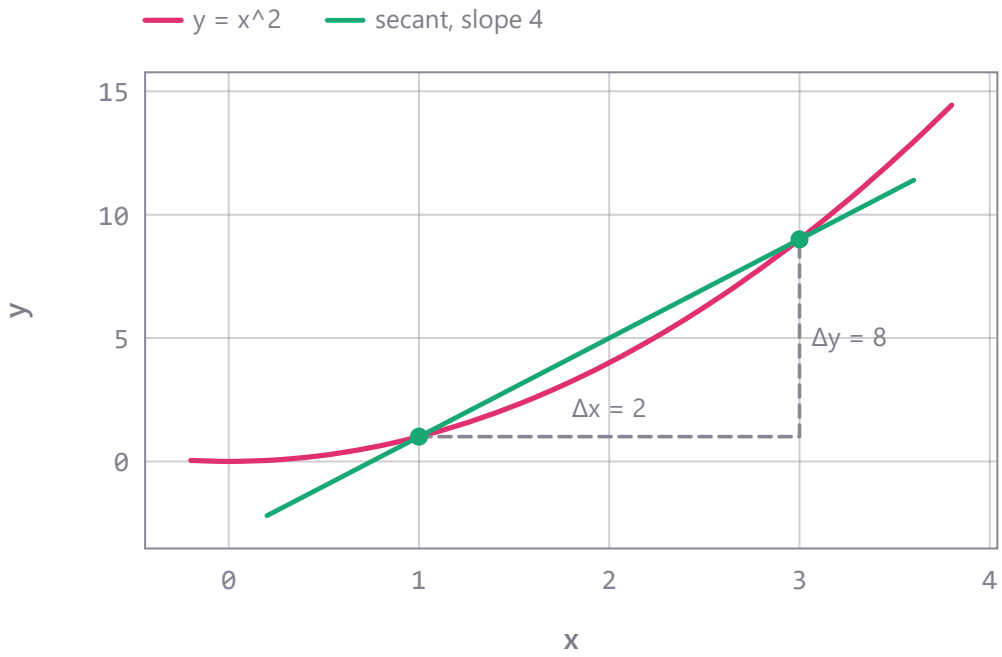
$$\frac{\Delta f}{\Delta x} = \frac{f(b) - f(a)}{b - a}$$

input	meaning	condition
`f`	the function f(x)	

input	meaning	condition
`a`	start of the interval	
`b`	end of the interval	

output	meaning
`average_rate`	(f(b) - f(a))/(b - a)
`fa`	f(a)
`fb`	f(b)

- **Method:** difference quotient of the two function values
- **Assumes:** $a \neq b$; f defined at a and b
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The secant of y = x^2 over [1, 3] (curve values computed by the package): f(1) = 1, f(3) = 9, average rate of change Δy/Δx = 4 (the rise Δy is derived from the two returned values). Marked: the two end points.

```
calculus_ai999x.solve('average_rate_of_change', f='x^2', a=1, b=3)
# -> average_rate = 4, fa = 1, fb = 9   (SOLVED)
```

```
calculus_ai999x.solve('average_rate_of_change', f='sin(x)', a=0, b=1.5707963267948966)
# -> average_rate = 0.63662, fa = 0, fb = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the average rate of change of f(x) = x^2 from x = 1 to x = 3.	SOLVED	average_rate = 4, fa = 1, fb = 9
What is the average rate of change of sin(x) on [0, pi/2]?	SOLVED	average_rate = 0.63662, fa = 0, fb = 1
Find the slope of the secant line of x^3 between x = 0 and x = 2.	SOLVED	average_rate = 4, fa = 0, fb = 8

question	outcome	result
Find the average rate of change of x^2 from x = 2 to x = 2.	INVALID_INPUT	Average rate of change: the two points must differ.

Velocity and acceleration from position, `velocity_acceleration`

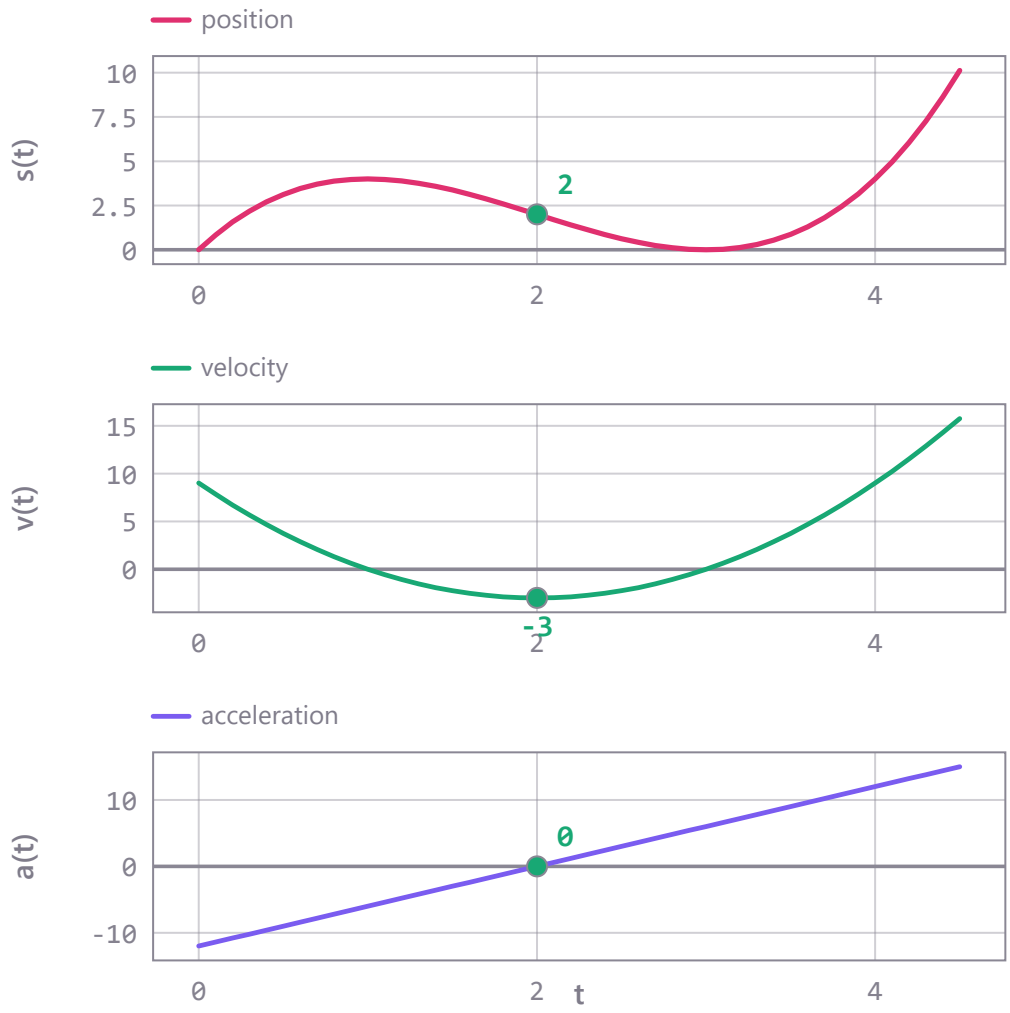
Velocity $v = s'(t)$ and acceleration $a = s''(t)$ at time t0 from a position function s(t).

$$v = \frac{ds}{dt}, \quad a = \frac{d^2s}{dt^2}$$

input	meaning	condition
`s`	the position s(t)	
`t0`	the time t0	

output	meaning
`velocity`	v(t0) = s'(t0)
`acceleration`	a(t0) = s''(t0)
`speed`	|v(t0)|
`position`	s(t0)

- **Method:** exact differentiation of the position once and twice
- **Assumes:** s twice differentiable at t0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Position s(t) = t^3 - 6t^2 + 9t, its velocity and its acceleration for t from 0 to 4.5, each computed by the package; velocity is zero where s turns. Marked at t = 2: s = 2, v = -3, a = 0, speed 3.

```
calculus_ai999x.solve('velocity_acceleration', s='t^3 - 6t^2 + 9t', t0=2)
# -> velocity = -3, acceleration = 0, speed = 3, position = 2   (SOLVED)
```

```
calculus_ai999x.solve('velocity_acceleration', s='5t^2 + 2t', t0=3)
# -> velocity = 32, acceleration = 10, speed = 32, position = 51   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
A particle has position s(t) = t^3 - 6t^2 + 9t. Find its velocity and acceleration at t = 2.	SOLVED	velocity = -3, acceleration = 0, speed = 3, position = 2
Find the velocity of s(t) = 5t^2 + 2t at t = 3.	SOLVED	velocity = 32, acceleration = 10, speed = 32, position = 51
The position of an object is s(t) = 4sin(t). Find its velocity at t = 0.	SOLVED	velocity = 4, acceleration = 0, speed = 4, position = 0

Linear approximation, `linear_approximation`

Estimate f(x1) by the tangent line at $x = a$: L(x1) = f(a) + f'(a)$(x_1 - a)$; with the actual value and error.

$$L(x) = f(a) + f'(a)\,(x - a)$$

input	meaning	condition
`f`	the function f(x)	
`a`	the base point a	
`x1`	the point to estimate	

output	meaning
`approximation`	L(x1)
`actual`	f(x1)
`error`	f(x1) - L(x1)
`slope`	f'(a)

- **Method:** linearization (tangent line) at the base point
- **Assumes:** f differentiable at a
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The tangent-line approximation of y = sqrt(x) at x = 4 (slope 0.25; curve values computed by the package). At the worked example x = 4.1 it gives 2.025 against 2.02485 (error -0.000154327); at x = 7 (chosen, further away) 2.75 against 2.64575, error -0.104249. Marked: the base point.

```
calculus_ai999x.solve('linear_approximation', f='sqrt(x)', a=4, x1=4.1)
# -> approximation = 2.025, actual = 2.02485, error = -0.000154327, slope = 0.25   (SOLVED)
```

```
calculus_ai999x.solve('linear_approximation', f='exp(x)', a=0, x1=0.1)
# -> approximation = 1.1, actual = 1.10517, error = 0.00517092, slope = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use the linear approximation of f(x) = sqrt(x) at x = 4 to estimate f(4.1).	SOLVED	approximation = 2.025, actual = 2.02485, error = -0.000154327, slope = 0.25
Find the linearization of exp(x) at x = 0 and evaluate it at x = 0.1.	SOLVED	approximation = 1.1, actual = 1.10517, error = 0.00517092, slope = 1
Use the tangent line approximation of x^3 at x = 2 to estimate f(2.1).	SOLVED	approximation = 9.2, actual = 9.261, error = 0.061, slope = 12

Curvature of a graph, `curvature`

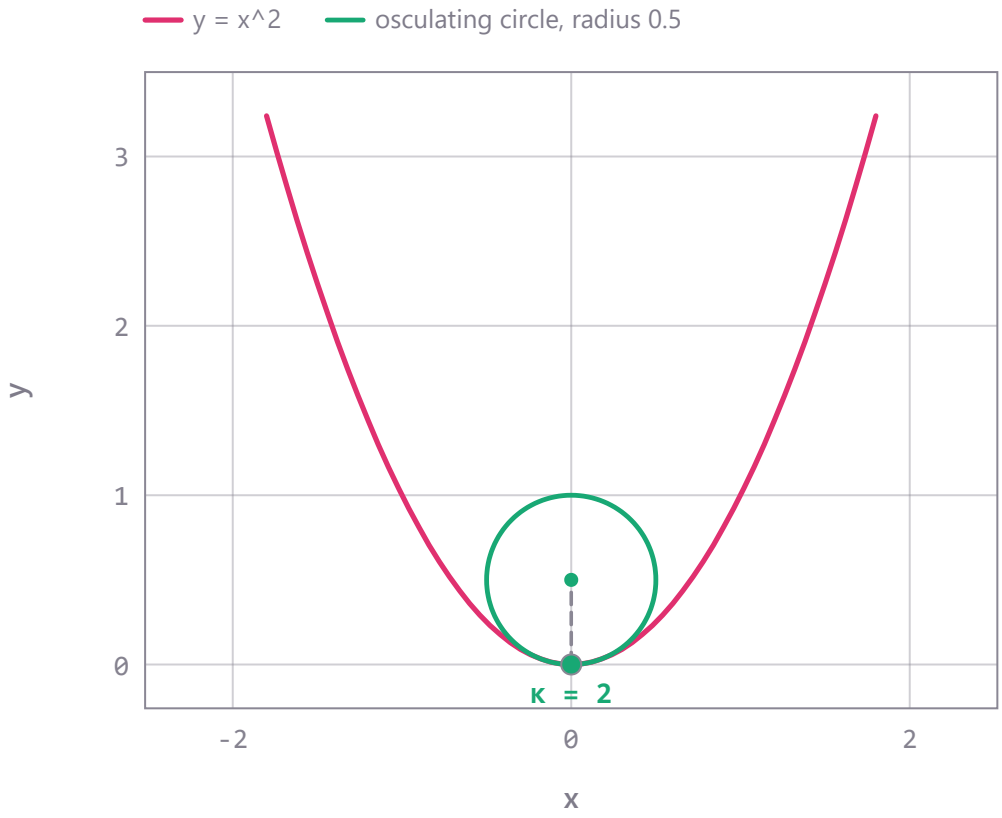
The curvature kappa = |f''| / (1 + f'^2)^(3/2) of $y = f(x)$ at $x = a$, and the radius of curvature $1/\kappa$.

$$\kappa = \frac{|f''(a)|}{(1 + f'(a)^2)^{3/2}}$$

input	meaning	condition
`f`	the function f(x)	
`a`	the point $x = a$	

output	meaning
`curvature`	kappa at $x = a$
`radius_of_curvature`	$1/\kappa$ (absent where kappa = 0)

- **Method:** curvature formula of a graph from the exact first and second derivatives
- **Assumes:** f twice differentiable at a
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y = x^2 (values computed by the package) and its osculating circle at x = 0 (the worked example): the package's curvature κ = 2 gives radius 1/κ = 0.5; the centre is placed along the normal from the returned slope 0 (derived). Marked: the point and its curvature.

```
calculus_ai999x.solve('curvature', f='x^2', a=0)
# -> curvature = 2, radius_of_curvature = 0.5   (SOLVED)
```

```
calculus_ai999x.solve('curvature', f='sin(x)', a=1.5707963267948966)
# -> curvature = 1, radius_of_curvature = 1   (SOLVED)
```

```
calculus_ai999x.solve('curvature', f='x^2', a=1)
# -> curvature = 0.178885, radius_of_curvature = 5.59017   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the curvature of y = x^2 at x = 0.	SOLVED	curvature = 2, radius_of_curvature = 0.5
What is the curvature of f(x) = sin(x) at x = pi/2?	SOLVED	curvature = 1, radius_of_curvature = 1
Find the radius of curvature of y = x^2 at x = 0.	SOLVED	curvature = 2, radius_of_curvature = 0.5

Definite integral, `definite_integral`

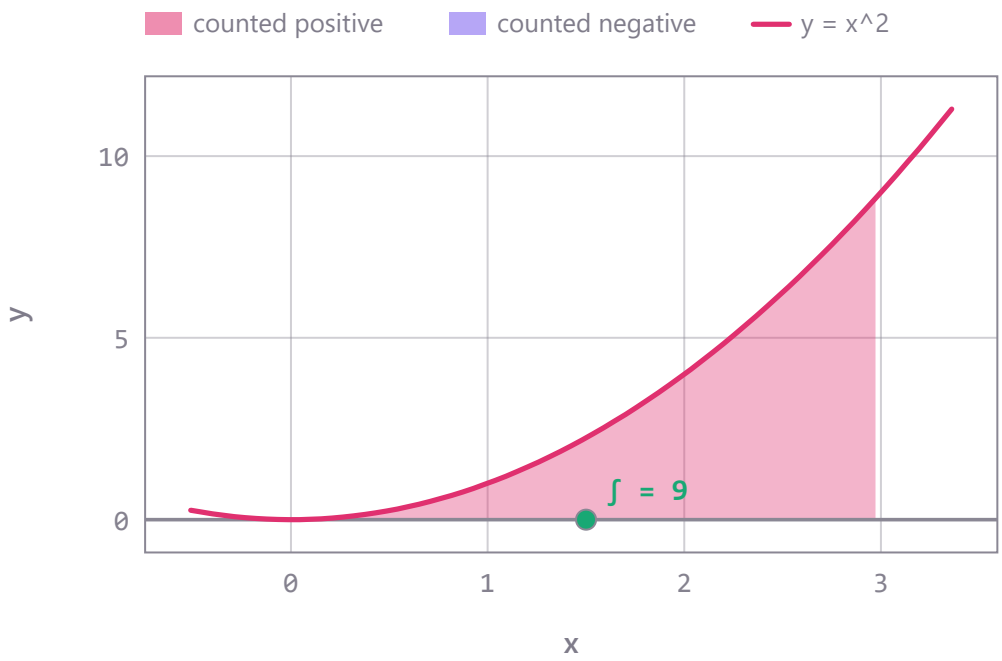
The definite integral of f(x) from $x = a$ to $x = b$ (net signed area).

$$\int_a^b f(x)\,dx$$

input	meaning	condition
`f`	the integrand f(x)	
`a`	lower limit	
`b`	upper limit	

output	meaning
`integral`	the value of the integral

- **Method:** high-precision tanh-sinh quadrature (30 digits) after an exact continuity check on [a, b]
- **Assumes:** f real and continuous on the closed interval between a and b
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The definite integral of y = x^2 from 0 to 3 (the worked example; curve values computed by the package) is the signed area: regions above the axis count positive, below negative. Marked: the integral 9.

```
calculus_ai999x.solve('definite_integral', f='x^2', a=0, b=3)
# -> integral = 9   (SOLVED)
```

```
calculus_ai999x.solve('definite_integral', f='sin(x)', a=0, b=3.141592653589793)
# -> integral = 2   (SOLVED)
```

```
calculus_ai999x.solve('definite_integral', f='exp(2x)', a=0, b=1)
# -> integral = 3.19453   (SOLVED)
```

```
calculus_ai999x.solve('definite_integral', f='1/x', a=1, b=2.718281828459045)
# -> integral = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate the integral of x^2 from 0 to 3.	SOLVED	integral = 9
Find the definite integral of sin(x) from 0 to pi.	SOLVED	integral = 2
Integrate exp(2x) from x = 0 to x = 1.	SOLVED	integral = 3.19453
Compute the integral of 3x^2 + 2x dx from 1 to 2.	SOLVED	integral = 10
Evaluate the integral of 1/x from -1 to 1.	INVALID_INPUT	Definite integral: the function is not continuous (or not real) on the whole interval [-1, 1].

Average value of a function, `average_value`

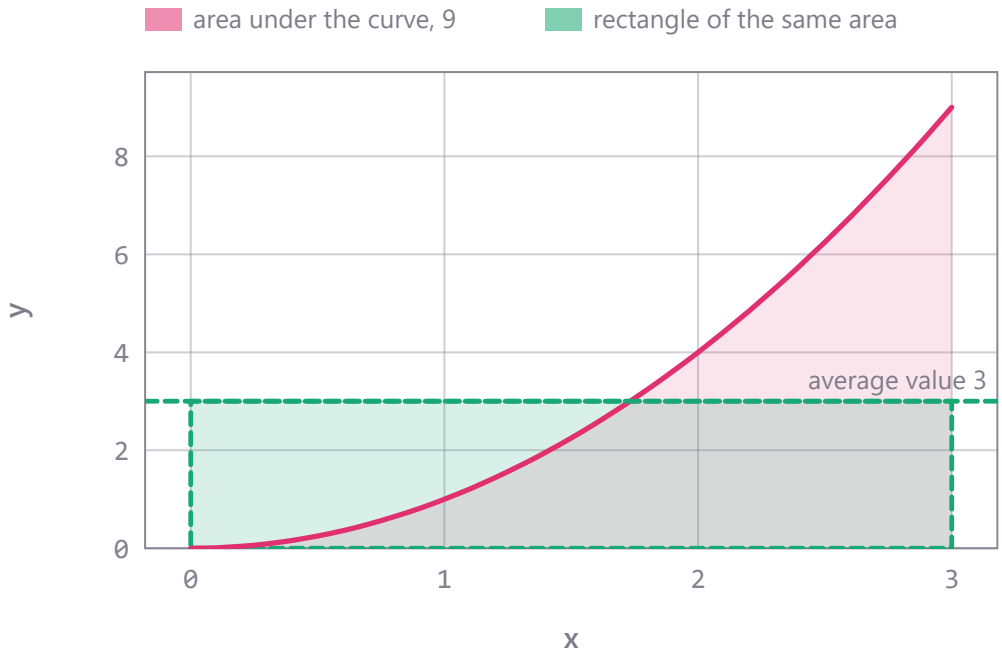
The average value (1/(b - a)) * integral of f from a to b of a function on [a, b].

$$f_{\text{avg}} = \frac{1}{b-a}\int_a^b f(x)\,dx$$

input	meaning	condition
`f`	the function f(x)	
`a`	start of the interval	
`b`	end of the interval	

output	meaning
`average_value`	the average value of f on [a, b]
`integral`	the integral of f over [a, b]

- **Method:** definite integral by high-precision quadrature, divided by the interval length
- **Assumes:** $a < b$; f continuous on [a, b]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y = x^2 on [0, 3] (curve values computed by the package): its integral 9 equals the area of a rectangle of width 3 and height the average value 3. Marked: the average value.

```
calculus_ai999x.solve('average_value', f='x^2', a=0, b=3)
# -> average_value = 3, integral = 9   (SOLVED)
```

```
calculus_ai999x.solve('average_value', f='sin(x)', a=0, b=3.141592653589793)
# -> average_value = 0.63662, integral = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the average value of f(x) = x^2 on [0, 3].	SOLVED	average_value = 3, integral = 9
What is the average value of sin(x) from 0 to pi?	SOLVED	average_value = 0.63662, integral = 2
Find the average value of x^2 on [3, 0].	INVALID_INPUT	Average value of a function: the interval needs a < b.

Area between two curves, `area_between_curves`

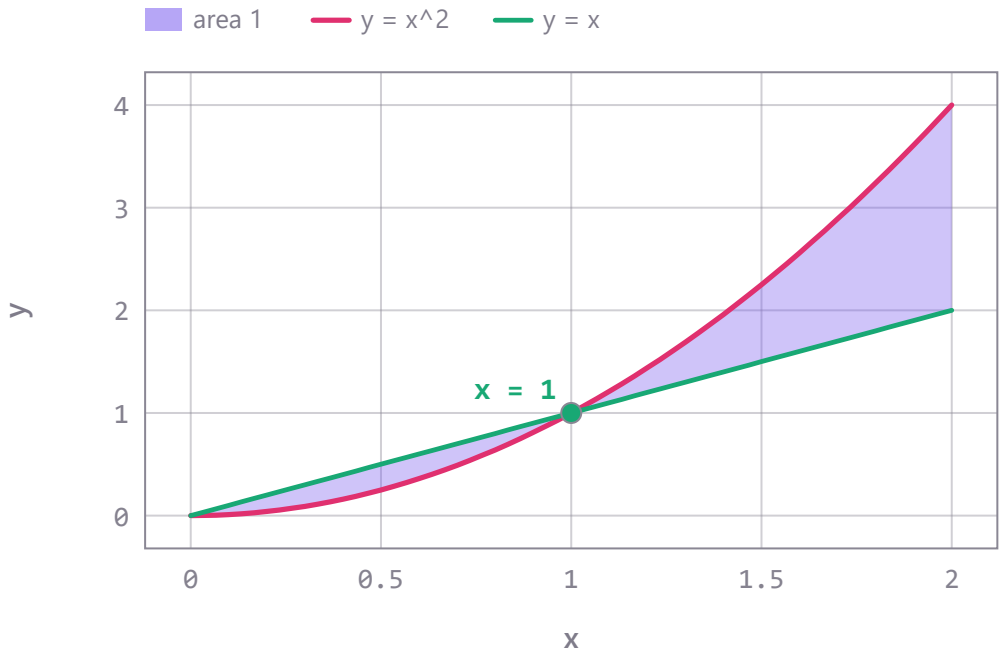
The area between the graphs of two polynomials $y = f(x)$ and $y = g(x)$ over [a, b], counting every piece as positive (the curves may cross).

$$A = \int_a^b |f(x) - g(x)|\,dx$$

input	meaning	condition
`f`	the first curve f(x) (a polynomial)	
`g`	the second curve g(x) (a polynomial)	
`a`	left end	
`b`	right end	

output	meaning
`area`	the area between the curves
`net_integral`	integral of f - g (signed)
`crossings`	x where the curves cross inside (a, b)

- **Method:** exact integration of |f - g|, split at the exact real roots of f - g
- **Assumes:** $a < b$; f and g polynomials with rational coefficients
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The region between y = x^2 and y = x on [0, 2] (curve values computed by the package): the area, the integral of |f − g|, is 1, while the net integral of f − g is 0.666667, because the curves cross. Marked: the crossing(s) at x = 1.

```
calculus_ai999x.solve('area_between_curves', f='x^2', g='x', a=0, b=2)
# -> area = 1, net_integral = 0.666667, crossings = [1.0]   (SOLVED)
```

```
calculus_ai999x.solve('area_between_curves', f='4 - x^2', g='x + 2', a=-2, b=1)
# -> area = 4.5, net_integral = 4.5, crossings = []   (SOLVED)
```

```
calculus_ai999x.solve('area_between_curves', f='x^3', g='0', a=-1, b=1)
# -> area = 0.5, net_integral = 0, crossings = [0.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area between y = x^2 and y = x from x = 0 to x = 2.	SOLVED	area = 1, net_integral = 0.666667, crossings = [1.0]
Find the area between f(x) = 4 - x^2 and g(x) = x + 2 on [-2, 1].	SOLVED	area = 4.5, net_integral = 4.5, crossings = []
Find the area between y = sin(x) and y = 0 from x = 0 to x = pi.	INVALID_INPUT	Area between two curves: this capability needs a polynomial.

Area enclosed by two curves, `area_enclosed_curves`

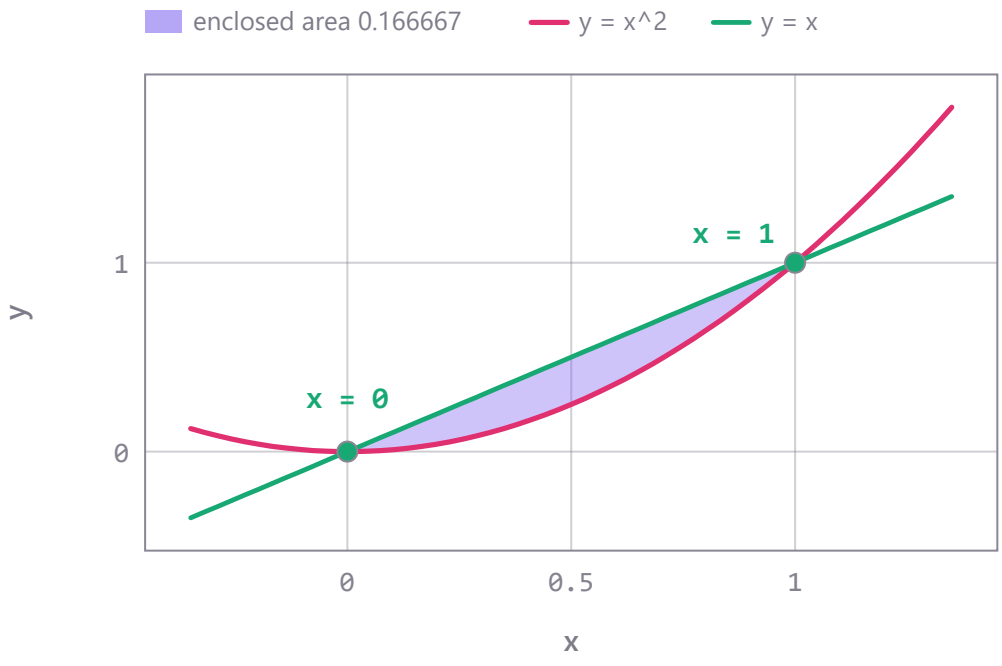
The area of the region enclosed by two polynomial curves $y = f(x)$ and $y = g(x)$, between their first and last intersection.

$$A = \int_{x_1}^{x_n} |f(x) - g(x)|\, dx$$

input	meaning	condition
`f`	the first curve f(x) (a polynomial)	
`g`	the second curve g(x) (a polynomial)	

output	meaning
`area`	the enclosed area
`x_left`	first intersection
`x_right`	last intersection
`intersections`	all intersection x-values

- **Method:** exact real roots of f - g, then exact integration of |f - g| between consecutive roots
- **Assumes:** f and g polynomials with rational coefficients meeting in at least two points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The region enclosed by y = x^2 and y = x (curve values computed by the package): the curves meet at x = 0, 1, and the enclosed area between x = 0 and x = 1 is 0.166667. Marked: the intersections.

```
calculus_ai999x.solve('area_enclosed_curves', f='x^2', g='x')
# -> area = 0.166667, x_left = 0, x_right = 1, intersections = [0.0, 1.0]   (SOLVED)
```

```
calculus_ai999x.solve('area_enclosed_curves', f='4 - x^2', g='0')
# -> area = 10.6667, x_left = -2, x_right = 2, intersections = [-2.0, 2.0]   (SOLVED)
```

```
calculus_ai999x.solve('area_enclosed_curves', f='x^3', g='x')
# -> area = 0.5, x_left = -1, x_right = 1, intersections = [-1.0, 0.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area enclosed by y = x^2 and y = x.	SOLVED	area = 0.166667, x_left = 0, x_right = 1, intersections = [0.0, 1.0]
Find the area of the region bounded by y = 4 - x^2 and y = 0.	SOLVED	area = 10.6667, x_left = -2, x_right = 2, intersections = [-2.0, 2.0]
Find the area enclosed by the curves y = x^3 and y = x.	SOLVED	area = 0.5, x_left = -1, x_right = 1, intersections = [-1.0, 0.0, 1.0]
Find the area enclosed by y = x^2 + 1 and y = 0.	INVALID_INPUT	Area enclosed by two curves: the curves meet in fewer than two points, so they enclose no bounded region.

Volume of revolution (disk method), `disk_volume`

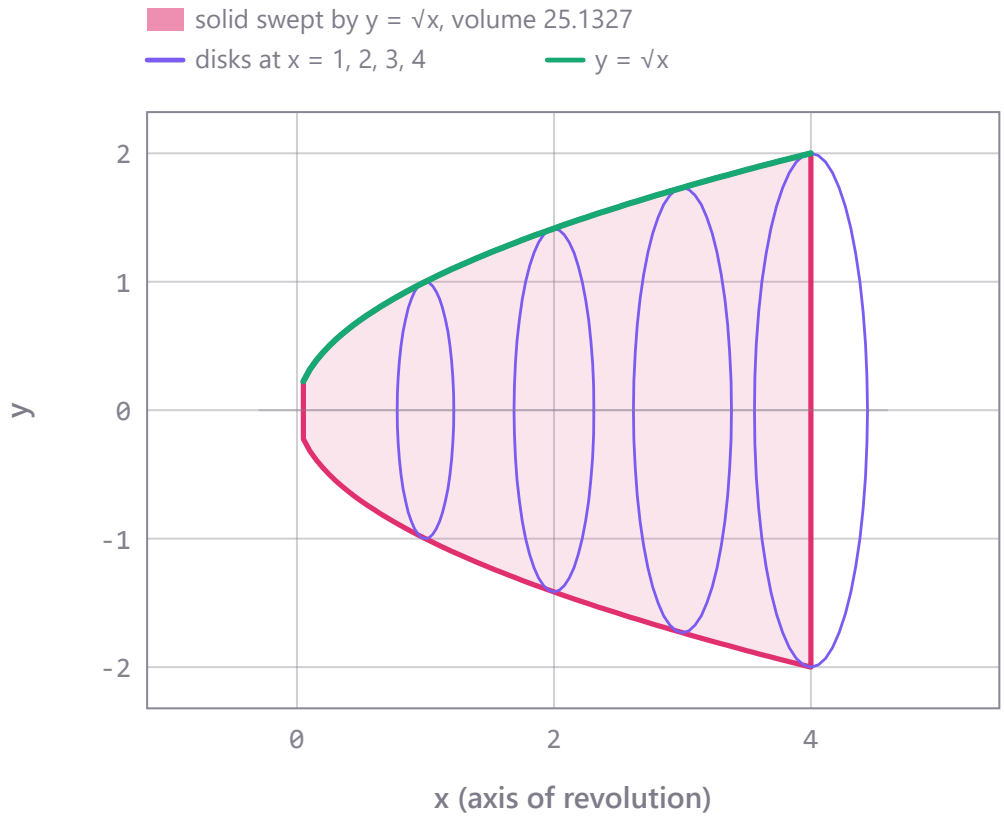
The volume of the solid formed by rotating $y = f$(x), $a \le x \le b$, about the x-axis.

$$V = \pi \int_a^b f(x)^2 \, dx$$

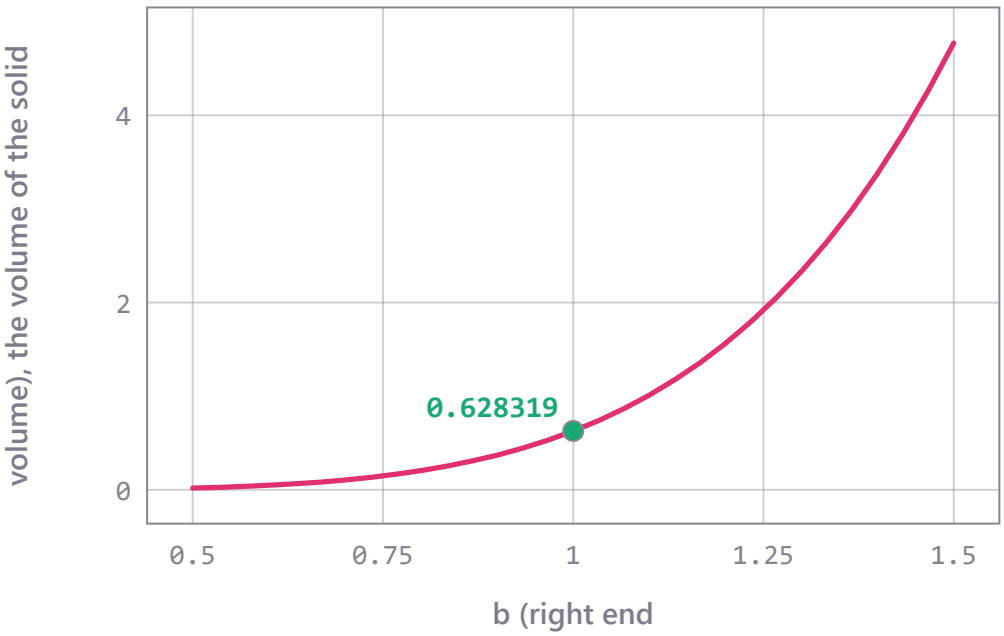
input	meaning	condition
`f`	the curve f(x)	
`a`	left end	
`b`	right end	

output	meaning
`volume`	the volume of the solid

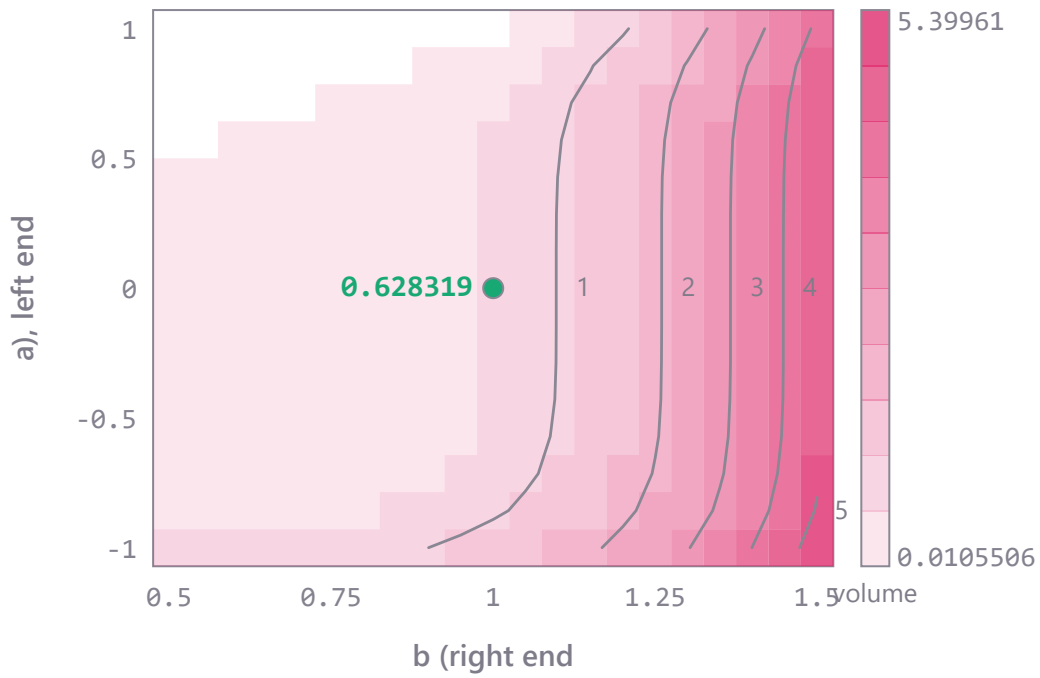
- **Method:** disk method: pi times the integral of f^2, by high-precision quadrature
- **Assumes:** $a < b$; f continuous on [a, b]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The solid made by turning y = √x (0 ≤ x ≤ 4) about the x-axis, with circular cross-sections drawn at x = 1, 2, 3, 4 (profile and radii computed by the package); the package's volume by disks, the integral of π(√x)² from 0 to 4, is 25.1327.



volume (the volume of the solid) against b (right end) from 0.5 to 1.5. Other inputs as in the worked example: f = x^2, a = 0. 31 points computed by calculus-ai999x; marked: the worked example, b = 1, volume = 0.628319.



volume (the volume of the solid) over b (right end) from 0.5 to 1.5 and a (left end) from -1 to 1: each of the 21 × 15 cells is one run of calculus-ai999x, shaded by its value (light 0.0105506 to dark 5.39961); iso-lines at volume = 1, 2, 3, 4, 5; blank cells: inputs the capability refuses. Other inputs as in the worked example: f = x^2. Marked: the worked example, b = 1, a = 0, volume = 0.628319.

```
calculus_ai999x.solve('disk_volume', f='x^2', a=0, b=1)
# -> volume = 0.628319   (SOLVED)
```

```
calculus_ai999x.solve('disk_volume', f='sqrt(x)', a=0, b=4)
# -> volume = 25.1327   (SOLVED)
```

```
calculus_ai999x.solve('disk_volume', f='sqrt(4 - x^2)', a=-2, b=2)
# -> volume = 33.5103   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume when y = x^2 from x = 0 to x = 1 is rotated about the x-axis.	SOLVED	volume = 0.628319
Find the volume of revolution of y = sqrt(x) about the x-axis on [0, 4].	SOLVED	volume = 25.1327
Find the volume of the solid obtained by rotating y = sqrt(4 - x^2) about the x-axis from x = -2 to x = 2.	SOLVED	volume = 33.5103

Volume of revolution (washer method), `washer_volume`

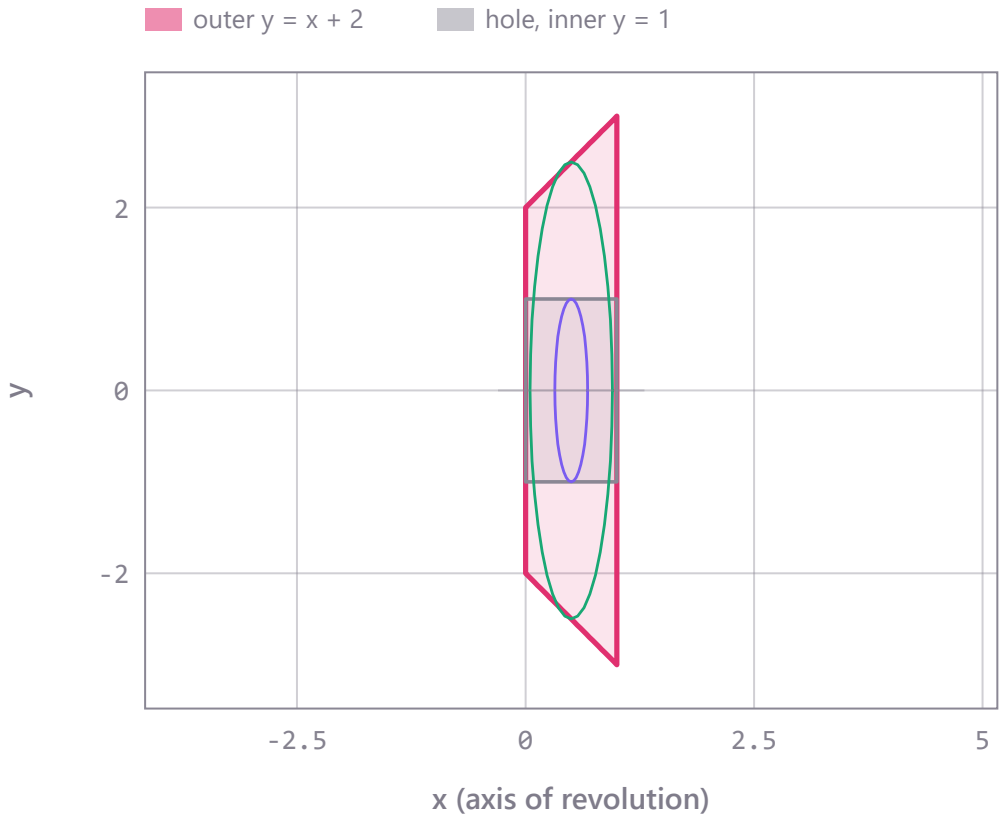
The volume swept when the region between an outer curve $y = f(x)$ and an inner curve $y = g(x)$ (polynomials, f >= g >= 0) on [a, b] is rotated about the x-axis.

$$V = \pi \int_a^b \left(f(x)^2 - g(x)^2 \right) dx$$

input	meaning	condition
`f`	outer radius f(x) (a polynomial)	
`g`	inner radius g(x) (a polynomial)	
`a`	left end	
`b`	right end	

output	meaning
`volume`	the volume of the solid

- **Method:** washer method, integrated exactly
- **Assumes:** $a < b$; $f \geq g \geq 0$ on [a, b] (checked exactly); polynomials
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The solid made by turning the region between y = x + 2 (outer) and y = 1 (inner) about the x-axis for 0 ≤ x ≤ 1 (profiles computed by the package), with the washer at x = 0.5 sketched (radii 2.5 and 1): volume 16.7552 from the package.

```
calculus_ai999x.solve('washer_volume', f='x + 2', g='1', a=0, b=1)
# -> volume = 16.7552   (SOLVED)
```

```
calculus_ai999x.solve('washer_volume', f='4 - x^2', g='1', a=0, b=1)
# -> volume = 39.3746   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the volume when the region between y = x + 2 and y = 1 on [0, 1] is rotated about the x-axis using	SOLVED	volume = 16.7552

question	outcome	result
washers.		
Use the washer method for outer radius 4 - x^2 and inner radius 1 on [0, 1].	SOLVED	volume = 39.3746
Use the washer method for outer radius 1 and inner radius x^2 + 2 on [0, 1].	INVALID_INPUT	Volume of revolution (washer method): the washer method needs outer radius f >= inner radius g >= 0 on [a, b].

Volume of revolution (shell method), `shell_volume`

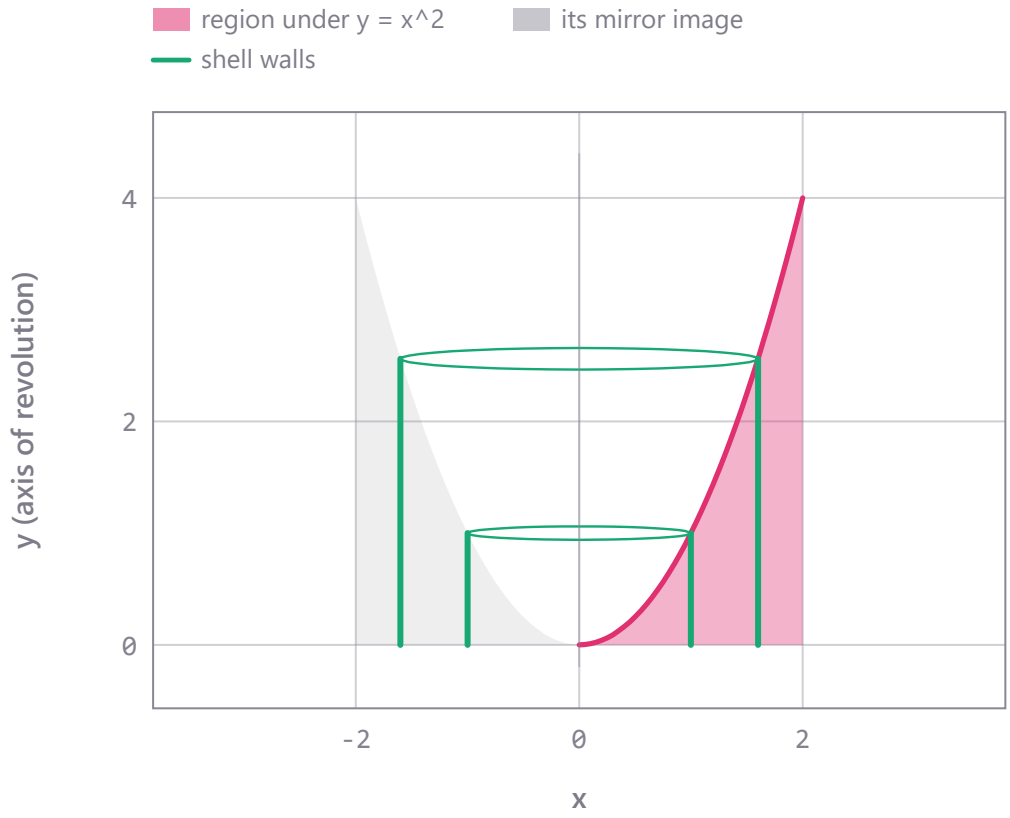
The volume swept when the region under $y = f(x) >= 0$ (a polynomial), $a \le x \le b$ with $a \ge 0$, is rotated about the y-axis.

$$V = 2\pi \int_a^b x\, f(x)\, dx$$

input	meaning	condition
`f`	the curve f(x) (a polynomial)	
`a`	left end (>= 0)	
`b`	right end	

output	meaning
`volume`	the volume of the solid

- **Method:** cylindrical shells, integrated exactly
- **Assumes:** $0 \le a < b$; f(x) >= 0 on [a, b] (checked exactly)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Cylindrical shells: the region under y = x^2 for 0 ≤ x ≤ 2 turned about the y-axis (curve values computed by the package), two shells of radius x and height f(x) sketched: volume 25.1327 from the package, the integral of 2πx f(x) dx.

```
calculus_ai999x.solve('shell_volume', f='x^2', a=0, b=2)
# -> volume = 25.1327   (SOLVED)
```

```
calculus_ai999x.solve('shell_volume', f='4 - x^2', a=0, b=2)
# -> volume = 25.1327   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use the shell method to find the volume when y = x^2 on [0, 2] is rotated about the y-axis.	SOLVED	volume = 25.1327
Find the volume of the solid formed by rotating the region under y = 4 - x^2 from x = 0 to x = 2 about the y-axis.	SOLVED	volume = 25.1327
Use the shell method to find the volume when y = x - 3 on [0, 2] is rotated about the y-axis.	INVALID_INPUT	Volume of revolution (shell method): the shell method needs f(x) >= 0 on [a, b].

Arc length of a graph, `arc_length`

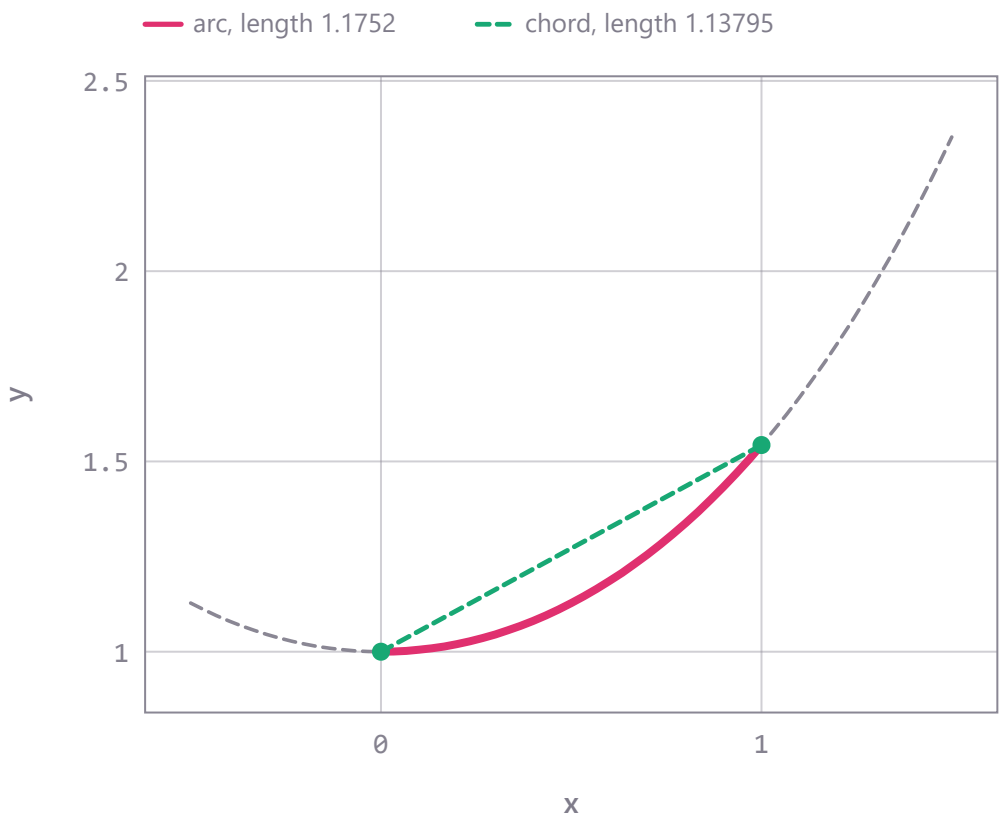
The length of the curve $y = f(x)$ from $x = a$ to $x = b$.

$$L = \int_a^b \sqrt{1 + f'(x)^2}\, dx$$

input	meaning	condition
`f`	the curve f(x)	
`a`	left end	
`b`	right end	

output	meaning
`arc_length`	the length of the curve

- **Method:** arc-length integral of sqrt(1 + f'(x)^2) with the exact derivative, by high-precision quadrature
- **Assumes:** $a < b$; f' continuous on [a, b]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The arc of y = cosh(x) from x = 0 to x = 1, to scale (curve values computed by the package): arc length 1.1752 from the package, against the straight chord between the same end points (length derived from the two returned values).

```
calculus_ai999x.solve('arc_length', f='cosh(x)', a=0, b=1)
# -> arc_length = 1.1752   (SOLVED)
```

```
calculus_ai999x.solve('arc_length', f='2x + 1', a=0, b=3)
# -> arc_length = 6.7082   (SOLVED)
```

```
calculus_ai999x.solve('arc_length', f='x^2', a=0, b=1)
# -> arc_length = 1.47894   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the arc length of y = x^2 from x = 0 to x = 1.	SOLVED	arc_length = 1.47894
What is the length of the curve y = cosh(x) on [0, 1]?	SOLVED	arc_length = 1.1752
Find the arc length of y = sqrt(x) from x = -1 to x = 1.	INVALID_INPUT	Arc length of a graph: the function is not continuous (or not real) on the whole interval [-1, 1].

Surface area of revolution, `surface_area_revolution`

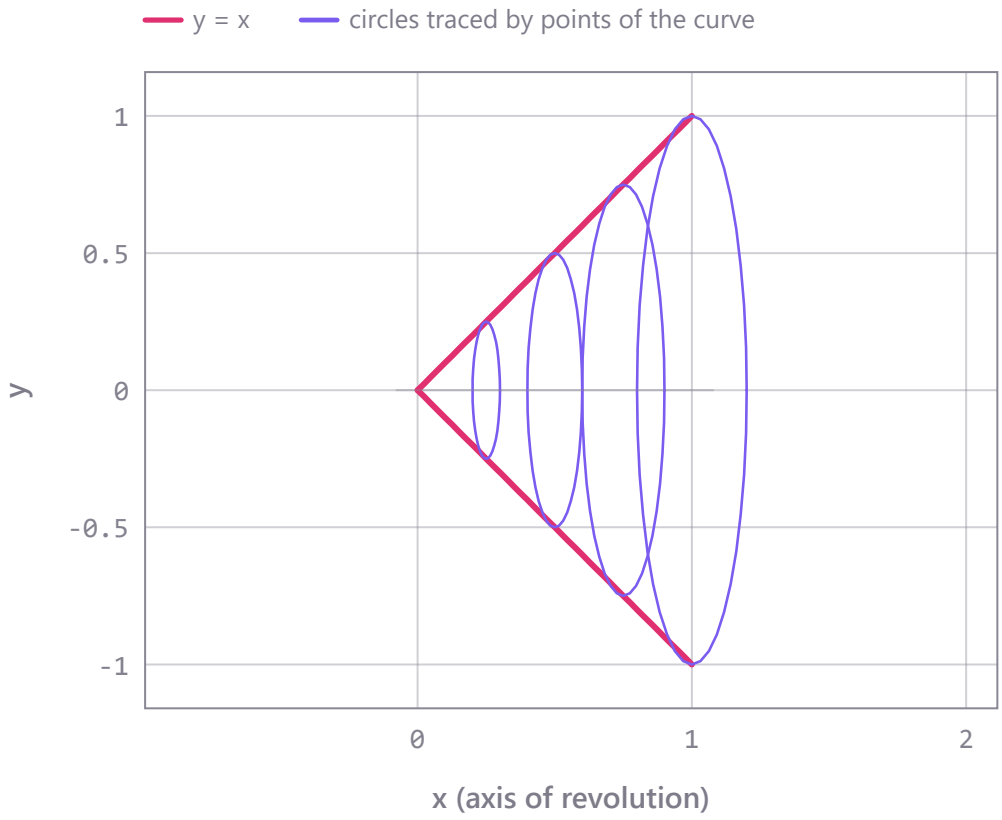
The area of the surface formed by rotating $y = f$(x), $a \le x \le b$, about the x-axis.

$$S = 2\pi \int_a^b |f(x)| \sqrt{1 + f'(x)^2}\, dx$$

input	meaning	condition
`f`	the curve f(x)	
`a`	left end	
`b`	right end	

output	meaning
`surface_area`	the area of the surface of revolution

- **Method:** $2pi$ times the integral of |f| * sqrt($1 + f$'^2) (split where f changes sign), high-precision quadrature
- **Assumes:** $a < b$; f' continuous on [a, b]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The surface traced by y = x for 0 ≤ x ≤ 1 turned about the x-axis (the worked example; profile computed by the package at 41 points, four of the circles its points trace sketched): surface area 4.44288 from the package, 2π times the integral of |f| √(1 + f′²) dx.

```
calculus_ai999x.solve('surface_area_revolution', f='x', a=0, b=1)
# -> surface_area = 4.44288   (SOLVED)
```

```
calculus_ai999x.solve('surface_area_revolution', f='sqrt(4 - x^2)', a=-1, b=1)
# -> surface_area = 25.1327   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the surface area when y = x from x = 0 to x = 1 is rotated about the x-axis.	SOLVED	surface_area = 4.44288
Find the area of the surface of revolution of y = sqrt(4 - x^2) about the x-axis on [-1, 1].	SOLVED	surface_area = 25.1327

Centroid of a region under a curve, `centroid_region`

The area and centroid (x_bar, y_bar) of the region between $y = f(x) >= 0$ (a polynomial) and the x-axis for $a \le x \le b$.

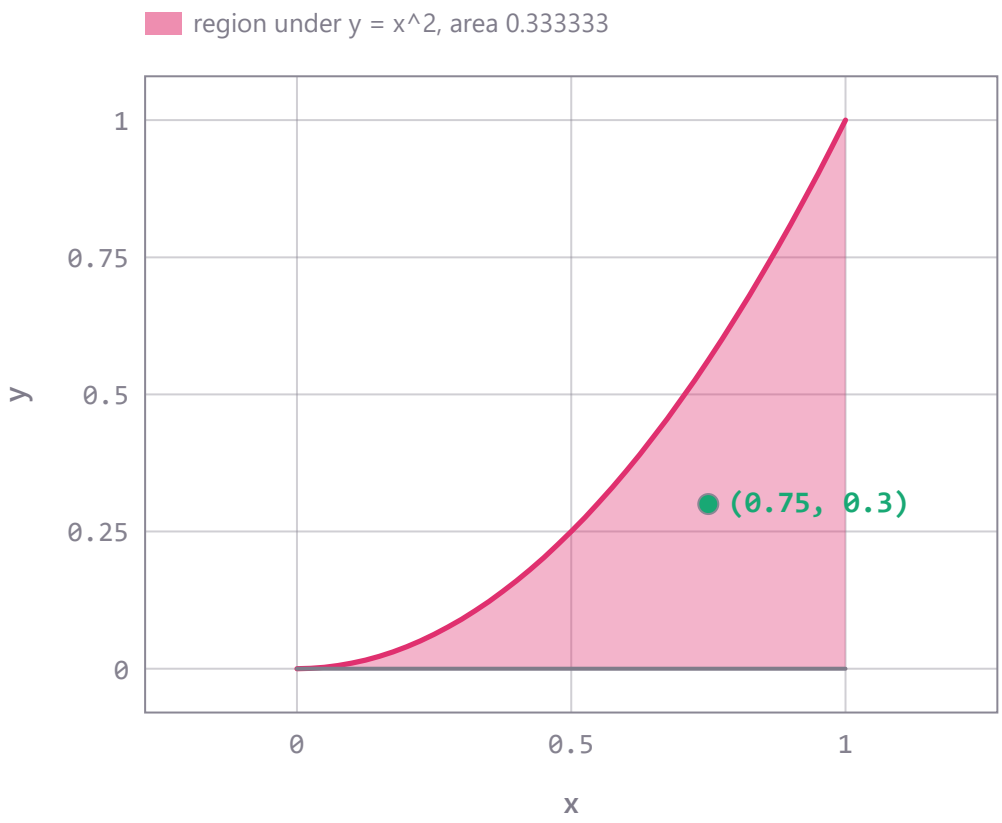
$$\bar{x} = \frac{1}{A}\int_a^b x f(x)\,dx, \quad \bar{y} = \frac{1}{2A}\int_a^b f(x)^2\,dx$$

input	meaning	condition
`f`	the curve f(x) (a polynomial)	
`a`	left end	

input	meaning	condition
`b`	right end	

output	meaning
`area`	area of the region
`x_bar`	x-coordinate of the centroid
`y_bar`	y-coordinate of the centroid

- **Method:** exact moments of the region: x_bar = (1/A) * integral of x f, y_bar = (1/2A) * integral of f^2
- **Assumes:** $a < b$; f(x) >= 0 on [a, b] (checked exactly)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The region under y = x^2 between x = 0 and x = 1, to scale (curve values computed by the package): area 0.333333. Marked: its centroid (x̄, ȳ) = (0.75, 0.3).

```
calculus_ai999x.solve('centroid_region', f='x^2', a=0, b=1)
# -> area = 0.333333, x_bar = 0.75, y_bar = 0.3   (SOLVED)
```

```
calculus_ai999x.solve('centroid_region', f='4 - x^2', a=-2, b=2)
# -> area = 10.6667, x_bar = 0, y_bar = 1.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the centroid of the region under y = x^2 from x = 0 to x = 1.	SOLVED	area = 0.333333, x_bar = 0.75, y_bar = 0.3

question	outcome	result
Find the centroid of the region bounded by y = 4 - x^2 and the x-axis on [-2, 2].	SOLVED	area = 10.6667, x_bar = 0, y_bar = 1.6

Improper integral to infinity, `improper_integral`

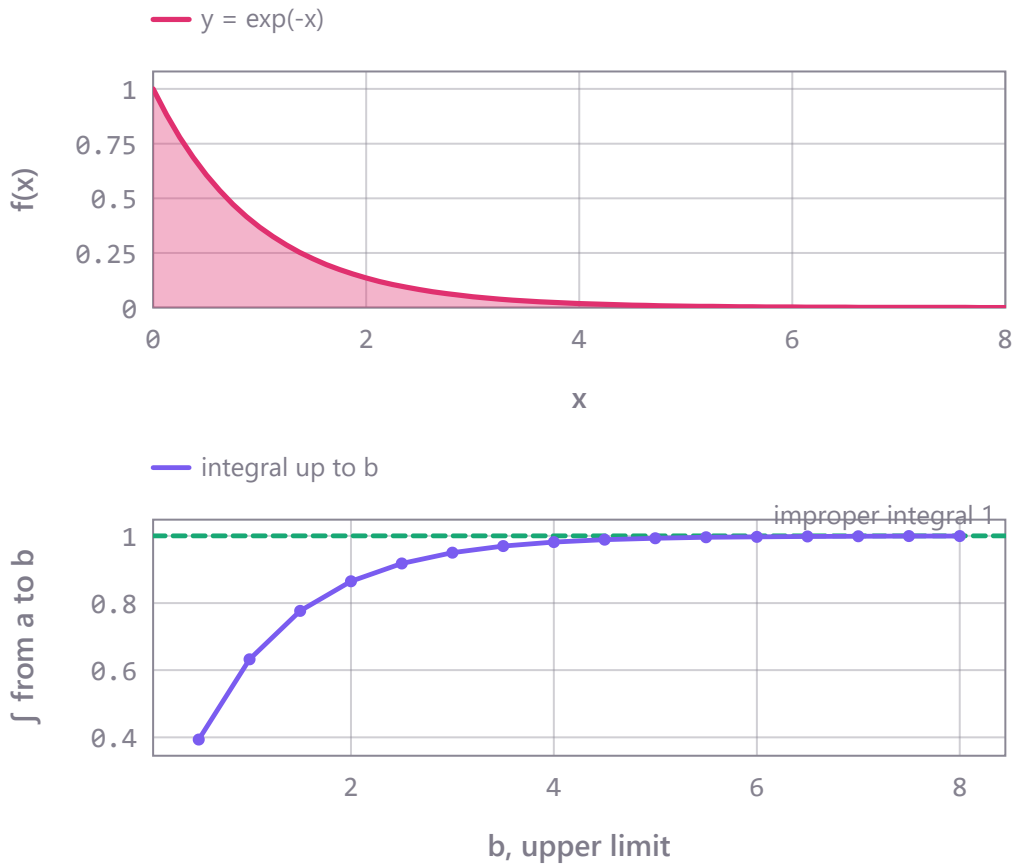
The improper integral of f(x) from $x = a$ to infinity, when it converges.

$$\int_a^\infty f(x)\,dx = \lim_{R\to\infty}\int_a^R f(x)\,dx$$

input	meaning	condition
`f`	the integrand f(x)	
`a`	lower limit	

output	meaning
`integral`	the value of the improper integral

- **Method:** convergence by a limit comparison with $1/x^{1.01}$ (exact limit), value by tanh-sinh quadrature on [a, infinity)
- **Assumes:** f continuous on [a, infinity); $x^{1.01}f$(x) has a finite limit (comparison test), so the integral converges absolutely
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: y = exp(-x) from x = 0 (values computed by the package), the area extending without end. Bottom: the integral from 0 to b for b up to 8, each computed by the package, approaching the improper integral 1 (marked).

```
calculus_ai999x.solve('improper_integral', f='exp(-x)', a=0)
# -> integral = 1   (SOLVED)
```

```
calculus_ai999x.solve('improper_integral', f='1/x^2', a=1)
# -> integral = 1   (SOLVED)
```

```
calculus_ai999x.solve('improper_integral', f='1/(x^2 + 1)', a=0)
# -> integral = 1.5708   (SOLVED)
```

```
calculus_ai999x.solve('improper_integral', f='exp(-x^2)', a=0)
# -> integral = 0.886227   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate the improper integral of exp(-x) from 0 to infinity.	SOLVED	integral = 1
Find the integral of 1/x^2 from 1 to infinity.	SOLVED	integral = 1
Find the integral of 1/x from 1 to infinity.	INVALID_INPUT	Improper integral to infinity: convergence cannot be established (the integrand does not decay faster than 1/x^1.01).
Evaluate the integral of sin(x) from 0 to infinity.	INVALID_INPUT	Improper integral to infinity: convergence cannot be established (the integrand does not decay faster than 1/x^1.01).

Left Riemann sum, `left_riemann_sum`

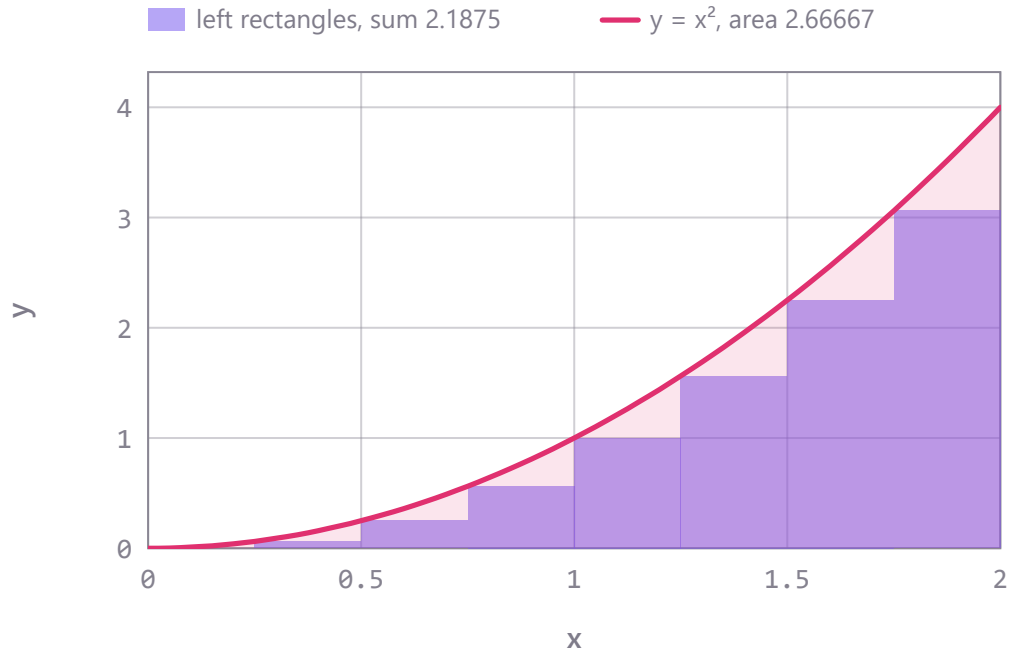
The left-endpoint Riemann sum of f on [a, b] with n equal subintervals.

$$L_n = h\sum_{k=0}^{n-1} f(a+kh), \quad h = \frac{b-a}{n}$$

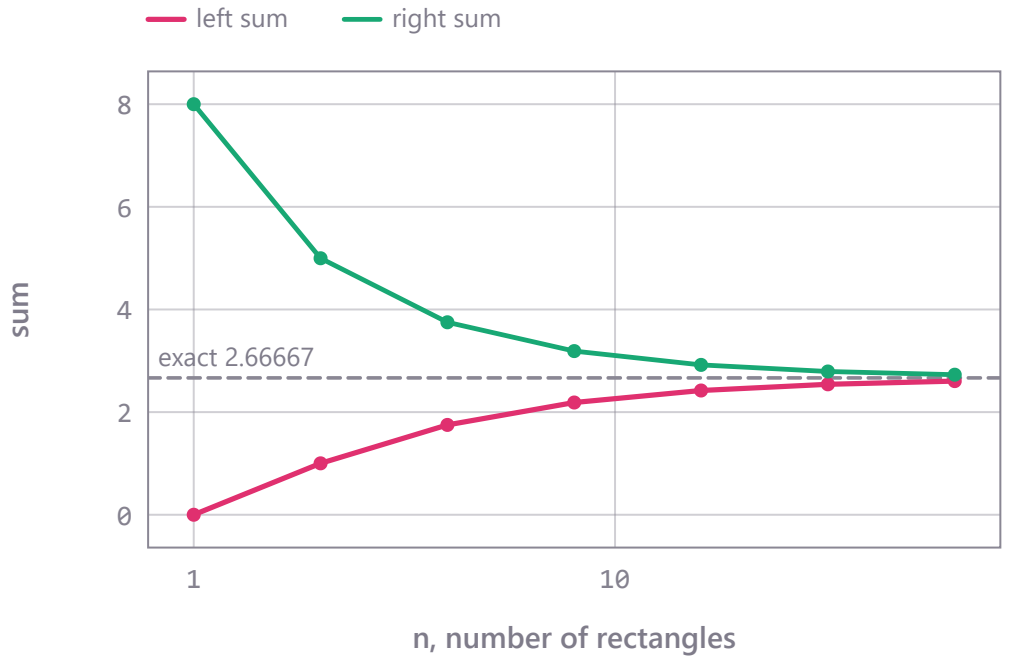
input	meaning	condition
`f`	the function f(x)	
`a`	left end	
`b`	right end	
`n`	number of equal subintervals	

output	meaning
`approximation`	the left Riemann sum

- **Method:** h times the sum of f at the left endpoints $x_k = a + k$ h, $k = 0..n-1$ $(h = (b-a)/n)$
- **Assumes:** $a < b$; $1 \le n \le 100000$; f defined at the sample points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The area under $y = x^2$ on [0, 2] and the left Riemann sum with 8 rectangles (heights $f(x_i)$ and the curve evaluated by the package): left sum 2.1875, right sum 3.1875, exact integral 2.66667.



Left and right Riemann sums of y = x^2 on [0, 2] for n = 1 … 64 rectangles, each computed by the package: for an increasing function the left sum lies below and the right sum above the exact integral 2.66667; at n = 64 they are 2.60449 and 2.72949.

```
calculus_ai999x.solve('left_riemann_sum', f='x^2', a=0, b=2, n=4)
# -> approximation = 1.75   (SOLVED)
```

```
calculus_ai999x.solve('left_riemann_sum', f='x^3', a=0, b=1, n=4)
# -> approximation = 0.140625   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the left Riemann sum of x^2 on [0, 2] with n = 4.	SOLVED	approximation = 1.75

question	outcome	result
Approximate the integral of x^3 from 0 to 1 using a left Riemann sum with 4 subintervals.	SOLVED	approximation = 0.140625
Find the left Riemann sum of x^2 on [0, 2] with n = 0.	INVALID_INPUT	n: must be at least 1, got 0.

Right Riemann sum, `right_riemann_sum`

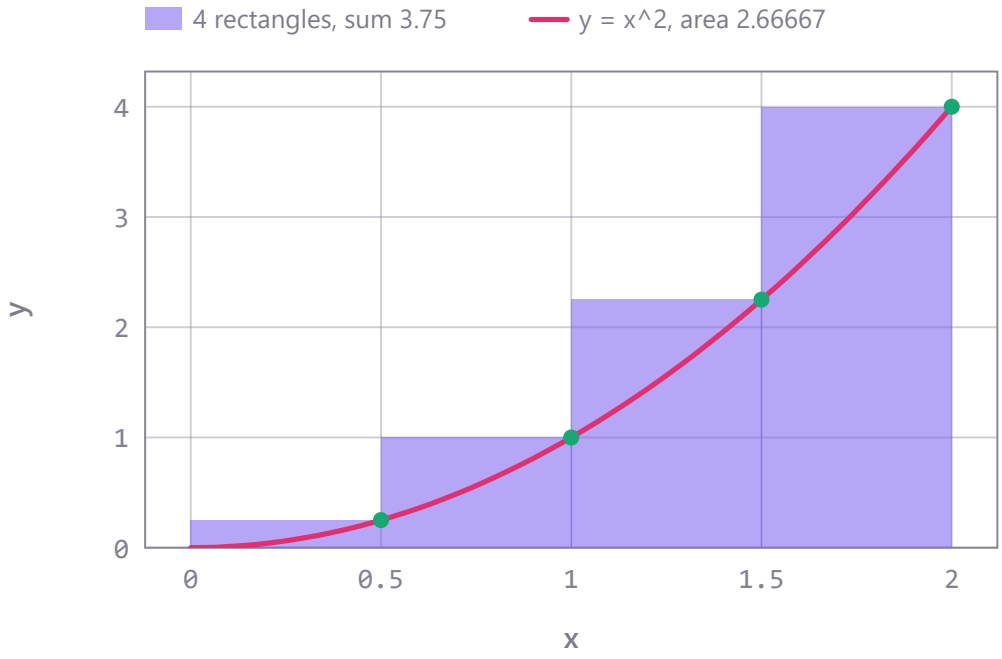
The right-endpoint Riemann sum of f on [a, b] with n equal subintervals.

$$R_n = h\sum_{k=1}^{n} f(a+kh), \quad h = \frac{b-a}{n}$$

input	meaning	condition
`f`	the function f(x)	
`a`	left end	
`b`	right end	
`n`	number of equal subintervals	

output	meaning
`approximation`	the right Riemann sum

- **Method:** h times the sum of f at the right endpoints $x_k = a + k$ h, $k = 1..n$
- **Assumes:** $a < b$; $1 \le n \le 100000$; f defined at the sample points
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y = x^2 on [0, 2] and the 4 rectangles whose heights are f at the right ends of the sub-intervals (heights and curve computed by the package; dots mark the sample points): sum 3.75, exact integral 2.66667.

```
calculus_ai999x.solve('right_riemann_sum', f='x^2', a=0, b=2, n=4)
# -> approximation = 3.75   (SOLVED)
```

```
calculus_ai999x.solve('right_riemann_sum', f='x^3', a=0, b=1, n=4)
# -> approximation = 0.390625   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the right Riemann sum of x^2 on [0, 2] with n = 4.	SOLVED	approximation = 3.75
Approximate the integral of x^3 from 0 to 1 using a right Riemann sum with 4 subintervals.	SOLVED	approximation = 0.390625

Trapezoidal rule, `trapezoid_rule`

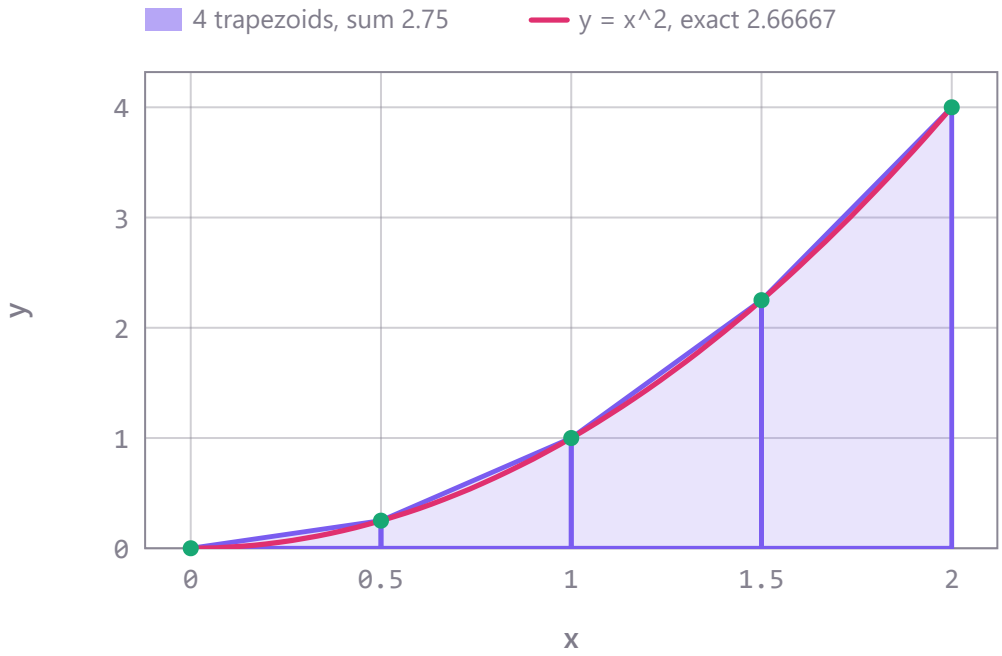
The composite trapezoidal approximation of the integral of f on [a, b] with n subintervals, with the exact value and the error (approximation - exact).

$$T_n = \frac{h}{2}\left[f(x_0) + 2\sum_{k=1}^{n-1} f(x_k) + f(x_n)\right]$$

input	meaning	condition
`f`	the function f(x)	
`a`	left end	
`b`	right end	
`n`	number of equal subintervals	

output	meaning
`approximation`	trapezoidal approximation T_n
`exact`	the integral
`error`	T_n - exact

- **Method:** composite trapezoidal rule; exact value by high-precision quadrature
- **Assumes:** $a < b$; $1 \le n \le 100000$; f continuous on [a, b]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y = x^2 on [0, 2] and the 4 trapezoids through the curve values at the nodes (computed by the package): trapezoid sum 2.75, exact 2.66667, error 0.0833333, the chords lie above a convex curve, so the rule overestimates.

```
calculus_ai999x.solve('trapezoid_rule', f='x^2', a=0, b=2, n=4)
# -> approximation = 2.75, exact = 2.66667, error = 0.0833333   (SOLVED)
```

```
calculus_ai999x.solve('trapezoid_rule', f='1/x', a=1, b=2, n=4)
# -> approximation = 0.697024, exact = 0.693147, error = 0.00387663   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Approximate the integral of x^2 from 0 to 2 using the trapezoidal rule with n = 4.	SOLVED	approximation = 2.75, exact = 2.66667, error = 0.0833333
Use the trapezoid rule with 4 subintervals to estimate the integral of 1/x from 1 to 2.	SOLVED	approximation = 0.697024, exact = 0.693147, error = 0.00387663
Use the trapezoidal rule with 4 subintervals to estimate the integral of 1/x from -1 to 1.	INVALID_INPUT	Trapezoidal rule: the expression is not defined there (ZeroDivisionError).

Simpson's rule, `simpson_rule`

The composite Simpson approximation of the integral of f on [a, b] with an even number n of subintervals, with the exact value and the error.

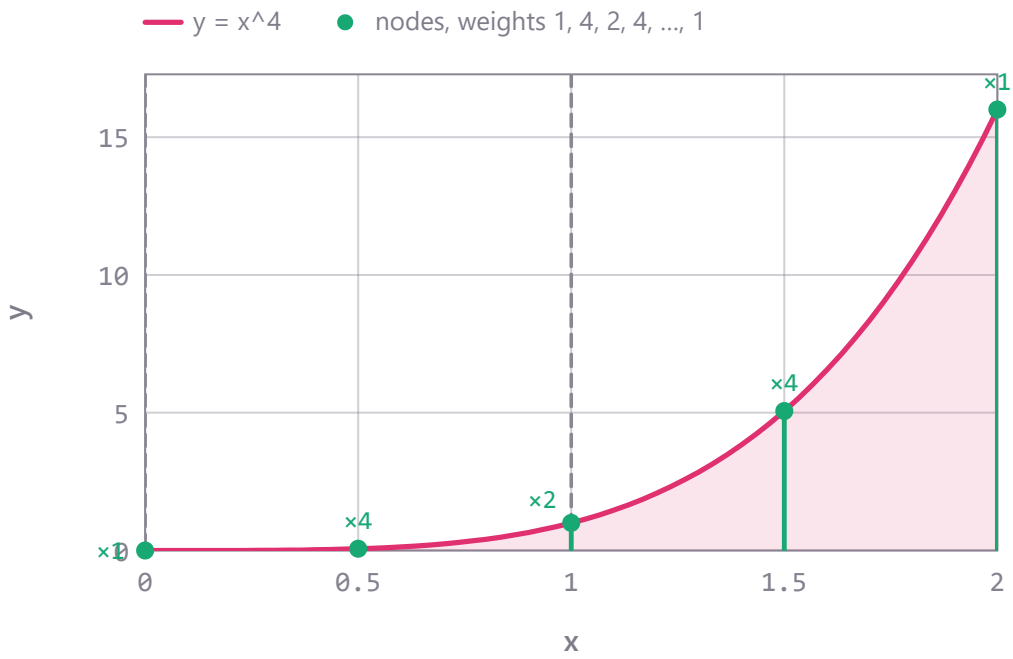
$$S_n = \frac{h}{3}\left[f(x_0) + 4\sum_{\text{odd } k} f(x_k) + 2\sum_{\text{even } k} f(x_k) + f(x_n)\right]$$

input	meaning	condition
`f`	the function f(x)	
`a`	left end	

input	meaning	condition
`b`	right end	
`n`	number of equal subintervals	

output	meaning
`approximation`	Simpson approximation S_n
`exact`	the integral
`error`	S_n - exact

- **Method:** composite Simpson's $1/3$ rule (weights 1, 4, 2, ..., 4, 1); exact value by high-precision quadrature
- **Assumes:** $a < b$; n even, $2 \le n \le 100000$; f continuous on [a, b]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Simpson's rule for y = x^4 on [0, 2] with n = 4: the curve values at the nodes (computed by the package) are weighted 1, 4, 2, 4, ..., 1 (a parabola through each pair of panels, separated by dashed lines). Approximation 6.41667, exact 6.4, error 0.0166667.

```
calculus_ai999x.solve('simpson_rule', f='x^4', a=0, b=2, n=4)
# -> approximation = 6.41667, exact = 6.4, error = 0.0166667   (SOLVED)
```

```
calculus_ai999x.solve('simpson_rule', f='x^3', a=0, b=2, n=2)
# -> approximation = 4, exact = 4, error = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Approximate the integral of x^4 from 0 to 2 using Simpson's rule with 4 subintervals.	SOLVED	approximation = 6.41667, exact = 6.4, error = 0.0166667
Use Simpson's rule with n = 2 to approximate the integral of x^3 from 0 to 2.	SOLVED	approximation = 4, exact = 4, error = 0

question	outcome	result
Use Simpson's rule with n = 3 to approximate the integral of x^3 from 0 to 2.	INVALID_INPUT	Simpson's rule: Simpson's rule needs an even number of subintervals.

Midpoint rule, `midpoint_rule`

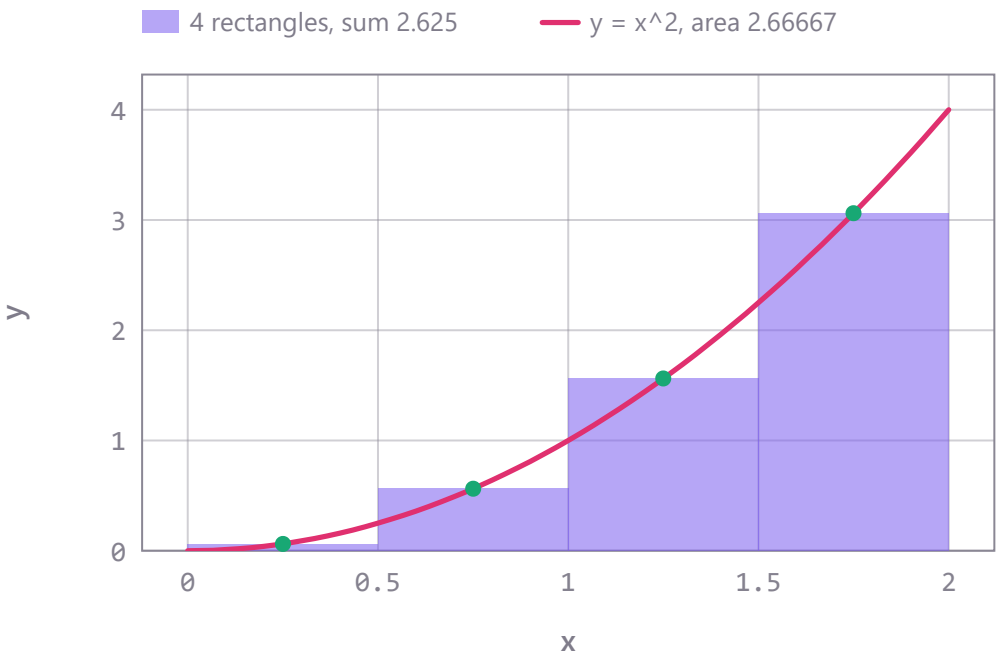
The composite midpoint approximation of the integral of f on [a, b] with n subintervals, with the exact value and the error.

$$M_n = h \sum_{k=0}^{n-1} f\left(a + (k + \tfrac{1}{2})h\right)$$

input	meaning	condition
`f`	the function f(x)	
`a`	left end	
`b`	right end	
`n`	number of equal subintervals	

output	meaning
`approximation`	midpoint approximation M_n
`exact`	the integral
`error`	M_n - exact

- **Method:** composite midpoint rule; exact value by high-precision quadrature
- **Assumes:** $a < b$; $1 \le n \le 100000$; f continuous on [a, b]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y = x^2 on [0, 2] and the 4 rectangles whose heights are f at the midpoints of the sub-intervals (heights and curve computed by the package; dots mark the sample points): sum 2.625, exact integral 2.66667, error -0.0416667.

```
calculus_ai999x.solve('midpoint_rule', f='x^2', a=0, b=2, n=4)
# -> approximation = 2.625, exact = 2.66667, error = -0.0416667   (SOLVED)
```

```
calculus_ai999x.solve('midpoint_rule', f='x^3', a=0, b=2, n=2)
# -> approximation = 3.5, exact = 4, error = -0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Approximate the integral of x^2 from 0 to 2 using the midpoint rule with n = 4.	SOLVED	approximation = 2.625, exact = 2.66667, error = -0.0416667
Use the midpoint rule with 2 subintervals to estimate the integral of x^3 from 0 to 2.	SOLVED	approximation = 3.5, exact = 4, error = -0.5

Gauss-Legendre quadrature, `gauss_legendre`

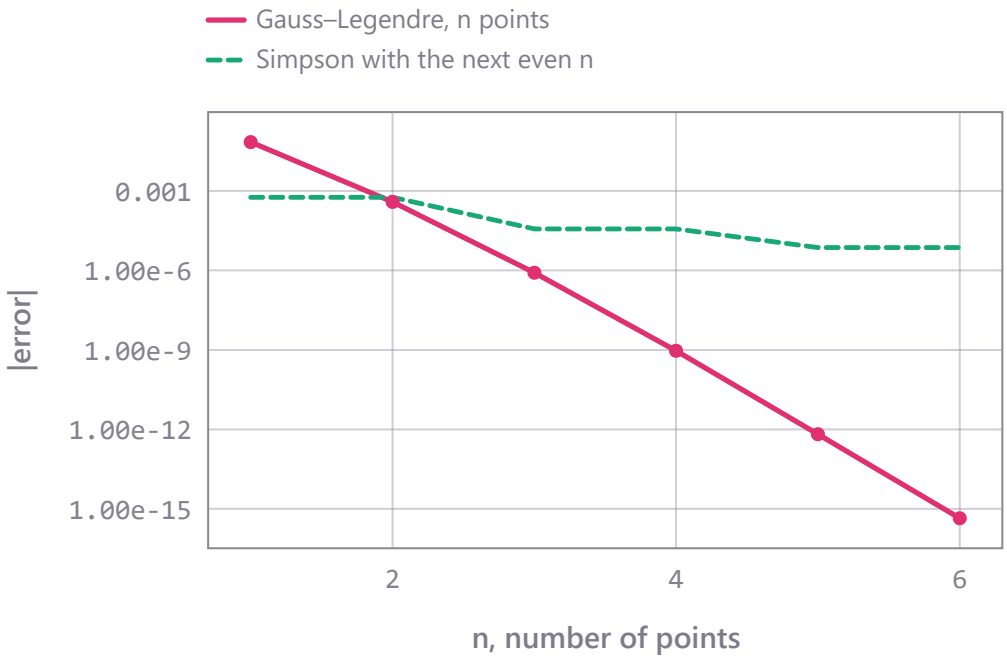
The n-point Gauss-Legendre approximation of the integral of f on [a, b] (exact for polynomials of degree <= 2n - 1), with the exact value and the error.

$$\int_a^b f\,dx \approx \frac{b-a}{2}\sum_{i=1}^{n} w_i\, f\left(\frac{a+b}{2}+\frac{b-a}{2}x_i\right)$$

input	meaning	condition
`f`	the function f(x)	
`a`	left end	
`b`	right end	
`n`	number of nodes	

output	meaning
`approximation`	n-point Gauss-Legendre value
`exact`	the integral
`error`	approximation - exact

- **Method:** nodes = roots of the Legendre polynomial P_n (Newton's method on the three-term recurrence), weights 2/((1 − x^2) P_n'(x)^2), mapped to [a, b]
- **Assumes:** $a < b$; $1 \le n \le 64$; f continuous on [a, b]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Error of n-point Gauss–Legendre quadrature for the integral of e^x from 0 to 1 (chosen), n = 1 … 6, each run of the package, against Simpson's rule (logarithmic error axis): each extra point gains about two orders of magnitude. Worked example ($x^3 + x^2$, 0 to 2, n = 2): 6.66667, exact 6.66667.

```
calculus_ai999x.solve('gauss_legendre', f='x^3 + x^2', a=0, b=2, n=2)
# -> approximation = 6.66667, exact = 6.66667, error = 0   (SOLVED)
```

```
calculus_ai999x.solve('gauss_legendre', f='x^4', a=-1, b=1, n=2)
# -> approximation = 0.222222, exact = 0.4, error = -0.177778   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Use 2-point Gauss-Legendre quadrature to approximate the integral of x^3 + x^2 from 0 to 2.	SOLVED	approximation = 6.66667, exact = 6.66667, error = 0
Use 2-point Gaussian quadrature to approximate the integral of x^4 from -1 to 1.	SOLVED	approximation = 0.222222, exact = 0.4, error = -0.177778

Finite-difference derivative, `numerical_derivative`

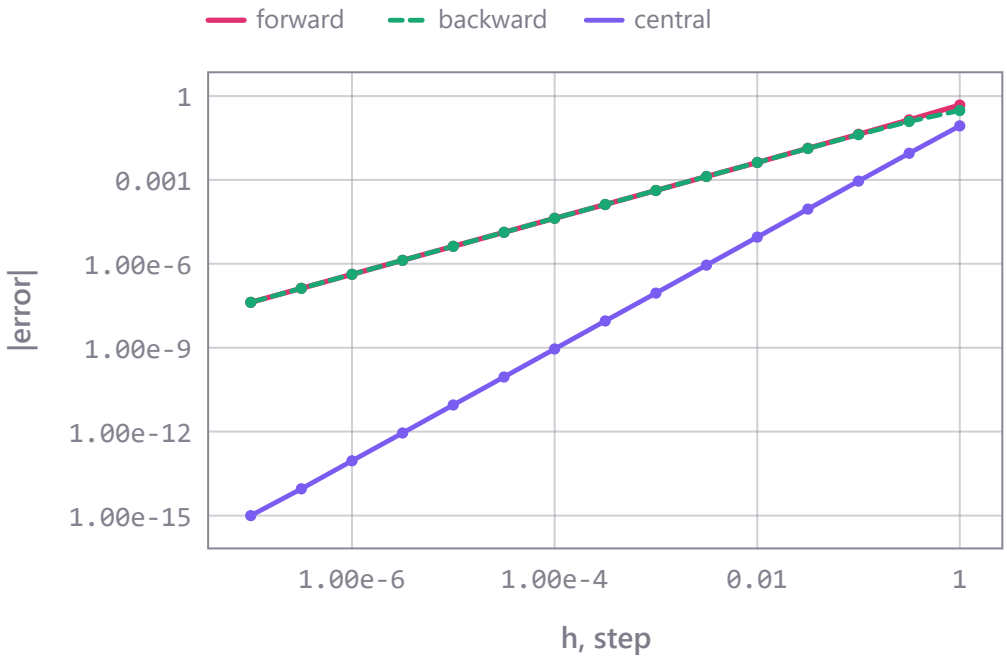
A forward, backward or central difference approximation of f'(a) with step h, with the exact derivative and the error.

$$f'(a) \approx \frac{f(a+h) - f(a-h)}{2h}$$

input	meaning	condition
`f`	the function f(x)	
`a`	the point $x = a$	
`h`	the step size $h > 0$	
`method`	forward, backward or central difference	

output	meaning
`approximation`	the difference quotient
`exact`	the exact f'(a)
`error`	approximation - exact

- **Method:** difference quotients (f(a+h)-f(a))/h, (f(a)-f(a-h))/h, (f(a+h)-f(a-h))/(2h) at 30 digits; exact derivative symbolically
- **Assumes:** $h > 0$; f defined at the points used
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Error of the forward, backward and central difference approximations of the derivative of sin x at x = 1 (chosen) for steps h from 1 down to 1e−7, each run of the package (logarithmic axes): one-sided errors fall like h, the central one like h^2. Worked example (x^2, h = 0.1, central): 2, error 0.

```
calculus_ai999x.solve('numerical_derivative', f='x^2', a=1, h=0.1, method='central')
# -> approximation = 2, exact = 2, error = 0   (SOLVED)
```

```
calculus_ai999x.solve('numerical_derivative', f='x^3', a=1, h=0.5, method='forward')
# -> approximation = 4.75, exact = 3, error = 1.75   (SOLVED)
```

```
calculus_ai999x.solve('numerical_derivative', f='x^3', a=1, h=0.5, method='backward')
# -> approximation = 1.75, exact = 3, error = -1.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Estimate the derivative of x^2 at x = 1 using a central difference with h = 0.1.	SOLVED	approximation = 2, exact = 2, error = 0
Use a forward difference with h = 0.5 to approximate the derivative of x^3 at x = 1.	SOLVED	approximation = 4.75, exact = 3, error = 1.75

question	outcome	result
Use a backward difference with h = 0.5 to approximate the derivative of x^3 at x = 1.	SOLVED	approximation = 1.75, exact = 3, error = -1.25
Use a central difference with h = -0.1 to approximate the derivative of x^3 at x = 1.	INVALID_INPUT	Finite-difference derivative: the step h must be positive.

Newton's method, `newton_method`

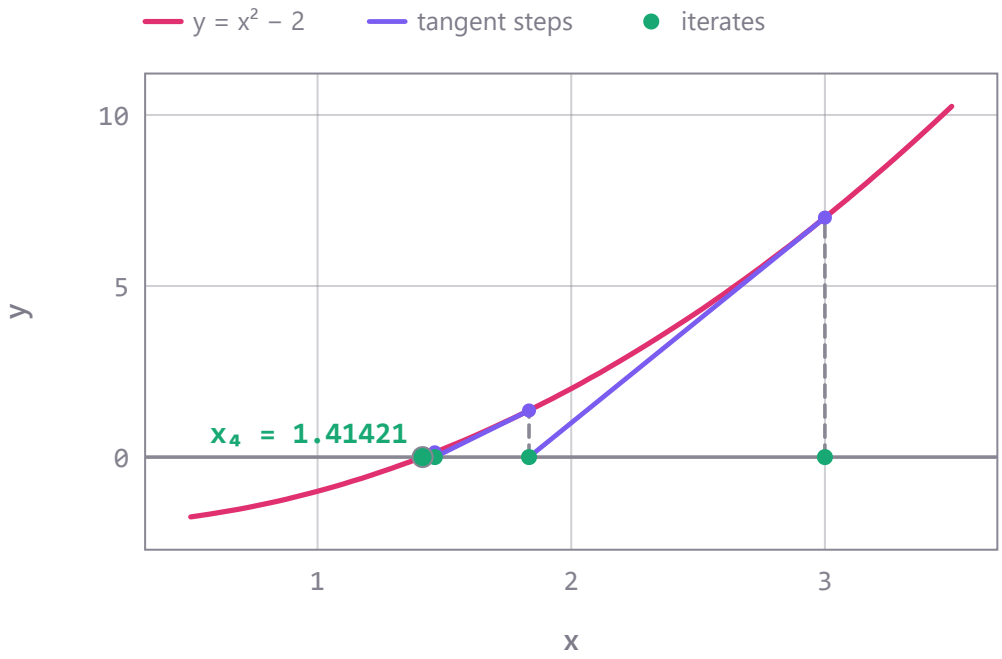
n iterations of Newton's method x_{$k+1$} = x_k - f(x_k)/f'(x_k) for a root of f, from x_0.

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

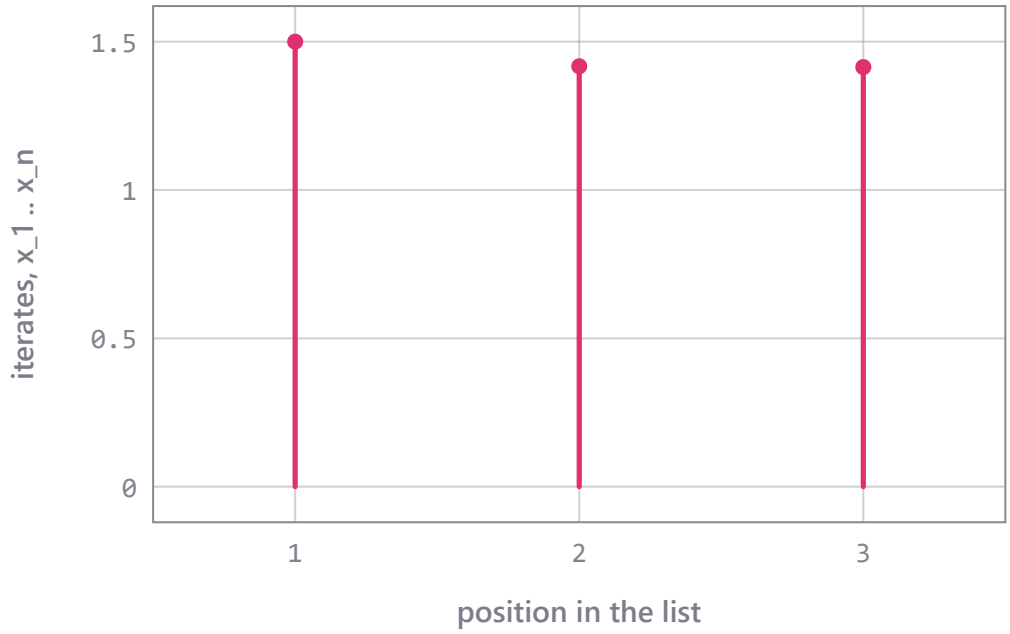
input	meaning	condition
`f`	the function f(x)	
`x0`	the starting value x_0	
`n`	number of iterations	

output	meaning
`x_n`	the n-th iterate
`f_x_n`	f(x_n)
`iterates`	x_1 .. x_n

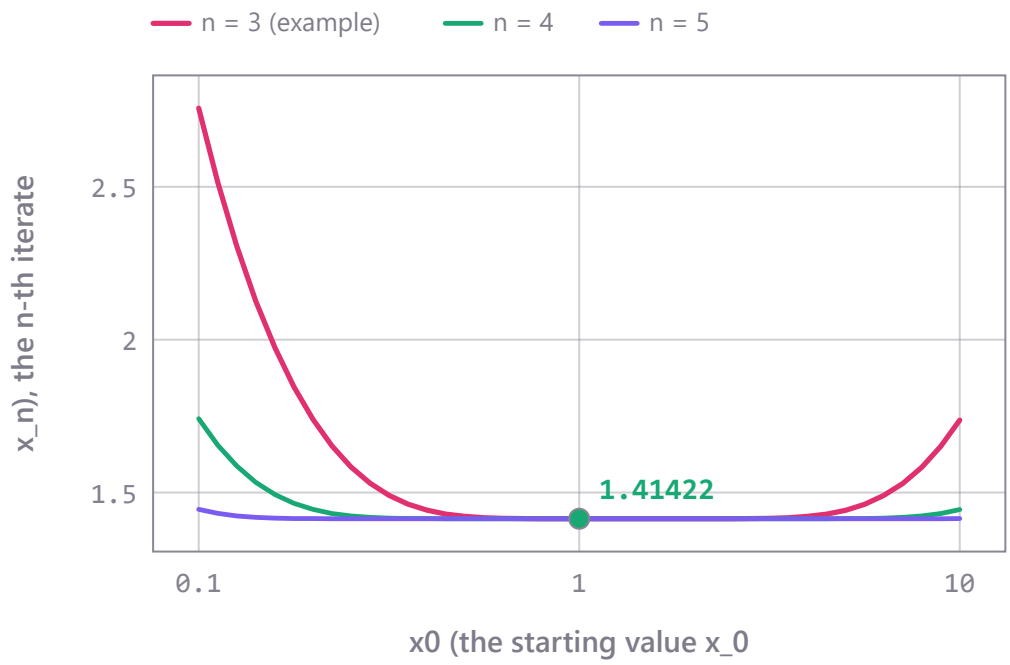
- **Method:** Newton-Raphson with the exact derivative, at 30 digits
- **Assumes:** f'$(x_k) \neq 0$ at every iterate; $1 \leq n \leq 50$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Newton's method on $x^2 - 2$ from $x_0 = 3$: each tangent (through the curve point computed by the package) meets the axis at the next iterate, 1.83333, 1.46212, 1.415, 1.41421; marked: x_4 = 1.41421, f(x_4) = 6.1568e-07.



x_1 .. x_n (iterates) returned by calculus-ai999x for the worked example, one value for each position in the list (3 values, first 1.5, last 1.41422, largest 1.5). Inputs: f = x^2 - 2, x0 = 1, n = 3.



x_n (the n-th iterate) against x0 (the starting value x_0) from 0.1 to 10 on a logarithmic axis. One curve for each value of n (number of iterations): 3, 4, 5. Other inputs as in the worked example: f = x^2 - 2. 41 points per curve computed by calculus-ai999x; marked: the worked example, x0 = 1, x_n = 1.41422.

```
calculus_ai999x.solve('newton_method', f='x^2 - 2', x0=1, n=3)
# -> x_n = 1.41422, f_x_n = 6.0073e-06, iterates = [1.5, 1.4166666666666667, 1.4142156862745099]
(SOLVED)
```

```
calculus_ai999x.solve('newton_method', f='x^3 - 2x - 5', x0=2, n=1)
# -> x_n = 2.1, f_x_n = 0.061, iterates = [2.1]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply Newton's method to f(x) = x^2 - 2 starting at x0 = 1 for 3 iterations.	SOLVED	x_n = 1.41422, f_x_n = 6.0073e-06, iterates = [1.5, 1.4166666666666667, 1.4142156862745099]
Use 1 iteration of Newton's method on x^3 - 2x - 5 from x0 = 2.	SOLVED	x_n = 2.1, f_x_n = 0.061, iterates = [2.1]
Apply Newton's method to f(x) = x^2 - 2 starting at x0 = 0 for 3 iterations.	INVALID_INPUT	Newton's method: f'(x) = 0 at an iterate, so Newton's method cannot continue.

Limit at a point, `limit_at_point`

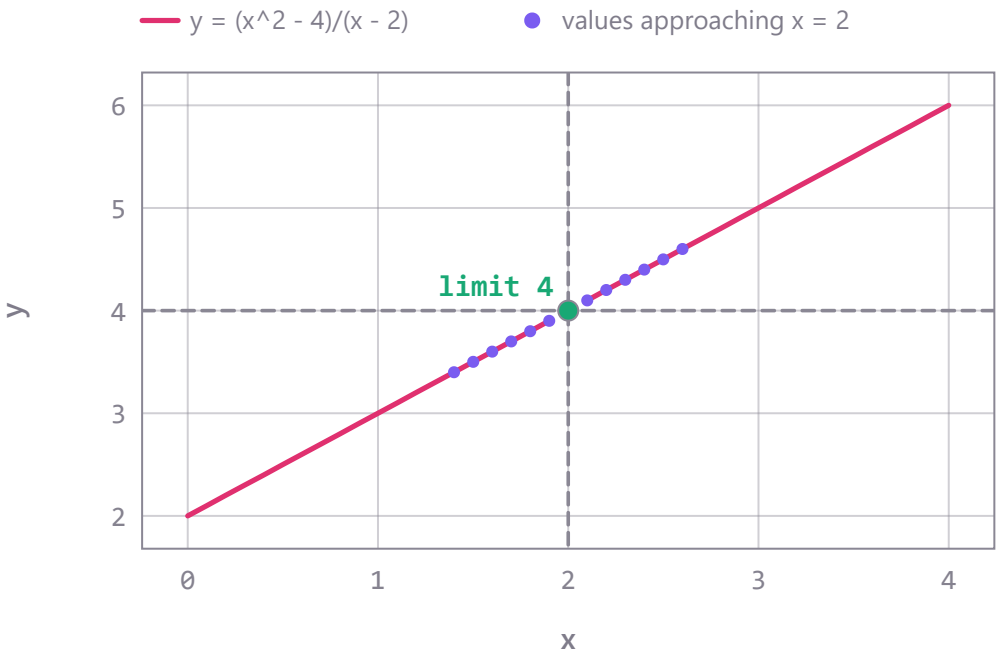
The two-sided limit of f(x) as x approaches a (including removable $0/0$ forms such as sin(x)/x).

$$\lim_{x \to a} f(x)$$

input	meaning	condition
`f`	the function f(x)	
`a`	the point $x = a$	

output	meaning
`limit`	the limit of f(x) as x -> a

- **Method:** exact symbolic limit (series expansion / Gruntz algorithm) from both sides; refused if the one-sided limits differ or are infinite
- **Assumes:** the left and right limits exist, are finite and agree
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y = (x^2 - 4)/(x - 2) near x = 2, where it is not defined (values computed by the package at 40 points, none at x = 2 itself): from both sides the values approach 4. Marked: the limit at the gap.

```
calculus_ai999x.solve('limit_at_point', f='(x^2 - 4)/(x - 2)', a=2)
# -> limit = 4   (SOLVED)
```

```
calculus_ai999x.solve('limit_at_point', f='sin(3x)/x', a=0)
# -> limit = 3   (SOLVED)
```

```
calculus_ai999x.solve('limit_at_point', f='(1 - cos(x))/x^2', a=0)
# -> limit = 0.5   (SOLVED)
```

```
calculus_ai999x.solve('limit_at_point', f='(exp(2x) - 1)/x', a=0)
# -> limit = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the limit of (x^2 - 4)/(x - 2) as x approaches 2.	SOLVED	limit = 4
What is the limit of sin(3x)/x as x -> 0?	SOLVED	limit = 3
Evaluate the limit as x approaches 0 of (1 - cos(x))/x^2.	SOLVED	limit = 0.5
Find the limit of 1/x as x approaches 0.	INVALID_INPUT	Limit at a point: the limit is infinite (it does not exist as a finite number).
Find the limit of 1/x^2 as x approaches 0.	INVALID_INPUT	Limit at a point: the limit is infinite (it does not exist as a finite number).

Limit at infinity, `limit_at_infinity`

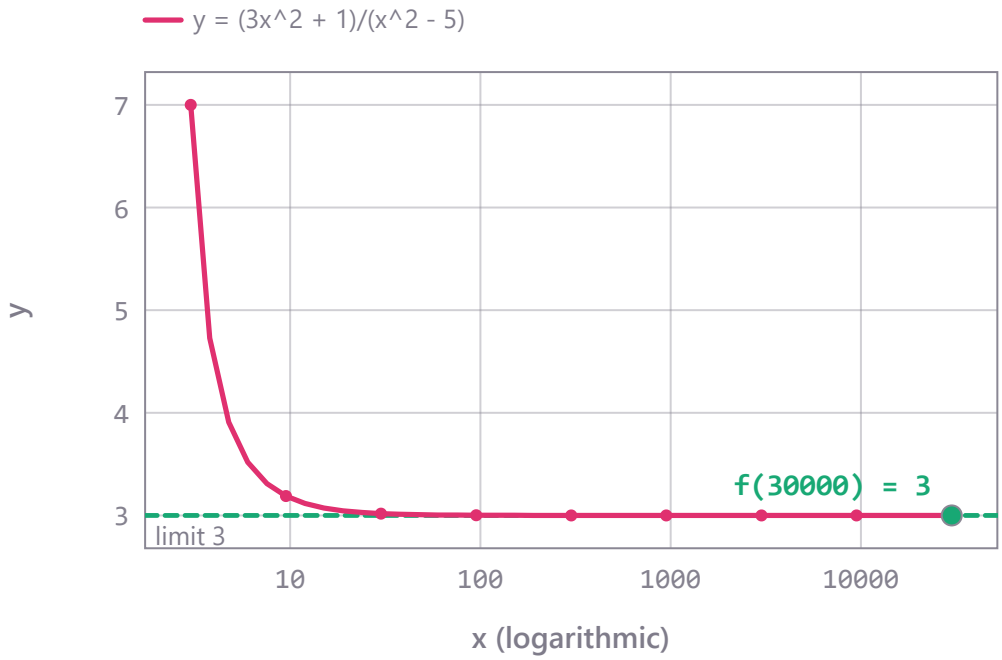
The limit of f(x) as x grows without bound (horizontal asymptote, end behaviour).

$$\lim_{x \to \infty} f(x)$$

input	meaning	condition
`f`	the function f(x)	

output	meaning
`limit`	the limit of f(x) as x -> infinity

- **Method:** exact symbolic limit at +infinity (Gruntz algorithm); refused if infinite or oscillating
- **Assumes:** the limit exists and is finite
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y = (3x^2 + 1)/(x^2 - 5) for x from 3 to 30000 (logarithmic axis; values computed by the package): the values settle onto the horizontal asymptote y = 3, the limit as x → ∞. Marked: the last value drawn.

```
calculus_ai999x.solve('limit_at_infinity', f='(3x^2 + 1)/(x^2 - 5)')
# -> limit = 3   (SOLVED)
```

```
calculus_ai999x.solve('limit_at_infinity', f='(1 + 2/x)^x')
# -> limit = 7.38906   (SOLVED)
```

```
calculus_ai999x.solve('limit_at_infinity', f='sqrt(x^2 + 4x) - x')
# -> limit = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the limit of (3x^2 + 1)/(x^2 - 5) as x approaches infinity.	SOLVED	limit = 3
What is the limit of (1 + 2/x)^x as x -> infinity?	SOLVED	limit = 7.38906
Find the limit as x approaches infinity of sqrt(x^2 + 4x) - x.	SOLVED	limit = 2
Find the limit of x^2 as x approaches infinity.	INVALID_INPUT	Limit at infinity: the limit is infinite (it does not exist as a finite number).
Find the limit of sin(x) as x approaches infinity.	INVALID_INPUT	Limit at infinity: the limit does not exist or could not be determined.

Implicit differentiation, `implicit_derivative`

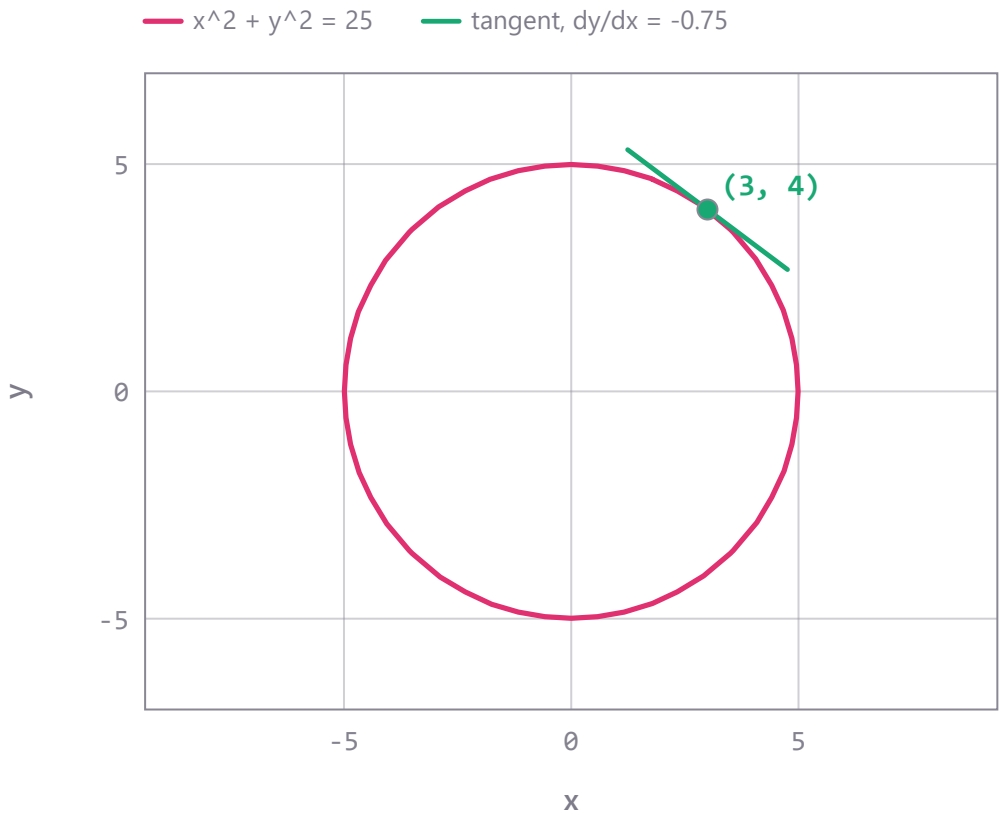
The slope dy/dx of a curve given implicitly by F(x, y) = c, at a point on the curve.

$$\frac{dy}{dx} = -\frac{F_x}{F_y}$$

input	meaning	condition
`F`	the curve as an equation in x and y	
`p`	the point (x, y) on the curve	

output	meaning
`dy_dx`	dy/dx at the point

- **Method:** $dy/dx = -F_x/F_y$ with exact symbolic partial derivatives of F(x, y) - c
- **Assumes:** the point lies on the curve; $F_y \neq 0$ there (no vertical tangent)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The curve x^2 + y^2 = 25, traced as the zero level of (x^2 + y^2) - (25) (values computed by the package on a 25 × 25 grid), and its tangent at (3, 4) with the slope found by implicit differentiation, dy/dx = -0.75. Marked: the point.

```
calculus_ai999x.solve('implicit_derivative', F='x^2 + y^2 = 25', p='3, 4')
# -> dy_dx = -0.75   (SOLVED)
```

```
calculus_ai999x.solve('implicit_derivative', F='x^3 + y^3 = 9', p=[1, 2])
# -> dy_dx = -0.25   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find dy/dx for x^2 + y^2 = 25 at the point (3, 4).	SOLVED	dy_dx = -0.75
Use implicit differentiation to find dy/dx for x^3 + y^3 = 9 at (1, 2).	SOLVED	dy_dx = -0.25

question	outcome	result
Find dy/dx for x^2 + y^2 = 25 at the point (1, 1).	INVALID_INPUT	Implicit differentiation: the point (1, 1) is not on the curve.
Find dy/dx for x^2 + y^2 = 25 at the point (5, 0).	INVALID_INPUT	Implicit differentiation: F_y = 0 at this point: the tangent is vertical (dy/dx is undefined).

Absolute extrema on a closed interval, `extrema_on_interval`

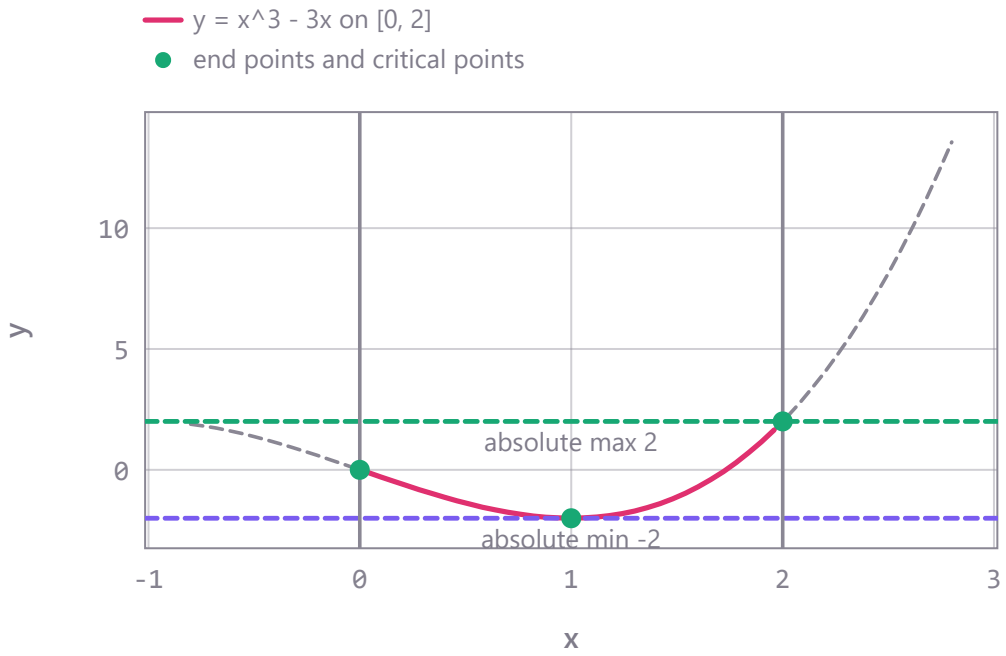
The absolute maximum and minimum values of a polynomial on [a, b] by the closed-interval method, with the critical points inside (a, b).

$$\max_{[a,b]} f = \max\{f(a), f(b), f(c) : f'(c) = 0\}$$

input	meaning	condition
`f`	the polynomial f(x)	
`a`	left end	
`b`	right end	

output	meaning
`absolute_max`	the largest value of f on [a, b]
`absolute_min`	the smallest value
`critical_points`	critical points in (a, b), ascending

- **Method:** exact real roots of f' in (a, b) (Sturm isolation), then f compared at the critical points and both ends
- **Assumes:** $a < b$; f is a polynomial with rational coefficients
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



y = x^3 - 3x on the closed interval [0, 2] (outside it dashed; curve values computed by the package). The candidates are the end points and the critical point(s) x = 1; marked: absolute maximum 2, absolute minimum -2.

```
calculus_ai999x.solve('extrema_on_interval', f='x^3 - 3x', a=0, b=2)
# -> absolute_max = 2, absolute_min = -2, critical_points = [1.0]   (SOLVED)
```

```
calculus_ai999x.solve('extrema_on_interval', f='x^2 - 4x + 1', a=-1, b=3)
# -> absolute_max = 6, absolute_min = -3, critical_points = [2.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the absolute maximum and minimum of x^3 - 3x on [0, 2].	SOLVED	absolute_max = 2, absolute_min = -2, critical_points = [1.0]
Find the absolute extrema of x^2 - 4x + 1 on the interval [-1, 3].	SOLVED	absolute_max = 6, absolute_min = -3, critical_points = [2.0]
Find the absolute extrema of x^2 on the interval [3, 1].	INVALID_INPUT	Absolute extrema on a closed interval: the interval needs a < b.
Find the absolute extrema of sin(x) on the interval [0, 3].	INVALID_INPUT	Absolute extrema on a closed interval: this capability needs a polynomial.

Critical points and their classification, `critical_points_classify`

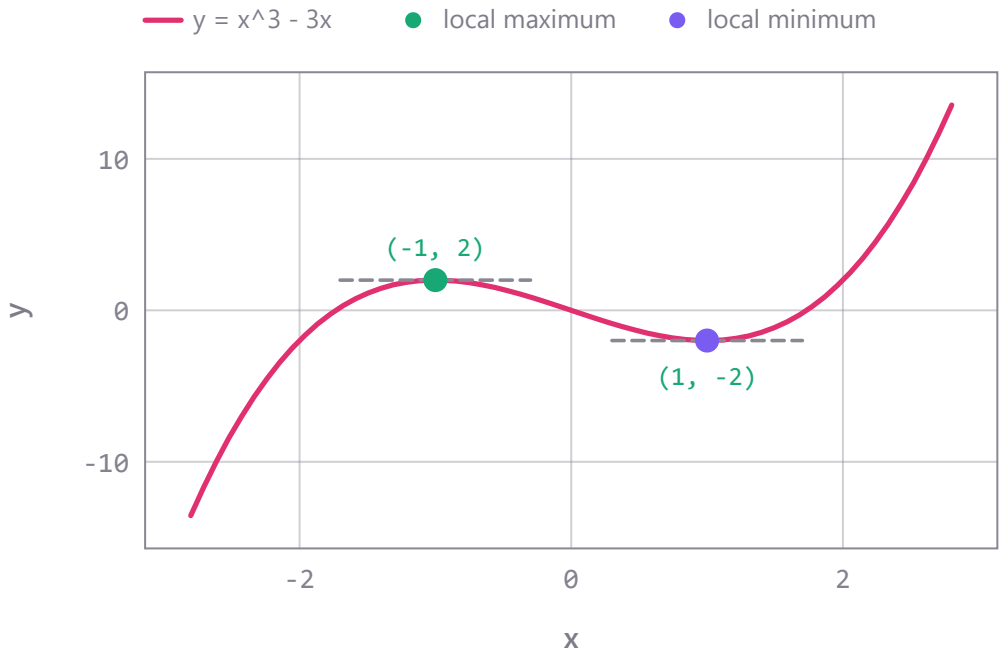
All critical points of a polynomial f(x) and which are local maxima or minima (first-derivative test).

$$f'(c) = 0,\ f' \text{ changes } + \to - \Rightarrow \text{local max}$$

input	meaning	condition
`f`	the polynomial f(x)	

output	meaning
`critical_points`	real solutions of f'$(x) = 0$, ascending
`local_maxima`	critical points that are local maxima
`local_minima`	critical points that are local minima

- **Method:** exact real roots of f' (Sturm isolation); the sign of f' on each side decides max / min / neither
- **Assumes:** f is a polynomial of degree >= 2 with rational coefficients
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The graph of y = x^3 - 3x (values computed by the package) with its critical points, where f'(x) = 0 (horizontal tangents dashed): local maxima at x = -1, local minima at x = 1.

```
calculus_ai999x.solve('critical_points_classify', f='x^3 - 3x')
# -> critical_points = [-1.0, 1.0], local_maxima = [-1.0], local_minima = [1.0]   (SOLVED)
```

```
calculus_ai999x.solve('critical_points_classify', f='x^4 - 2x^2')
# -> critical_points = [-1.0, 0.0, 1.0], local_maxima = [0.0], local_minima = [-1.0, 1.0]
(SOLVED)
```

```
calculus_ai999x.solve('critical_points_classify', f='x^3')
# -> critical_points = [0.0], local_maxima = [], local_minima = []   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find and classify the critical points of x^3 - 3x.	SOLVED	critical_points = [-1.0, 1.0], local_maxima = [-1.0], local_minima = [1.0]
Find the local extrema of x^4 - 2x^2.	SOLVED	critical_points = [-1.0, 0.0, 1.0], local_maxima = [0.0], local_minima = [-1.0, 1.0]
Classify the critical points of 2x + 1.	INVALID_INPUT	Critical points and their classification: needs a polynomial of degree 2 or more (a line has no isolated critical points

Inflection points, `inflection_points`

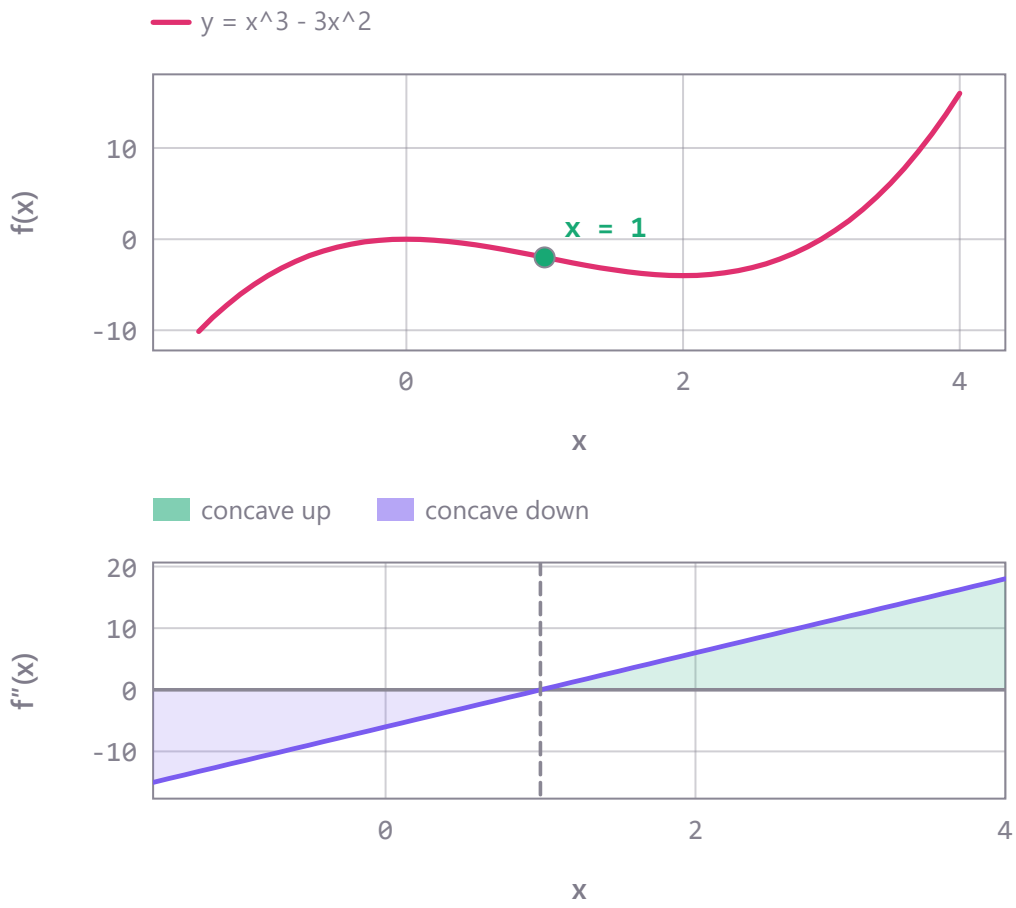
The points where a polynomial changes concavity (f'' changes sign).

$$f''(c) = 0 \text{ and } f'' \text{ changes sign at } c$$

input	meaning	condition
`f`	the polynomial f(x)	

output	meaning
`inflection_points`	x-values of the inflection points, ascending

- **Method:** exact real roots of f''; kept only where f'' changes sign (odd multiplicity)
- **Assumes:** f is a polynomial of degree >= 2 with rational coefficients
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: y = x^3 - 3x^2 (values computed by the package) with its inflection points marked. Bottom: f''(x) from the package changes sign there, so the curve turns from concave down to concave up. Marked: x = 1.

```
calculus_ai999x.solve('inflection_points', f='x^3 - 3x^2')
# -> inflection_points = [1.0]   (SOLVED)
```

```
calculus_ai999x.solve('inflection_points', f='x^4 - 6x^2')
# -> inflection_points = [-1.0, 1.0]   (SOLVED)
```

```
calculus_ai999x.solve('inflection_points', f='x^4')
# -> inflection_points = []   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the inflection points of x^3 - 3x^2.	SOLVED	inflection_points = [1.0]
Find the points of inflection of x^4 - 6x^2.	SOLVED	inflection_points = [-1.0, 1.0]
Find the inflection points of x^4.	SOLVED	inflection_points = []

Mean value theorem, `mean_value_theorem`

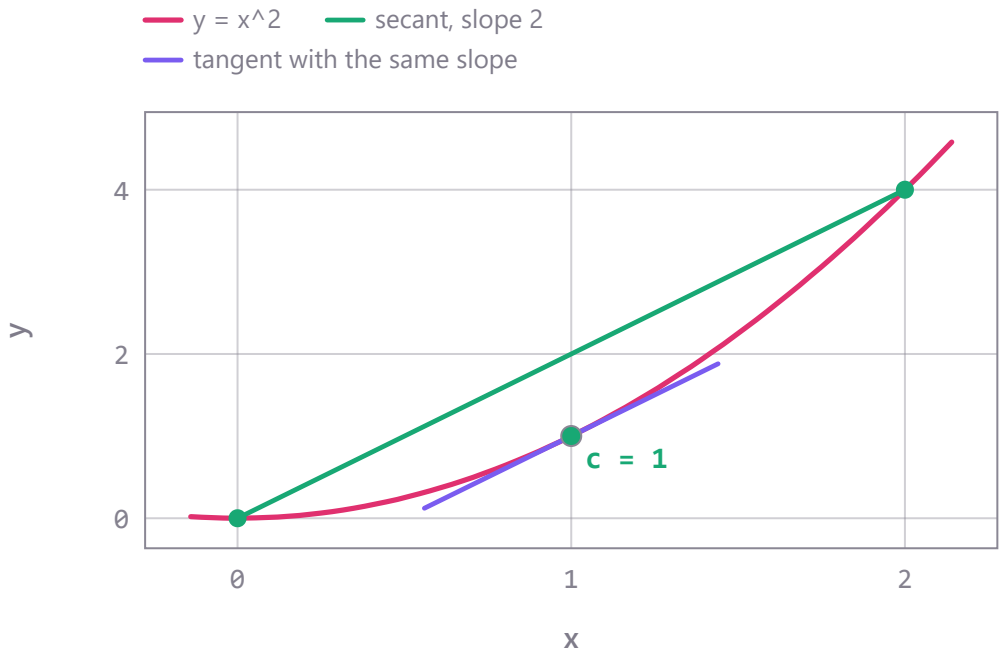
The secant slope of a polynomial on [a, b] and every c in (a, b) where f'(c) equals it.

$$f'(c) = \frac{f(b) - f(a)}{b - a}$$

input	meaning	condition
`f`	the polynomial f(x)	
`a`	left end	
`b`	right end	

output	meaning
`secant_slope`	(f(b) - f(a))/(b - a)
`c_values`	all c in (a, b) with f'(c) = the slope

- **Method:** exact secant slope, then the exact real roots of f'(x) - slope in (a, b)
- **Assumes:** $a < b$; f a non-linear polynomial with rational coefficients
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The mean value theorem for y = x^2 on [0, 2] (the worked example; curve values computed by the package): the secant slope is 2, and the tangent at each returned c has the same slope (tangent segments drawn with that slope through (c, f(c))). Marked: c = 1.

```
calculus_ai999x.solve('mean_value_theorem', f='x^2', a=0, b=2)
# -> secant_slope = 2, c_values = [1.0]   (SOLVED)
```

```
calculus_ai999x.solve('mean_value_theorem', f='x^3', a=0, b=3)
# -> secant_slope = 9, c_values = [1.7320508075688772]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the value of c guaranteed by the mean value theorem for x^3 on [0, 3].	SOLVED	secant_slope = 9, c_values = [1.7320508075688772]
Apply the mean value theorem to x^2 on [0, 2].	SOLVED	secant_slope = 2, c_values = [1.0]
Apply the mean value theorem to 3x + 1 on [0, 2].	INVALID_INPUT	Mean value theorem: f is linear: every c in (a, b) satisfies the mean value theorem.

Derivative of an integral (fundamental theorem), `ftc_derivative`

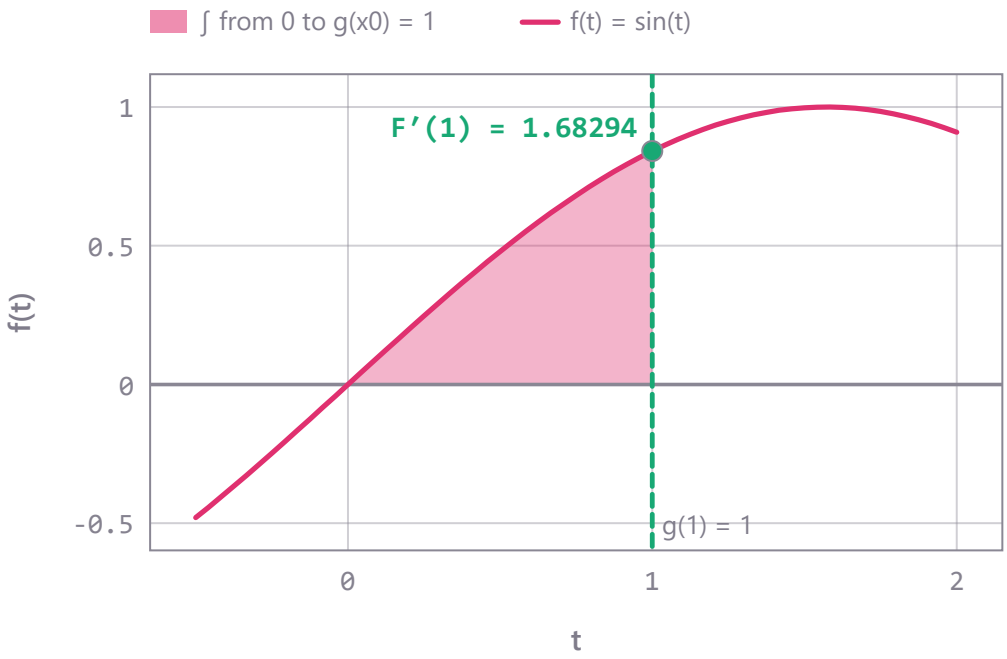
d/dx of the integral of f(t) from a to g(x), evaluated at $x = x_0$ (first fundamental theorem with the chain rule).

$$\frac{d}{dx}\int_a^{g(x)} f(t)\,dt = f(g(x))\,g'(x)$$

input	meaning	condition
`f`	the integrand f(t)	
`a`	the constant lower limit	
`g`	the upper limit g(x)	
`x0`	the point $x = x_0$	

output	meaning
`derivative`	d/dx of the integral at x0

- **Method:** f(g(x0)) * g'(x0) with the exact derivative of g
- **Assumes:** f continuous between a and g(x0); g differentiable at x0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



F(x) = ∫ from 0 to g(x) = x^2 of f(t) = sin(t) dt: the shaded area up to the moving limit g(1) = 1 (values computed by the package, t written as x). By the fundamental theorem its rate of change is f(g(x))·g'(x); the package gives F'(1) = 1.68294 (marked).

```
calculus_ai999x.solve('ftc_derivative', f='sin(t)', a=0, g='x^2', x0=1)
# -> derivative = 1.68294   (SOLVED)
```

```
calculus_ai999x.solve('ftc_derivative', f='t^2 + 1', a=1, g='3x', x0=2)
# -> derivative = 111   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the derivative of the integral from 0 to x^2 of sin(t) dt at x = 1.	SOLVED	derivative = 1.68294
What is d/dx of the integral from 1 to 3x of t^2 + 1 dt at x = 2?	SOLVED	derivative = 111
Find the derivative of the integral from 1 to x of 1/t dt at x = -1.	INVALID_INPUT	Derivative of an integral (fundamental theorem): the function is not continuous (or not real) on the whole interval [-1,

Sum of an infinite geometric series, `geometric_series_sum`

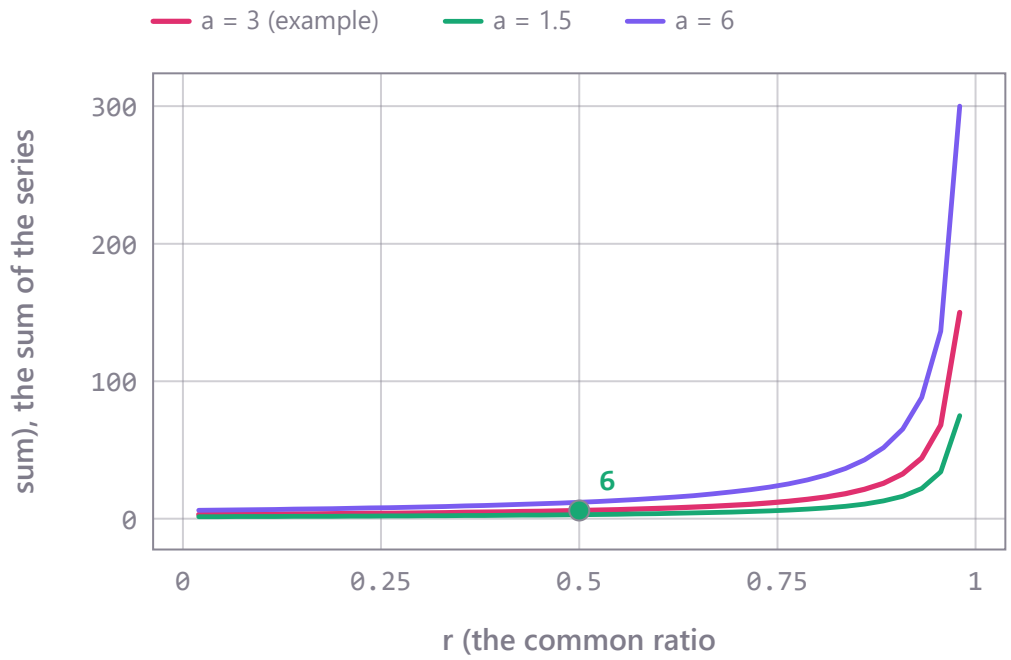
The sum a + a r + a $r^2 + \ldots$ of a convergent geometric series.

$$\sum_{n=0}^{\infty} ar^n = \frac{a}{1-r}$$

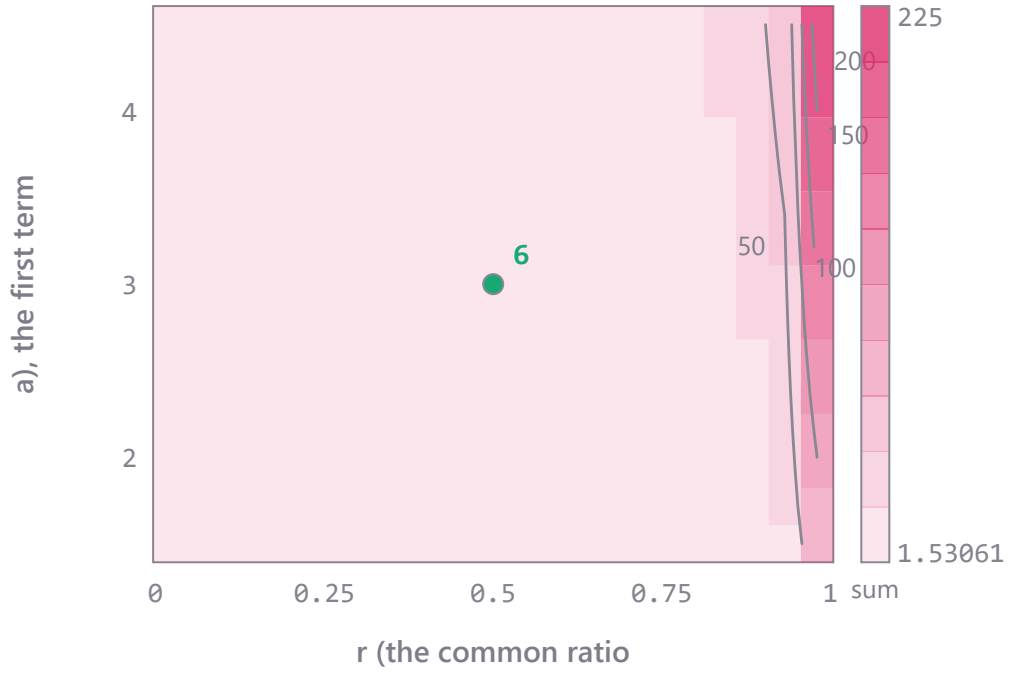
input	meaning	condition
`a`	the first term	
`r`	the common ratio	

output	meaning
`sum`	the sum of the series

- **Method:** $a/(1-r)$, exact in the stated decimals
- **Assumes:** |r| < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sum (the sum of the series) against r (the common ratio) from 0.02 to 0.98. One curve for each value of a (the first term): 3, 1.5, 6. 41 points per curve computed by calculus-ai999x; marked: the worked example, r = 0.5, sum = 6.



sum (the sum of the series) over r (the common ratio) from 0.02 to 0.98 and a (the first term) from 1.5 to 4.5: each of the 21 × 15 cells is one run of calculus-ai999x, shaded by its value (light 1.53061 to dark 225); iso-lines at sum = 50, 100, 150, 200. Marked: the worked example, r = 0.5, a = 3, sum = 6.

```
calculus_ai999x.solve('geometric_series_sum', a=3, r=0.5)
# -> sum = 6   (SOLVED)
```

```
calculus_ai999x.solve('geometric_series_sum', a=1, r=-0.25)
# -> sum = 0.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sum of the geometric series with first term 3 and ratio 0.5.	SOLVED	sum = 6
What is the sum of the infinite geometric series with first term 1 and common ratio -1/3?	SOLVED	sum = 0.75
Find the sum of the geometric series with first term 3 and ratio 2.	INVALID_INPUT	Sum of an infinite geometric series: an infinite geometric series converges only when |r| < 1.

Partial sum of a geometric series, `geometric_partial_sum`

The sum of the first n terms a + a r + ... + a r^{n-1}.

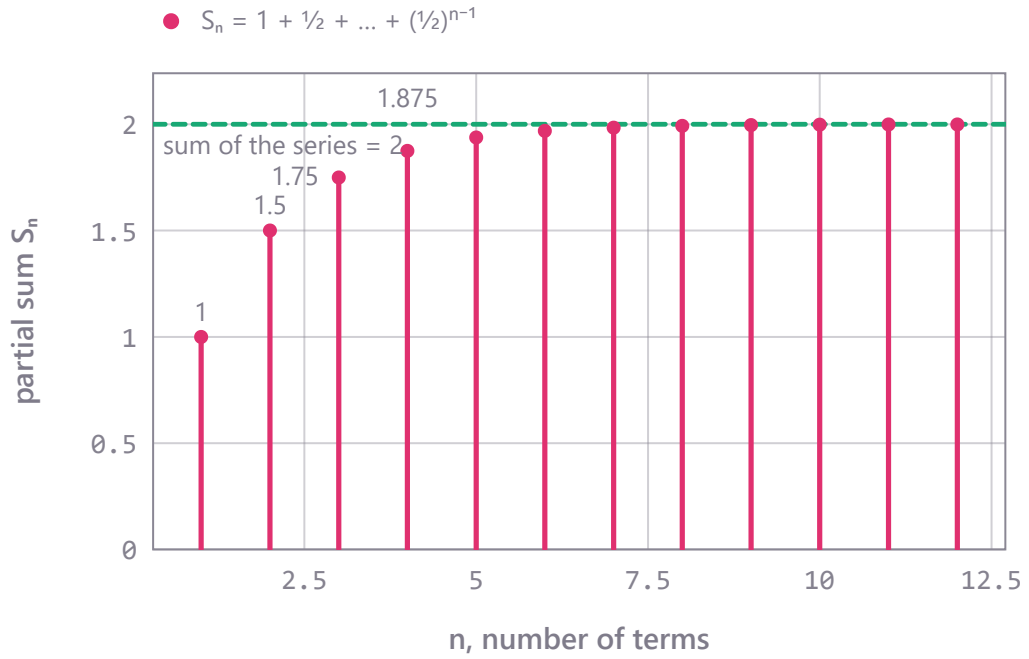
$$S_n = a\,\frac{1 - r^n}{1 - r}$$

input	meaning	condition
`a`	the first term	
`r`	the common ratio	

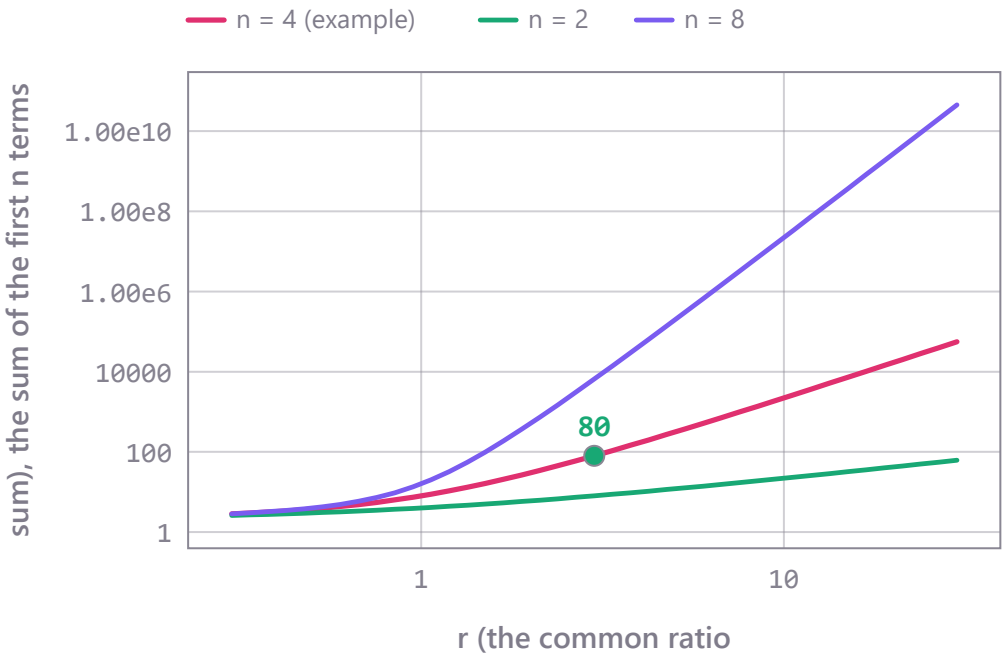
input	meaning	condition
`n`	the number of terms	

output	meaning
`sum`	the sum of the first n terms

- **Method:** a $(1 - r^n)/(1 - r)$ (n a when r = 1), exact in the stated decimals
- **Assumes:** $1 \leq n \leq 500$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Partial sums of the geometric series 1 + ½ + ¼ + …, each computed by the package: $S_1 = 1$, $S_4 = 1.875$, $S_{12} = 1.99951$; the gap to the sum of the series, 2, halves with every term.



sum (the sum of the first n terms) against r (the common ratio) from 0.3 to 30 on a logarithmic axis, logarithmic output axis. One curve for each value of n (the number of terms): 4, 2, 8. Other inputs as in the worked example: a = 2. 41 points per curve computed by calculus-ai999x; marked: the worked example, r = 3, sum = 80.

```
calculus_ai999x.solve('geometric_partial_sum', a=2, r=3, n=4)
# -> sum = 80   (SOLVED)
```

```
calculus_ai999x.solve('geometric_partial_sum', a=1, r=0.5, n=3)
# -> sum = 1.75   (SOLVED)
```

```
calculus_ai999x.solve('geometric_partial_sum', a=5, r=1, n=6)
# -> sum = 30   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sum of the first 4 terms of the geometric series with first term 2 and ratio 3.	SOLVED	sum = 80
What is the sum of the first 3 terms of a geometric sequence with first term 1 and common ratio 0.5?	SOLVED	sum = 1.75

Sum of an arithmetic series, `arithmetic_series_sum`

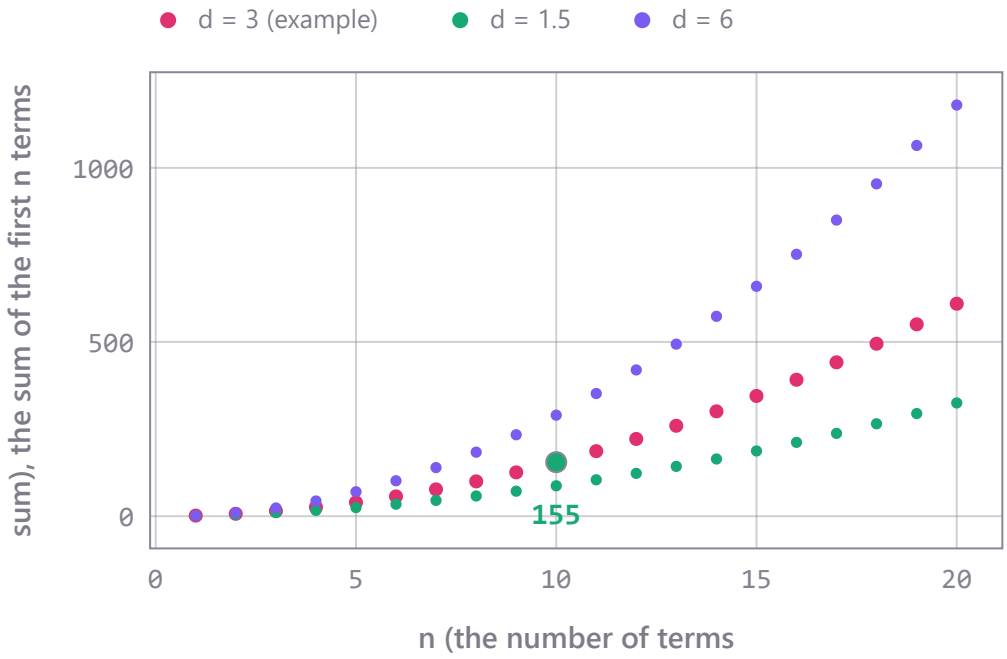
The sum of the first n terms of an arithmetic sequence, and its last term.

$$S_n = \frac{n}{2}(2a_1 + (n-1)d)$$

input	meaning	condition
`a1`	the first term	
`d`	the common difference	
`n`	the number of terms	

output	meaning
`sum`	the sum of the first n terms
`last_term`	the n-th term

- **Method:** $S_n = n$ (2 $a_1 + (n - 1)$ d)/2, exact in the stated decimals
- **Assumes:** $n \geq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sum (the sum of the first n terms) against n (the number of terms) from 1 to 20. One curve for each value of d (the common difference): 3, 1.5, 6. Other inputs as in the worked example: a1 = 2. 20 points per curve computed by calculus-ai999x; marked: the worked example, n = 10, sum = 155.

```
calculus_ai999x.solve('arithmetic_series_sum', a1=2, d=3, n=10)
# -> sum = 155, last_term = 29   (SOLVED)
```

```
calculus_ai999x.solve('arithmetic_series_sum', a1=1, d=1, n=100)
# -> sum = 5050, last_term = 100   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sum of the first 10 terms of the arithmetic series with first term 2 and common difference 3.	SOLVED	sum = 155, last_term = 29
What is the sum of the first 100 terms of an arithmetic sequence with first term 1 and difference 1?	SOLVED	sum = 5050, last_term = 100

Sum of an infinite series, `series_sum`

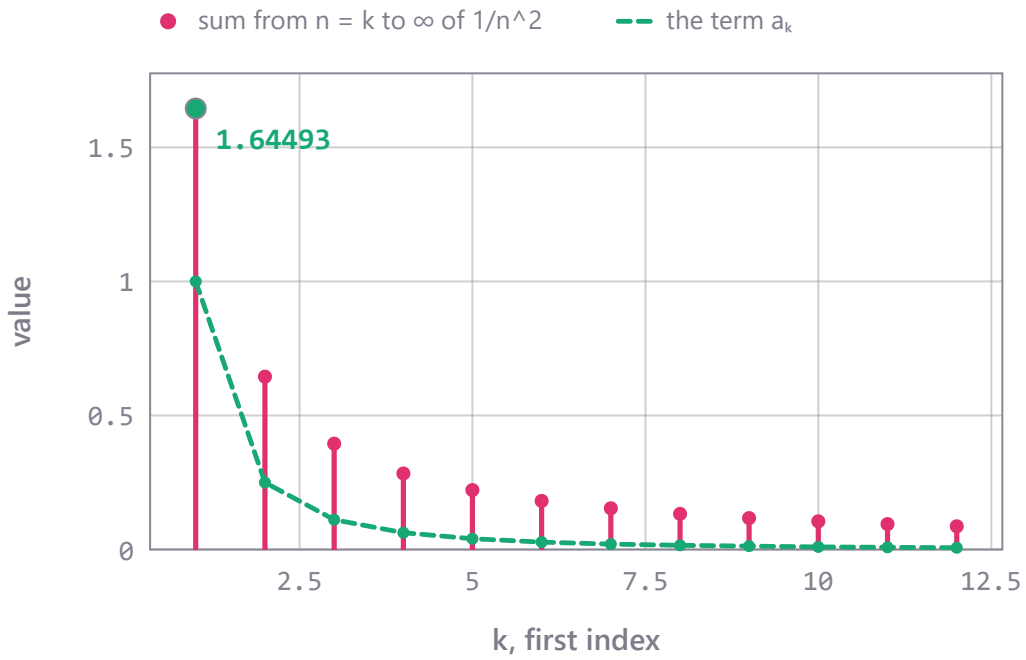
The exact sum of a convergent series sum_{$n = k$}^infinity a_n (telescoping, geometric-type, zeta values, alternating harmonic, ...).

$$\sum_{n=k}^{\infty} a_n$$

input	meaning	condition
`a`	the general term a_n, in n	
`k`	the starting index	

output	meaning
`sum`	the sum of the series

- **Method:** exact symbolic summation; refused if the series diverges or no closed form is found
- **Assumes:** the series converges; a closed form exists
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The infinite series of a_n = 1/n^2 summed from n = k onward, for k = 1 ... 12, each sum computed by the package, beside the terms themselves (computed by the package, n written as x): the tails shrink to 0, more slowly than the terms. Marked: the full sum from n = 1, 1.64493.

```
calculus_ai999x.solve('series_sum', a='1/n^2', k=1)
# -> sum = 1.64493   (SOLVED)
```

```
calculus_ai999x.solve('series_sum', a='1/(n*(n + 1))', k=1)
# -> sum = 1   (SOLVED)
```

```
calculus_ai999x.solve('series_sum', a='(1/2)^n', k=0)
# -> sum = 2   (SOLVED)
```

```
calculus_ai999x.solve('series_sum', a='(-1)^(n + 1)/n', k=1)
# -> sum = 0.693147   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the sum of the series 1/n^2 from n = 1 to infinity.	SOLVED	sum = 1.64493
Find the sum of 1/(n*(n + 1)) from n = 1 to infinity.	SOLVED	sum = 1
Find the sum of the series 1/n from n = 1 to infinity.	INVALID_INPUT	Sum of an infinite series: the series diverges.

Ratio test, `ratio_test`

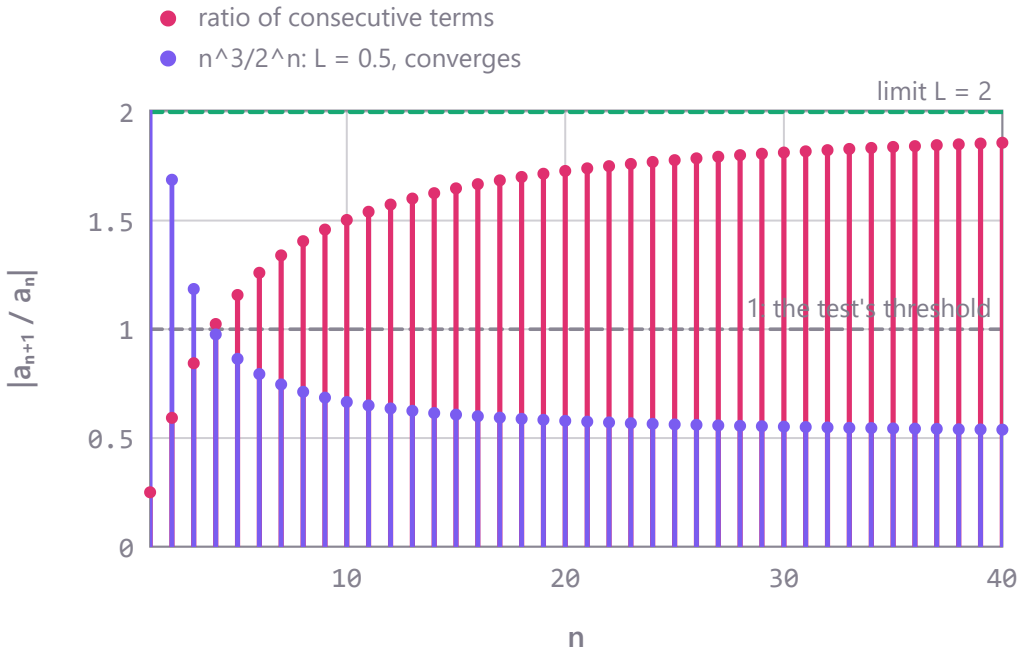
The limit L of |a_(n+1)/a_n| and the ratio test's verdict for the series sum a_n (converges if $L < 1$, diverges if L > 1).

$$L = \lim_{n \to \infty} \left| \frac{a_{n+1}}{a_n} \right|$$

input	meaning	condition
`a`	the general term a_n, in n	

output	meaning
`ratio_limit`	L = lim |a_(n+1)/a_n|
`verdict`	converges or diverges

- **Method:** exact simplification of a_(n+1)/a_n and its symbolic limit as n -> infinity
- **Assumes:** L exists, is finite and is not 1 (L = 1 is inconclusive and refused)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The ratio test: $|a_{n+1}/a_n|$ for n = 1 ... 40, the ratios formed from term values computed by the package (n written as x). For a_n = 2^n/n^3 they approach L = 2 > 1, so the series diverges; for a_n = n^3/2^n (chosen) they approach 0.5 < 1 and it converges.

```
calculus_ai999x.solve('ratio_test', a='2^n/n^3')
# -> ratio_limit = 2, verdict = diverges   (SOLVED)
```

```
calculus_ai999x.solve('ratio_test', a='n^2/3^n')
# -> ratio_limit = 0.333333, verdict = converges   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Apply the ratio test to the series 2^n/n^3.	SOLVED	ratio_limit = 2, verdict = diverges
Use the ratio test on n^2/3^n.	SOLVED	ratio_limit = 0.333333, verdict = converges
Apply the ratio test to the series 1/n^2.	INVALID_INPUT	Ratio test: the ratio limit is 1: the ratio test is inconclusive.

Taylor polynomial, `taylor_polynomial`

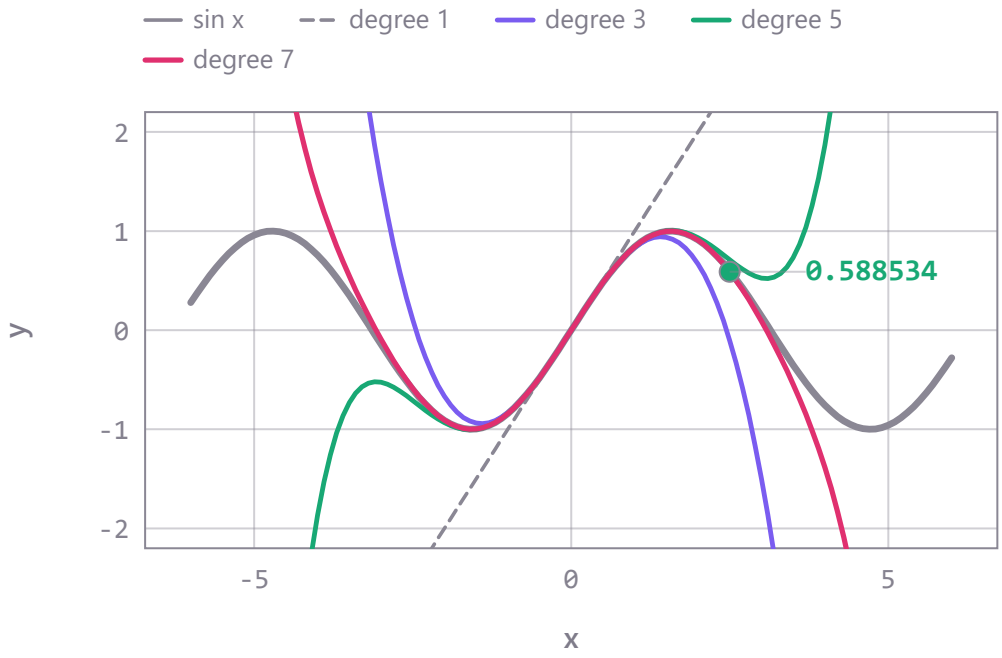
The degree-n Taylor polynomial of f about $x = a$ (Maclaurin when a = 0): its coefficients, its value at x1, f(x1) and the truncation error.

$$P_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k$$

input	meaning	condition
`f`	the function f(x)	
`a`	the centre $x = a$	
`n`	the degree	
`x1`	the evaluation point	

output	meaning
`coefficients`	c_0 .. c_n with $c_k = f^k$(a)/k!
`value`	P_n(x1)
`function_value`	f(x1)
`error`	f(x1) - P_n(x1)

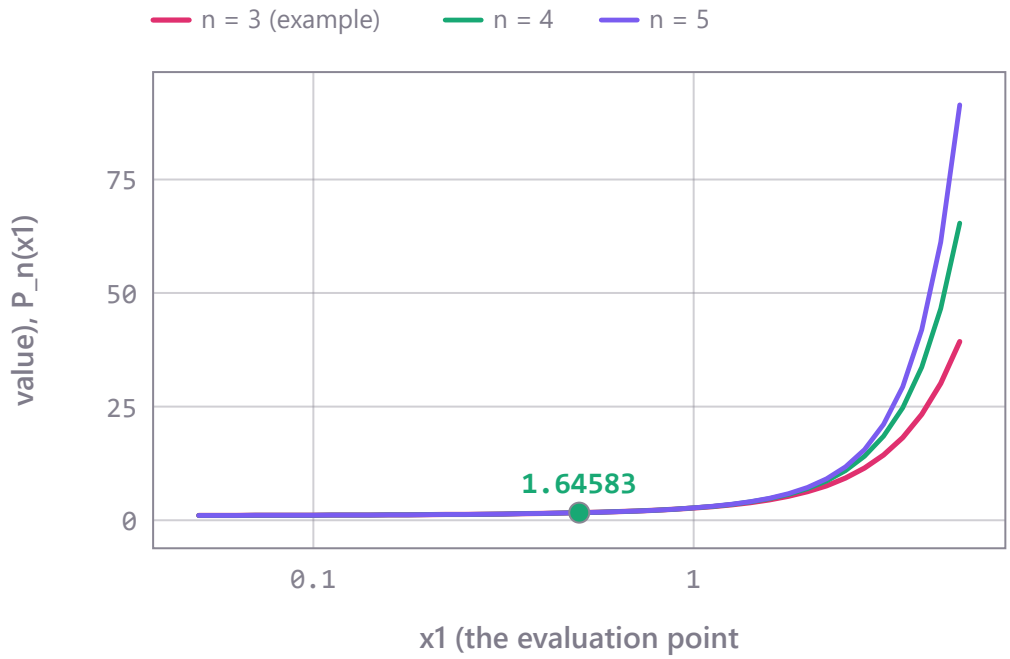
- **Method:** exact symbolic derivatives at a divided by k!, then Horner evaluation in $(x_1 - a)$
- **Assumes:** f is n times differentiable at a and defined at x1; $0 \le n \le 12$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Taylor polynomials of sin x about 0 of degree 1, 3, 5 and 7 against sin x itself, each evaluated by the package at 121 points; marked: x = 2.5, degree-7 value = 0.588534 (sin 2.5 = 0.598472, error 0.00993823).



c_0 .. c_n with c_k = f^(k)(a)/k! (coefficients) returned by calculus-ai999x for the worked example, one value for each position in the list (4 values, first 1, last 0.166667, largest 1). Inputs: f = exp(x), a = 0, n = 3, x1 = 0.5.



value (P_n(x1)) against x1 (the evaluation point) from 0.05 to 5 on a logarithmic axis. One curve for each value of n (the degree): 3, 4, 5. Other inputs as in the worked example: f = exp(x), a = 0. 41 points per curve computed by calculus-ai999x; marked: the worked example, x1 = 0.5, value = 1.64583.

```
calculus_ai999x.solve('taylor_polynomial', f='exp(x)', a=0, n=3, x1=0.5)
# -> coefficients = [1.0, 1.0, 0.5, 0.16666666666666666], value = 1.64583, function_value =
1.64872, error = 0.00288794   (SOLVED)
```

```
calculus_ai999x.solve('taylor_polynomial', f='cos(x)', a=0, n=4, x1=0.3)
# -> coefficients = [1.0, 0.0, -0.5, 0.0, 0.041666666666666664], value = 0.955337, function_value =
0.955336, error = -1.01087e-06   (SOLVED)
```

```
calculus_ai999x.solve('taylor_polynomial', f='log(x)', a=1, n=2, x1=1.5)
# -> coefficients = [0.0, 1.0, -0.5], value = 0.375, function_value = 0.405465, error = 0.0304651
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Taylor polynomial of degree 3 of exp(x) about x = 0 and evaluate it at x = 0.5.	SOLVED	coefficients = [1.0, 1.0, 0.5, 0.16666666666666666], value = 1.64583, function_value = 1.64872, error = 0.00288794
Evaluate the degree 4 Maclaurin polynomial of cos(x) at x = 0.3.	SOLVED	coefficients = [1.0, 0.0, -0.5, 0.0, 0.041666666666666664], value = 0.955337, function_value = 0.955336, error = -1.01087e-06
Find the Taylor polynomial of degree 2 of log(x) about x = 0 and evaluate it at x = 0.5.	INVALID_INPUT	Taylor polynomial: the result is not finite.

Partial derivative, `partial_derivative`

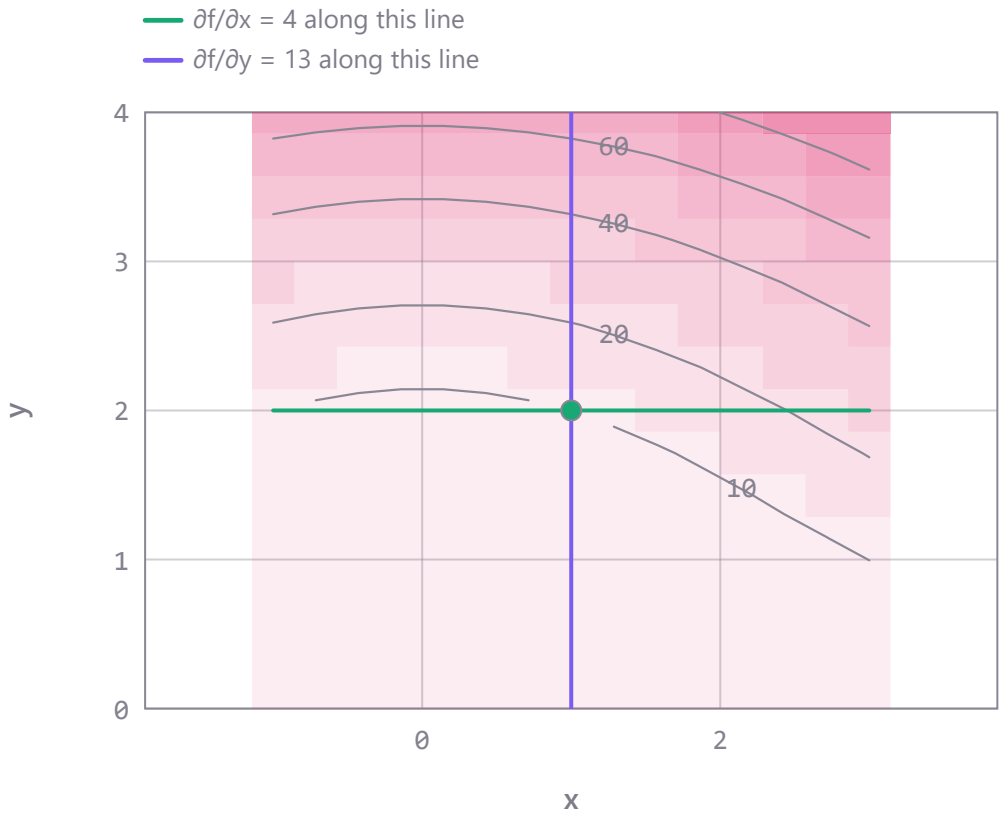
The partial derivative of f(x, y) or f(x, y, z) with respect to one variable, at a point.

$$\frac{\partial f}{\partial x}(p)$$

input	meaning	condition
`f`	the function f(x, y) or f(x, y, z)	
`var`	the variable	
`p`	the point (x, y) or (x, y, z)	

output	meaning
`partial`	the partial derivative at the point

- **Method:** exact symbolic partial differentiation, then evaluation at the point
- **Assumes:** f differentiable at the point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Level curves of f(x, y) = x^2*y + y^3 around (1, 2) (f computed by the package): the partial derivatives are the slopes of f along the two lines through the point, ∂f/∂x = 4 (the worked example) and ∂f/∂y = 13, both from the package. Marked: the point, f = 10.

```
calculus_ai999x.solve('partial_derivative', f='x^2*y + y^3', var='x', p='1, 2')
# -> partial = 4   (SOLVED)
```

```
calculus_ai999x.solve('partial_derivative', f='x^2*y + y^3', var='y', p=[1, 2])
# -> partial = 13   (SOLVED)
```

```
calculus_ai999x.solve('partial_derivative', f='x*y*z', var='z', p='1, 2, 3')
# -> partial = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the partial derivative of x^2*y + y^3 with respect to x at (1, 2).	SOLVED	partial = 4
What is the partial derivative of x*y*z with respect to z at the point (1, 2, 3)?	SOLVED	partial = 2
Find the partial derivative of x^2*y with respect to z at (1, 2).	INVALID_INPUT	Partial derivative: the variable must be one of x, y.

Gradient, `gradient`

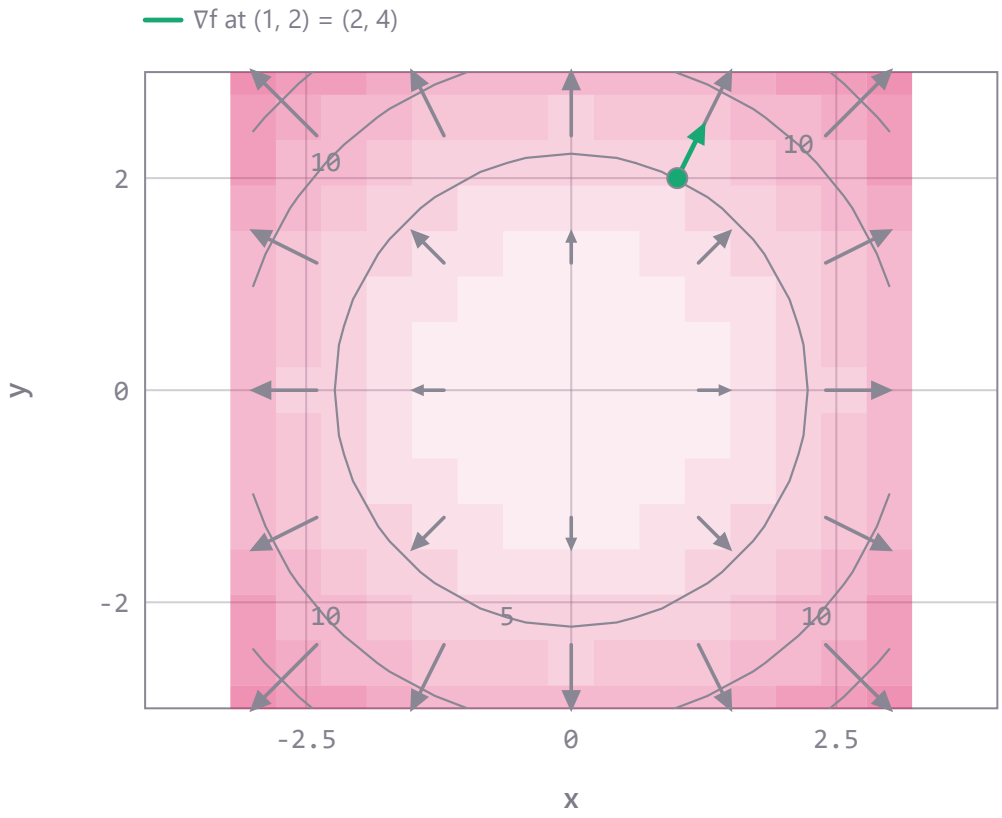
The gradient vector of f(x, y) or f(x, y, z) at a point and its length (the greatest rate of increase).

$$\nabla f = (f_x, f_y, f_z)$$

input	meaning	condition
`f`	the function f(x, y) or f(x, y, z)	
`p`	the point (x, y) or (x, y, z)	

output	meaning
`gradient`	the gradient vector
`magnitude`	its length

- **Method:** exact symbolic partial derivatives at the point
- **Assumes:** f differentiable at the point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Level curves of f(x, y) = x^2 + y^2 (shaded by value, f computed by the package on a 15 × 15 grid) and its gradient from the package at 25 points (arrows, all scaled by the same factor): the gradient is perpendicular to the level curves and points uphill. Marked: ∇f(1, 2) = (2, 4), magnitude 4.47214.

```
calculus_ai999x.solve('gradient', f='x^2 + y^2', p='1, 2')
# -> gradient = [2.0, 4.0], magnitude = 4.47214   (SOLVED)
```

```
calculus_ai999x.solve('gradient', f='x*y*z', p=[1, 2, 3])
# -> gradient = [6.0, 3.0, 2.0], magnitude = 7   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the gradient of x^2 + y^2 at (1, 2).	SOLVED	gradient = [2.0, 4.0], magnitude = 4.47214
What is the gradient of x*y*z at the point (1, 2, 3)?	SOLVED	gradient = [6.0, 3.0, 2.0], magnitude = 7
Find the gradient of log(x) + y at (-1, 2).	INVALID_INPUT	Gradient: the expression is not real there.

Directional derivative, `directional_derivative`

The rate of change of f at a point in the direction of a vector u (u is normalised to unit length).

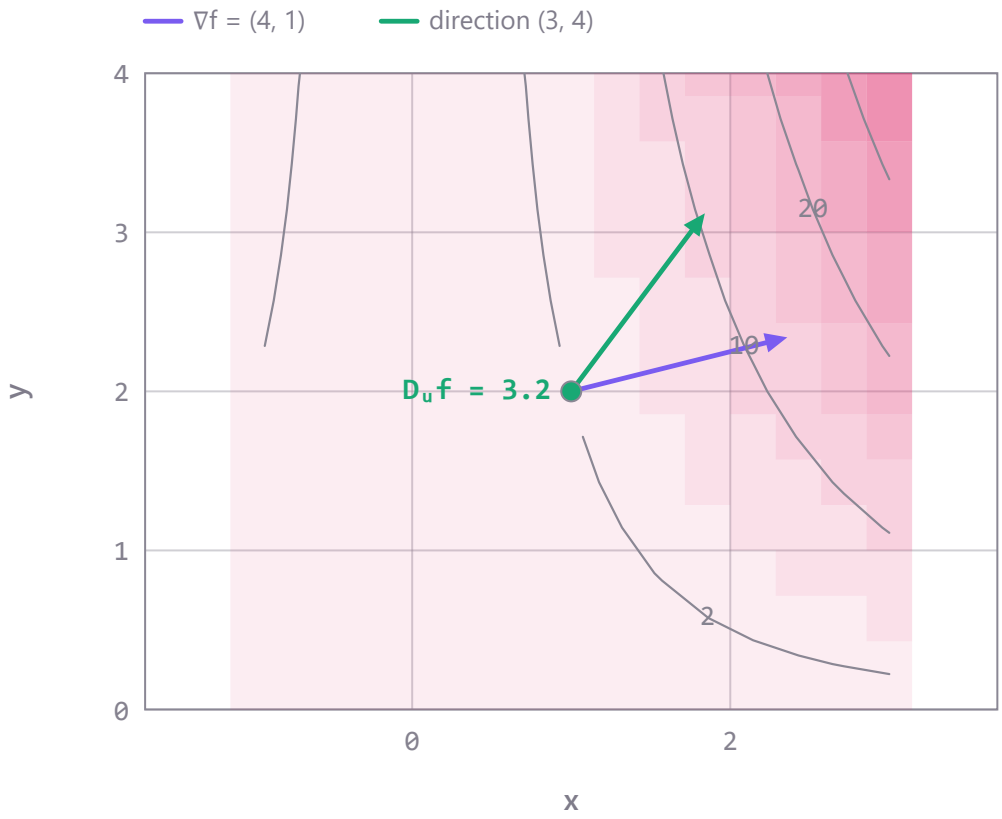
$$D_{\mathbf{u}} f = \nabla f \cdot \frac{\mathbf{u}}{|\mathbf{u}|}$$

input	meaning	condition
`f`	the function f(x, y) or f(x, y, z)	
`p`	the point	

input	meaning	condition
`u`	the direction vector	

output	meaning
`directional_derivative`	D_u f at the point

- **Method:** gradient (exact partial derivatives) dotted with u/|u|
- **Assumes:** $u \neq 0$ with as many components as p; f differentiable at p
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Level curves of f(x, y) = x^2*y around (1, 2) (f computed by the package; the level through the point is f = 2), the gradient there (4, 1) and the stated direction (3, 4) as unit arrows: the directional derivative, the gradient's component along that direction, is 3.2 (marked).

```
calculus_ai999x.solve('directional_derivative', f='x^2*y', p='1, 2', u='3, 4')
# -> directional_derivative = 3.2   (SOLVED)
```

```
calculus_ai999x.solve('directional_derivative', f='x*y*z', p=[1, 2, 3], u=[1, 2, 2])
# -> directional_derivative = 5.33333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the directional derivative of x^2*y at (1, 2) in the direction (3, 4).	SOLVED	directional_derivative = 3.2
Find the directional derivative of x*y*z at the point (1, 2, 3) in the direction of the vector (1, 2, 2).	SOLVED	directional_derivative = 5.33333

question	outcome	result
Find the directional derivative of x^2*y at (1, 2) in the direction (0, 0).	INVALID_INPUT	Directional derivative: the direction vector must not be zero.

Tangent plane, `tangent_plane`

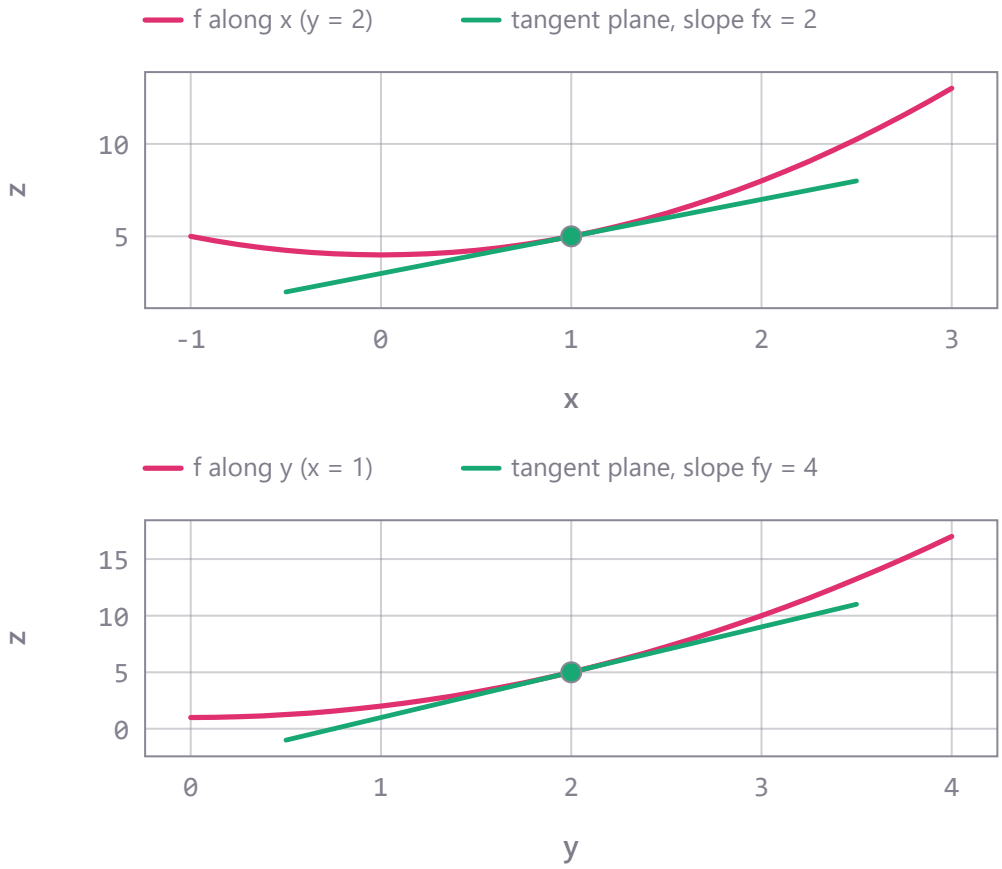
The tangent plane $z = f_x$ x + f_y y + c to the surface $z = f$(x, y) at (x0, y0).

$$z = f(x_0, y_0) + f_x(x - x_0) + f_y(y - y_0)$$

input	meaning	condition
`f`	the surface $z = f$(x, y)	
`p`	the point (x0, y0)	

output	meaning
`z0`	f(x0, y0)
`fx`	f_x(x0, y0)
`fy`	f_y(x0, y0)
`c`	the constant term c

- **Method:** exact partial derivatives; $z = z_0 + f_x\,(x - x_0) + f_y\,(y - y_0)$ expanded
- **Assumes:** f differentiable at (x0, y0)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Two slices through the surface z = x^2 + y^2 at (1, 2) (surface values computed by the package): along x and along y the tangent plane z = 5 + 2(x − 1) + 4(y − 2) touches the surface with slopes fx = 2 and fy = 4. Marked: z0 = 5.

```
calculus_ai999x.solve('tangent_plane', f='x^2 + y^2', p='1, 2')
# -> z0 = 5, fx = 2, fy = 4, c = -5   (SOLVED)
```

```
calculus_ai999x.solve('tangent_plane', f='x*y', p=[2, 3])
# -> z0 = 6, fx = 3, fy = 2, c = -6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the tangent plane to z = x^2 + y^2 at (1, 2).	SOLVED	z0 = 5, fx = 2, fy = 4, c = -5
Find the equation of the tangent plane to the surface z = x*y at the point (2, 3).	SOLVED	z0 = 6, fx = 3, fy = 2, c = -6

Second derivative test (two variables), `hessian_classification`

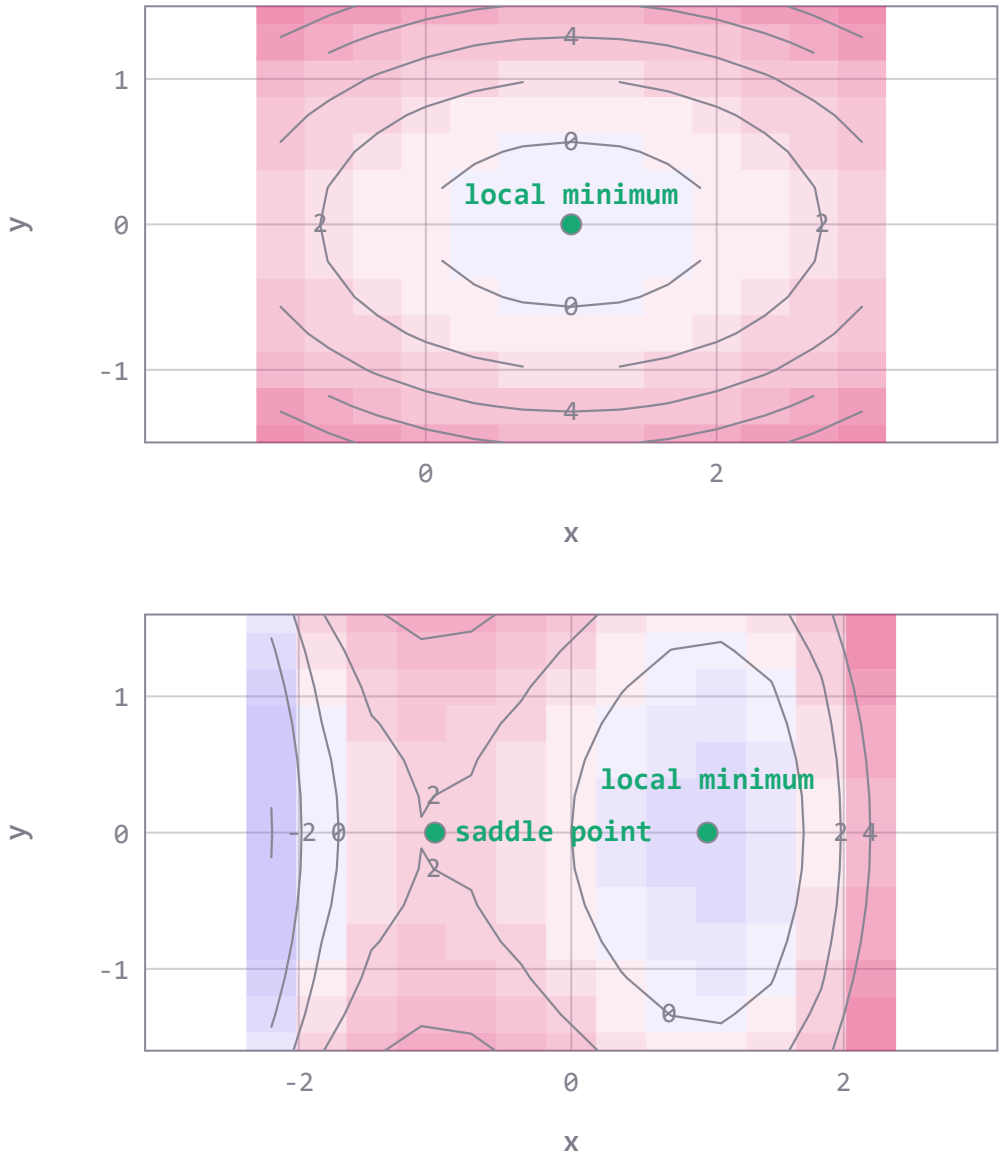
Classifies a critical point of f(x, y) as a local minimum, local maximum or saddle point from the Hessian.

$$D = f_{xx} f_{yy} - f_{xy}^2$$

input	meaning	condition
`f`	the function f(x, y)	
`p`	the critical point (x, y)	

output	meaning
`fxx`	f_xx
`fxy`	f_xy
`fyy`	f_yy
`D`	$D = f_x x\ f_y y - f_x y^2$
`classification`	local minimum, local maximum or saddle point

- **Method:** exact second partial derivatives at the point; $D > 0$ and $f_x x > 0$ minimum, $D > 0$ and $f_x x < 0$ maximum, $D < 0$ saddle
- **Assumes:** the gradient is zero at the point; $D \neq 0$ (D = 0 is inconclusive and refused)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Top: level curves of the worked example f = x^2 + 3y^2 - 2x near (1, 0) (f computed by the package): fxx = 2, fyy = 6, fxy = 0, D = 12, local minimum. Bottom (chosen): f = x^3 - 3x + y^2 with D = -12 at (−1, 0), saddle point, and D = 12 at (1, 0), local minimum.

```
calculus_ai999x.solve('hessian_classification', f='x^2 + 3y^2 - 2x', p='1, 0')
# -> fxx = 2, fxy = 0, fyy = 6, D = 12, classification = local minimum   (SOLVED)
```

```
calculus_ai999x.solve('hessian_classification', f='x^2 - y^2', p=[0, 0])
# -> fxx = 2, fxy = 0, fyy = -2, D = -4, classification = saddle point   (SOLVED)
```

```
calculus_ai999x.solve('hessian_classification', f='4 - x^2 - y^2 + x*y', p=[0, 0])
# -> fxx = -2, fxy = 1, fyy = -2, D = 3, classification = local maximum   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Classify the critical point of x^2 + 3y^2 - 2x at (1, 0).	SOLVED	fxx = 2, fxy = 0, fyy = 6, D = 12, classification = local minimum
Use the second derivative test for x^2 - y^2 at the point (0, 0).	SOLVED	fxx = 2, fxy = 0, fyy = -2, D = -4, classification = saddle point
Classify the critical point of x^2 + y^2 at (1, 0).	INVALID_INPUT	Second derivative test (two variables): (1, 0) is not a critical point (the gradient is (2, 0)).
Classify the critical point of x^4 + y^4 at (0, 0).	INVALID_INPUT	Second derivative test (two variables): D = 0: the second derivative test is inconclusive.

Laplacian, `laplacian`

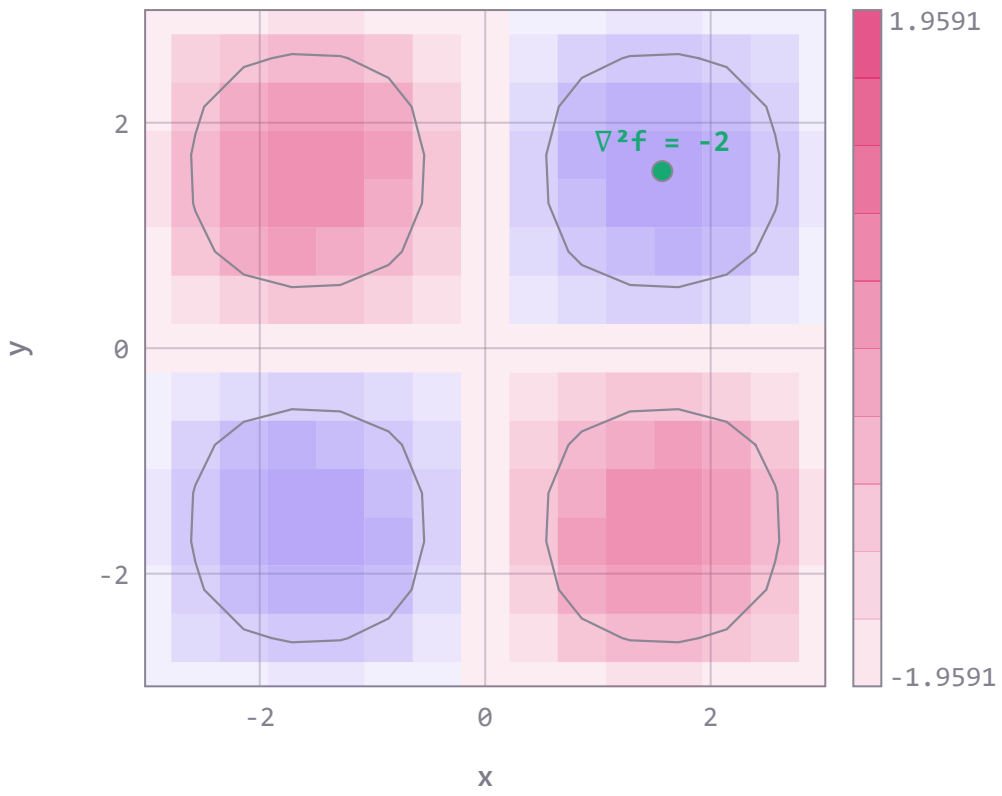
The Laplacian f_xx + f_yy (+ f_zz) of a function at a point.

$$\nabla^2 f = f_{xx} + f_{yy} + f_{zz}$$

input	meaning	condition
`f`	the function f(x, y) or f(x, y, z)	
`p`	the point	

output	meaning
`laplacian`	the Laplacian at the point

- **Method:** exact second partial derivatives summed
- **Assumes:** f twice differentiable at the point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The Laplacian of f(x, y) = sin(x)*sin(y) (chosen) on $[-3, 3]^2$, computed by the package at 225 points: accent where $\nabla^2 f > 0$ (f below the average of its surroundings, a hollow), violet where $\nabla^2 f < 0$ (a hump); the grey lines are level curves of f. Marked: $\nabla^2 f$ at (π/2, π/2) = -2. Worked example: ∇^2(x^2 + y^2) at (1, 1) = 4.

```
calculus_ai999x.solve('laplacian', f='x^2 + y^2', p='1, 1')
# -> laplacian = 4   (SOLVED)
```

```
calculus_ai999x.solve('laplacian', f='x^2*y + z^3', p=[1, 2, 3])
# -> laplacian = 22   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Laplacian of x^2*y + z^3 at (1, 2, 3).	SOLVED	laplacian = 22
Find the Laplacian of x^2 + y^2 at the point (1, 1).	SOLVED	laplacian = 4

Divergence, `divergence`

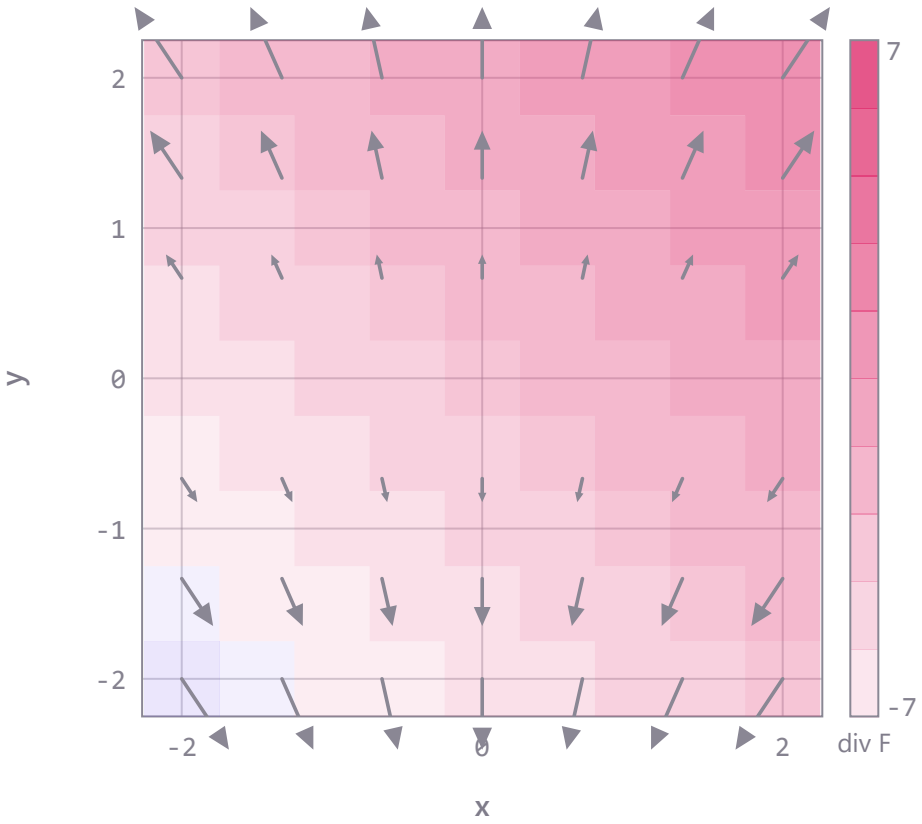
The divergence P_x + Q_y + R_z of a vector field F = (P, Q, R) at a point.

$$\nabla \cdot \mathbf{F} = P_x + Q_y + R_z$$

input	meaning	condition
`P`	the x-component P(x, y, z)	
`Q`	the y-component Q(x, y, z)	
`R`	the z-component R(x, y, z)	
`p`	the point (x, y, z)	

output	meaning
`divergence`	div F at the point

- **Method:** exact symbolic partial derivatives
- **Assumes:** the components are differentiable at the point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The divergence of F = (x*y, y*z, x*z) in the plane z = 3, computed by the package at 81 points (accent: a source, violet: a sink), with the field's x and y components as arrows (computed by the package, z written in). Worked example: div F at (1, 2, 3) = 6.

```
calculus_ai999x.solve('divergence', P='x*y', Q='y*z', R='x*z', p='1, 2, 3')
# -> divergence = 6   (SOLVED)
```

```
calculus_ai999x.solve('divergence', P='x^2', Q='y^2', R='z^2', p=[1, 1, 1])
# -> divergence = 6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the divergence of F = (x*y, y*z, x*z) at (1, 2, 3).	SOLVED	divergence = 6
Find the divergence of the vector field F = (x^2, y^2, z^2) at the point (1, 1, 1).	SOLVED	divergence = 6

Curl, `curl`

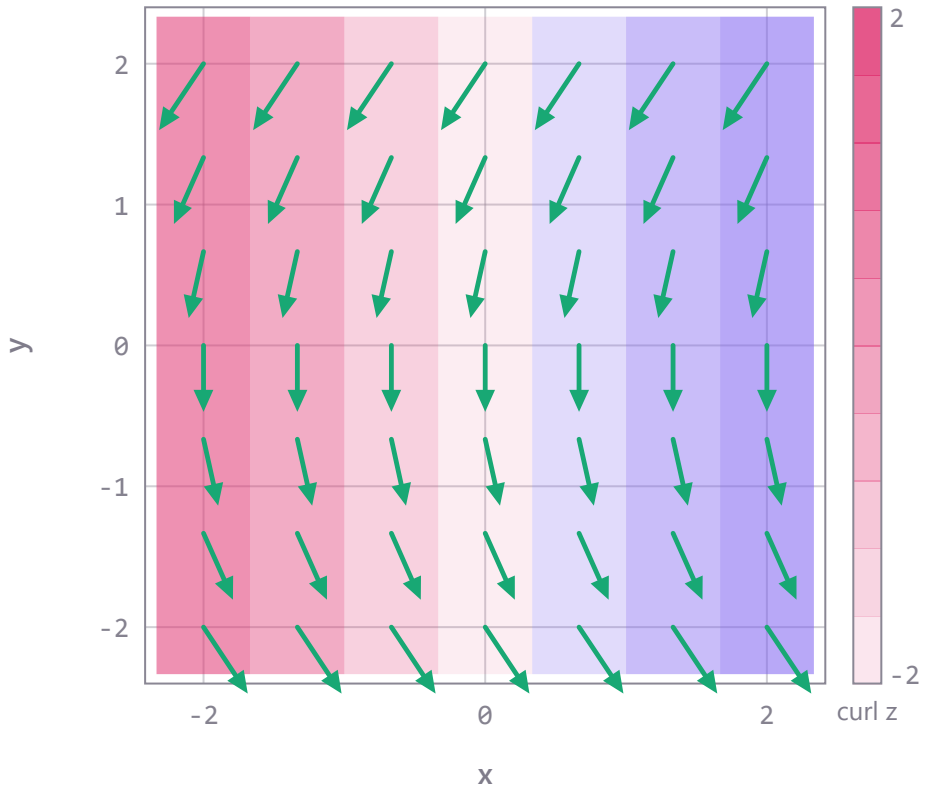
The curl of a vector field F = (P, Q, R) at a point.

$$\nabla \times \mathbf{F} = (R_y - Q_z,\ P_z - R_x,\ Q_x - P_y)$$

input	meaning	condition
`P`	the x-component P(x, y, z)	
`Q`	the y-component Q(x, y, z)	
`R`	the z-component R(x, y, z)	
`p`	the point (x, y, z)	

output	meaning
`curl`	curl F at the point

- **Method:** exact symbolic partial derivatives: (R_y - Q_z, P_z - R_x, Q_x - P_y)
- **Assumes:** the components are differentiable at the point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The curl of F = (x*y, y*z, z*x) in the plane z = 3, computed by the package at 49 points: arrows show its x and y components (common scale), shading its z component (accent positive, violet negative). Worked example: curl at (1, 2, 3) = (-2, -3, -1).

```
calculus_ai999x.solve('curl', P='x*y', Q='y*z', R='z*x', p='1, 2, 3')
# -> curl = [-2.0, -3.0, -1.0]   (SOLVED)
```

```
calculus_ai999x.solve('curl', P='-y', Q='x', R='z', p=[1, 2, 3])
# -> curl = [0.0, 0.0, 2.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the curl of F = (x*y, y*z, z*x) at (1, 2, 3).	SOLVED	curl = [-2.0, -3.0, -1.0]

question	outcome	result
Find the curl of the vector field F = (-y, x, z) at the point (1, 2, 3).	SOLVED	curl = [0.0, 0.0, 2.0]

Jacobian determinant, `jacobian_determinant`

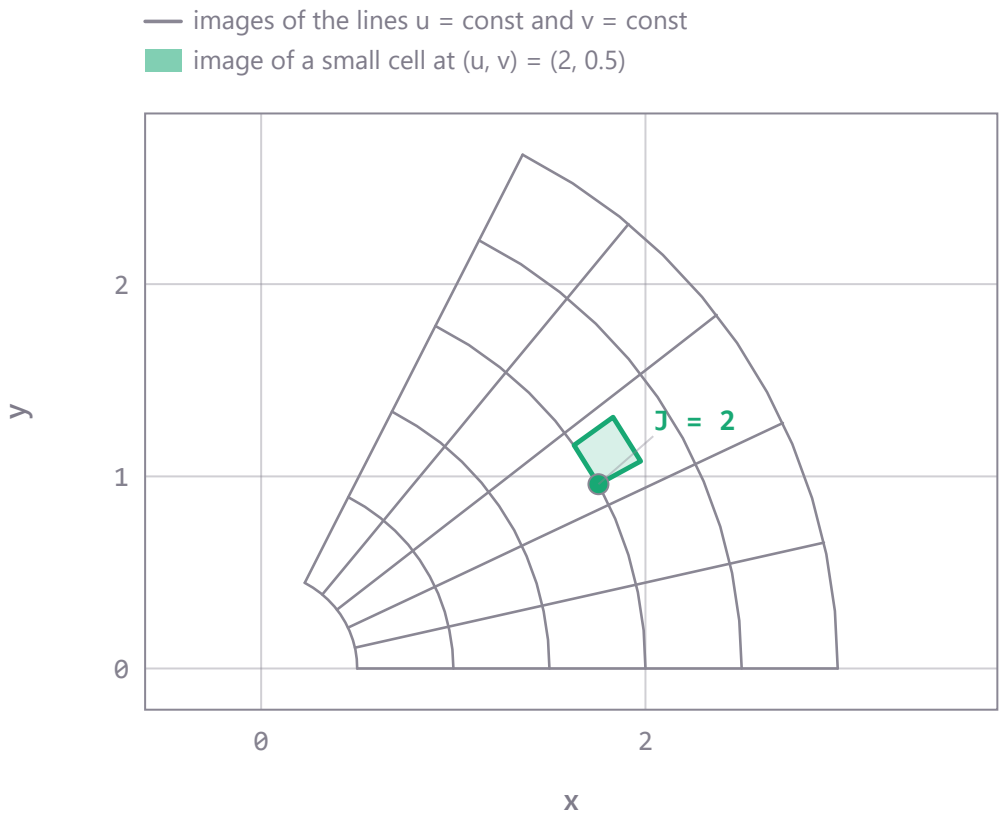
The Jacobian determinant d(x, y)/d(u, v) of a transformation $x = x$(u, v), $y = y$(u, v) at a point.

$$\frac{\partial(x, y)}{\partial(u, v)} = x_u y_v - x_v y_u$$

input	meaning	condition
`xu`	x as a function of u and v	
`yv`	y as a function of u and v	
`p`	the point (u, v)	

output	meaning
`jacobian`	the Jacobian determinant at the point

- **Method:** exact partial derivatives; x_u y_v - x_v y_u
- **Assumes:** x and y differentiable at the point
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The map x = u*cos(v), y = u*sin(v) applied to a grid in the (u, v) plane (images computed by the package, u and v written as x and y in the expressions): the image of a small du × dv cell at (2, 0.5) has area about |J| du dv with the package's Jacobian determinant J = 2 (marked).

```
calculus_ai999x.solve('jacobian_determinant', xu='u*cos(v)', yv='u*sin(v)', p='2, 0.5')
# -> jacobian = 2   (SOLVED)
```

```
calculus_ai999x.solve('jacobian_determinant', xu='u + v', yv='u - v', p=[1, 1])
# -> jacobian = -2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Jacobian of x = u*cos(v), y = u*sin(v) at (2, 0.5).	SOLVED	jacobian = 2
Find the Jacobian determinant of the transformation x = u + v, y = u - v at the point (1, 1).	SOLVED	jacobian = -2

Double integral over a rectangle, `double_integral_rectangle`

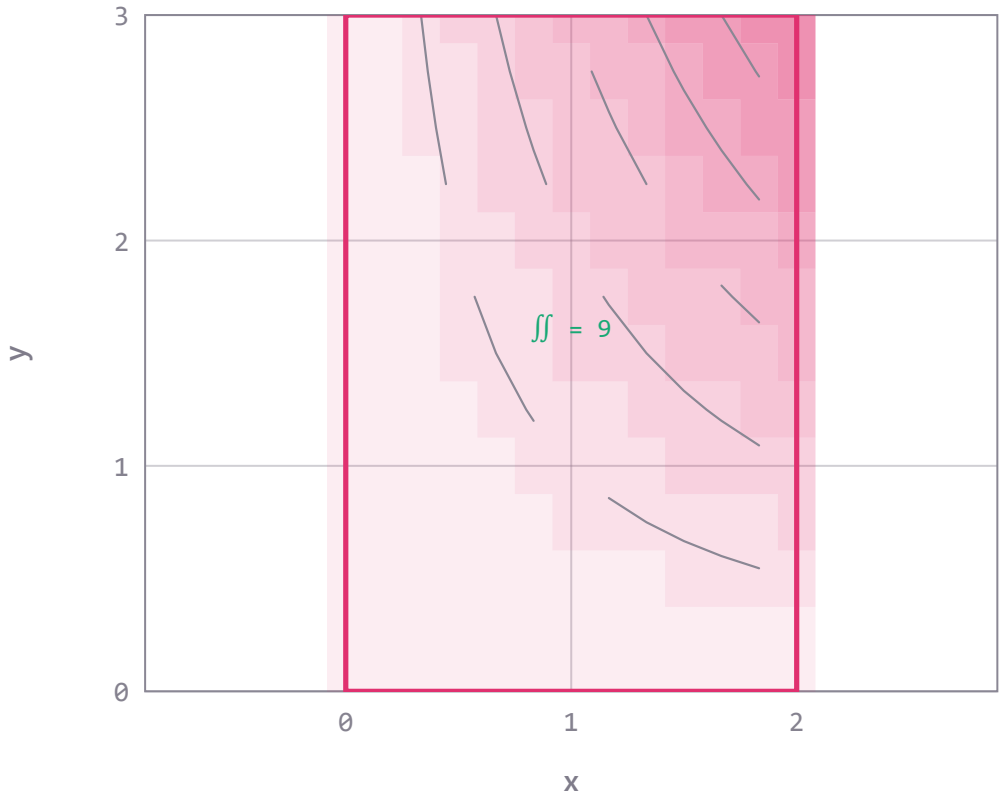
The double integral of f(x, y) over the rectangle [a, b] x [c, d].

$$\int_a^b \int_c^d f(x, y)\, dy\, dx$$

input	meaning	condition
`f`	the integrand f(x, y)	
`a`	x from	
`b`	x to	
`c`	y from	
`d`	y to	

output	meaning
`integral`	the value of the double integral

- **Method:** product Gauss-Legendre rules of increasing order (24, 40, 64 nodes per direction) until two agree to $1e-12$
- **Assumes:** $a < b$, $c < d$; f continuous on the rectangle
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The integrand f(x, y) = x*y over the rectangle [0, 2] × [0, 3], shaded by value with level curves (computed by the package): the double integral, the volume under the surface, is 9.

```
calculus_ai999x.solve('double_integral_rectangle', f='x*y', a=0, b=2, c=0, d=3)
# -> integral = 9   (SOLVED)
```

```
calculus_ai999x.solve('double_integral_rectangle', f='x^2 + y', a=0, b=1, c=0, d=2)
# -> integral = 2.66667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the double integral of x*y over [0, 2] x [0, 3].	SOLVED	integral = 9
Evaluate the double integral of x^2 + y over the rectangle [0, 1] x [0, 2].	SOLVED	integral = 2.66667
Find the double integral of x*y over [2, 0] x [0, 3].	INVALID_INPUT	Double integral over a rectangle: the rectangle needs a < b and c < d.

Double integral over a region, `double_integral_region`

The double integral of f(x, y) over the region $a \le x \le b$, g1(x) <= y <= g2(x).

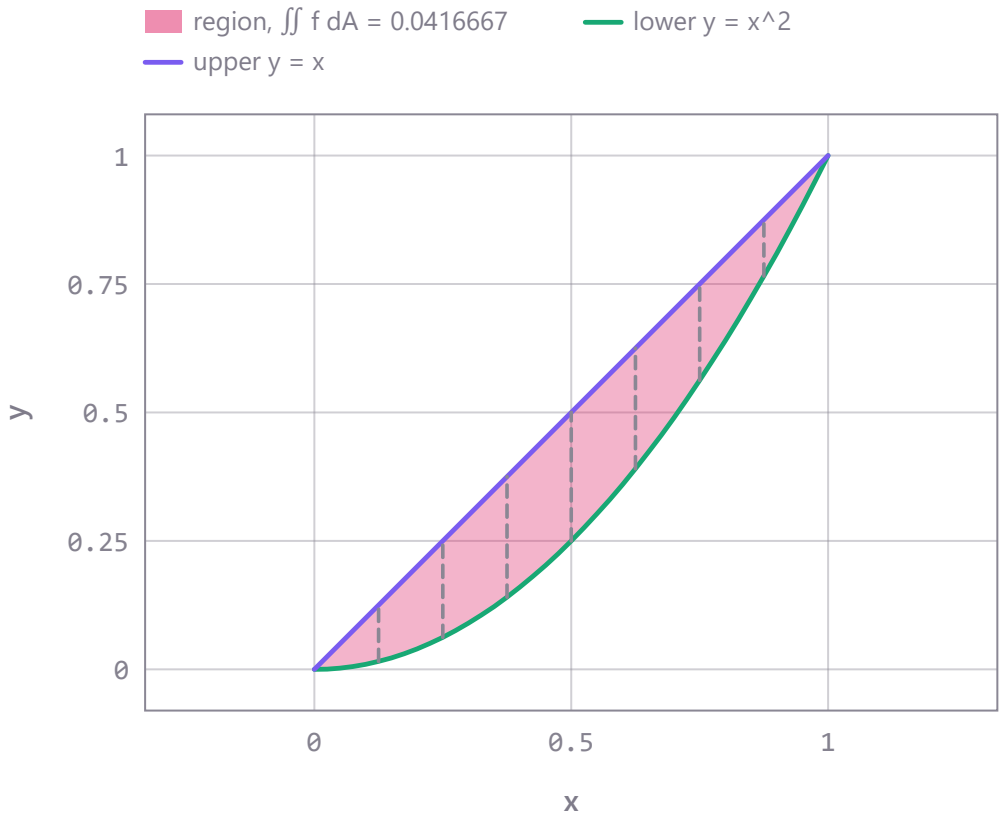
$$\int_a^b \int_{g_1(x)}^{g_2(x)} f(x,y)\,dy\,dx$$

input	meaning	condition
`f`	the integrand f(x, y)	

input	meaning	condition
`a`	x from	
`b`	x to	
`g1`	the lower curve $y = g_1(x)$	
`g2`	the upper curve $y = g_2(x)$	

output	meaning
`integral`	the value of the double integral

- **Method:** iterated Gauss-Legendre rules of increasing order (inner rule mapped to [g1(x), g2(x)]) until two agree
- **Assumes:** $a < b$; $g_1 \le g_2$ on [a, b]; f, g1, g2 continuous
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The region 0 ≤ x ≤ 1, x^2 ≤ y ≤ x (boundary curves computed by the package), with vertical strips marking the inner integration in y: the double integral of f = x*y over it is 0.0416667.

```
calculus_ai999x.solve('double_integral_region', f='x*y', a=0, b=1, g1='x^2', g2='x')
# -> integral = 0.0416667   (SOLVED)
```

```
calculus_ai999x.solve('double_integral_region', f='1', a=0, b=2, g1='0', g2='x^2')
# -> integral = 2.66667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the double integral of x*y over the region 0 <= x <= 1, x^2 <= y <= x.	SOLVED	integral = 0.0416667
Evaluate the double integral of 1 over 0 <= x <= 2, 0 <= y <= x^2.	SOLVED	integral = 2.66667
Find the double integral of x*y over the region 0 <= x <= 1, x <= y <= x^2 - 1.	INVALID_INPUT	Double integral over a region: the lower boundary must lie below the upper boundary on [a, b].

Triple integral over a box, `triple_integral_box`

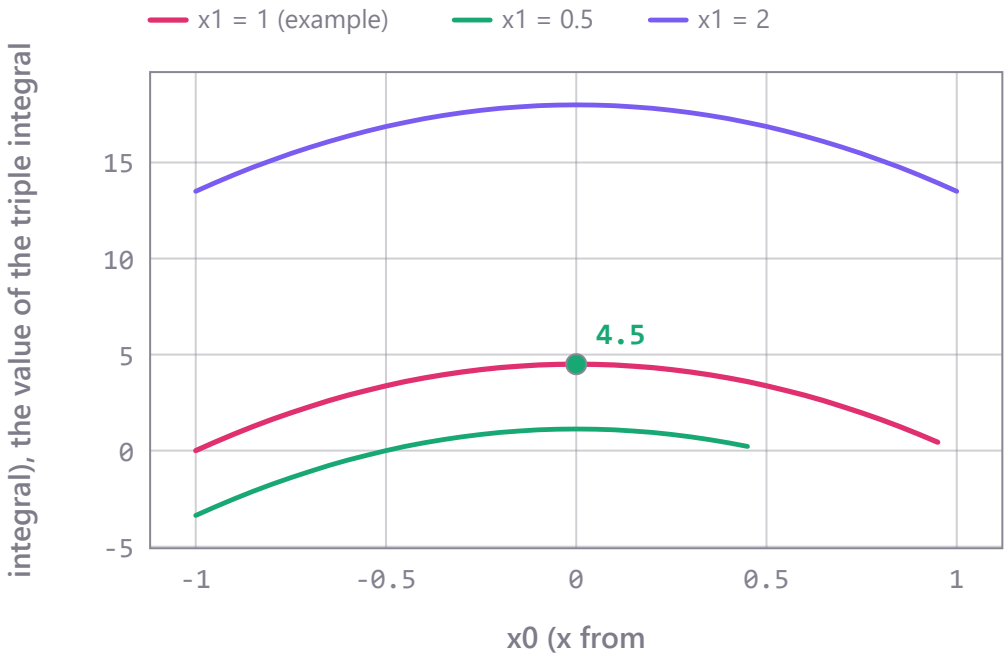
The triple integral of f(x, y, z) over the box [x0, x1] x [y0, y1] x [z0, z1].

$$\iiint_B f(x, y, z)\, dV$$

input	meaning	condition
`f`	the integrand f(x, y, z)	
`x0`	x from	
`x1`	x to	
`y0`	y from	
`y1`	y to	
`z0`	z from	
`z1`	z to	

output	meaning
`integral`	the value of the triple integral

- **Method:** exact iterated integration for a polynomial integrand; otherwise product Gauss-Legendre rules of increasing order until two agree
- **Assumes:** $x_0 < x_1$, $y_0 < y_1$, $z_0 < z_1$; f continuous on the box
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



integral (the value of the triple integral) against x0 (x from) from -1 to 1. One curve for each value of x1 (x to): 1, 0.5, 2. Other inputs as in the worked example: f = x*y*z, y0 = 0, y1 = 2, z0 = 0, z1 = 3. 40 points per curve computed by calculus-ai999x; marked: the worked example, x0 = 0, integral = 4.5.

```
calculus_ai999x.solve('triple_integral_box', f='x*y*z', x0=0, x1=1, y0=0, y1=2, z0=0, z1=3)
# -> integral = 4.5   (SOLVED)
```

```
calculus_ai999x.solve('triple_integral_box', f='x^2 + y^2 + z^2', x0=0, x1=1, y0=0, y1=1, z0=0,
z1=1)
# -> integral = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the triple integral of x*y*z over [0, 1] x [0, 2] x [0, 3].	SOLVED	integral = 4.5
Evaluate the triple integral of x^2 + y^2 + z^2 over the box [0, 1] x [0, 1] x [0, 1].	SOLVED	integral = 1

Surface area of a graph, `surface_area_graph`

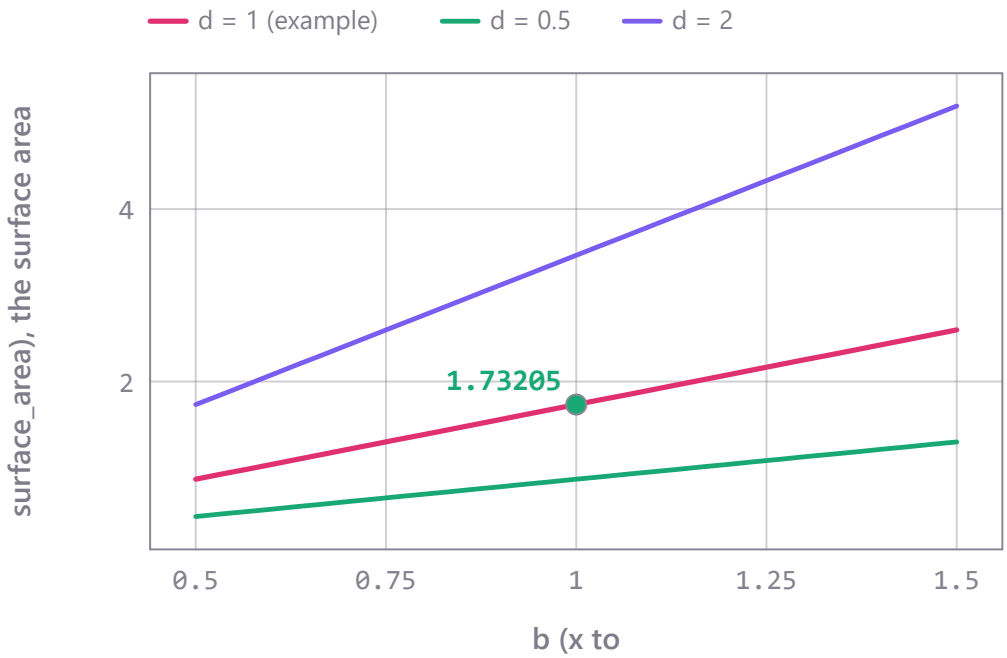
The area of the surface $z = f(x, y)$ over the rectangle [a, b] x [c, d].

$$A = \iint_R \sqrt{1 + f_x^2 + f_y^2}\, dA$$

input	meaning	condition
`f`	the surface $z = f(x, y)$	
`a`	x from	
`b`	x to	
`c`	y from	
`d`	y to	

output	meaning
`surface_area`	the surface area

- **Method:** exact partial derivatives; Gauss-Legendre product rules of sqrt(1 + f_x^2 + f_y^2) until two orders agree
- **Assumes:** $a < b$, $c < d$; f continuously differentiable on the rectangle
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



surface_area (the surface area) against b (x to) from 0.5 to 1.5. One curve for each value of d (y to): 1, 0.5, 2. Other inputs as in the worked example: f = x + y, a = 0, c = 0. 31 points per curve computed by calculus-ai999x; marked: the worked example, b = 1, surface_area = 1.73205.

```
calculus_ai999x.solve('surface_area_graph', f='x + y', a=0, b=1, c=0, d=1)
# -> surface_area = 1.73205   (SOLVED)
```

```
calculus_ai999x.solve('surface_area_graph', f='3', a=0, b=2, c=0, d=2)
# -> surface_area = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the surface area of z = x + y over [0, 1] x [0, 1].	SOLVED	surface_area = 1.73205
Find the area of the surface z = 2x + 2y over the rectangle [0, 1] x [0, 2].	SOLVED	surface_area = 6

Line integral of a vector field, `line_integral_vector`

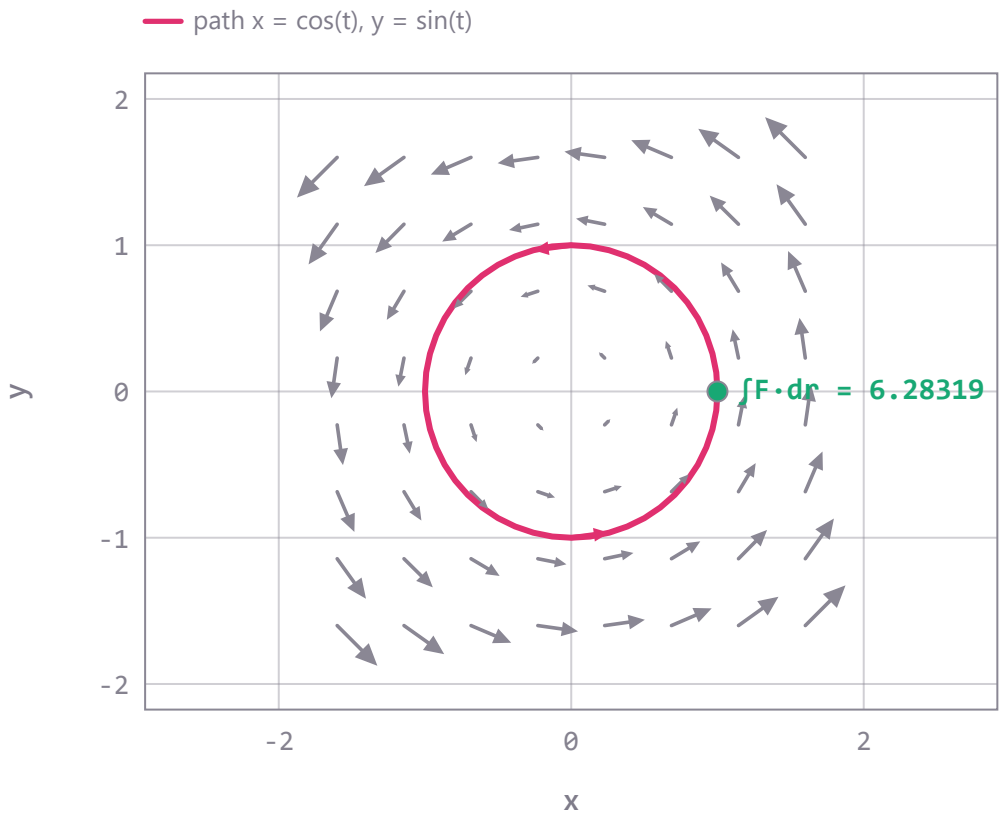
The line integral (work) of F = (P, Q) along the path $x = x(t)$, $y = y(t)$ from t0 to t1.

$$\int_C \mathbf{F} \cdot d\mathbf{r} = \int_{t_0}^{t_1} \big(P\,x'(t) + Q\,y'(t)\big)\,dt$$

input	meaning	condition
`P`	the x-component P(x, y)	
`Q`	the y-component Q(x, y)	
`xt`	the path x(t)	
`yt`	the path y(t)	
`t0`	start parameter	
`t1`	end parameter	

output	meaning
`line_integral`	the integral of F . dr along the path

- **Method:** F(r(t)) . r'(t) built exactly (symbolic r'), then tanh-sinh quadrature at 20 digits
- **Assumes:** the integrand is continuous for t between t0 and t1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The field F = (-y, x) (arrows, computed by the package) and the path x = cos(t), y = sin(t) for t from 0 to 6.28319 (points computed by the package with t written as x): where the path runs along the arrows the work is positive; the line integral is 6.28319 (marked at the start).

```
calculus_ai999x.solve('line_integral_vector', P='-y', Q='x', xt='cos(t)', yt='sin(t)', t0=0,
t1=6.283185307179586)
# -> line_integral = 6.28319   (SOLVED)
```

```
calculus_ai999x.solve('line_integral_vector', P='y', Q='x', xt='t', yt='t^2', t0=0, t1=1)
# -> line_integral = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the line integral of F = (-y, x) along x = cos(t), y = sin(t) from t = 0 to t = 2pi.	SOLVED	line_integral = 6.28319
Compute the line integral of the vector field F = (y, x) along the curve x = t, y = t^2 from t = 0 to t = 1.	SOLVED	line_integral = 1

Line integral with respect to arc length, `line_integral_scalar`

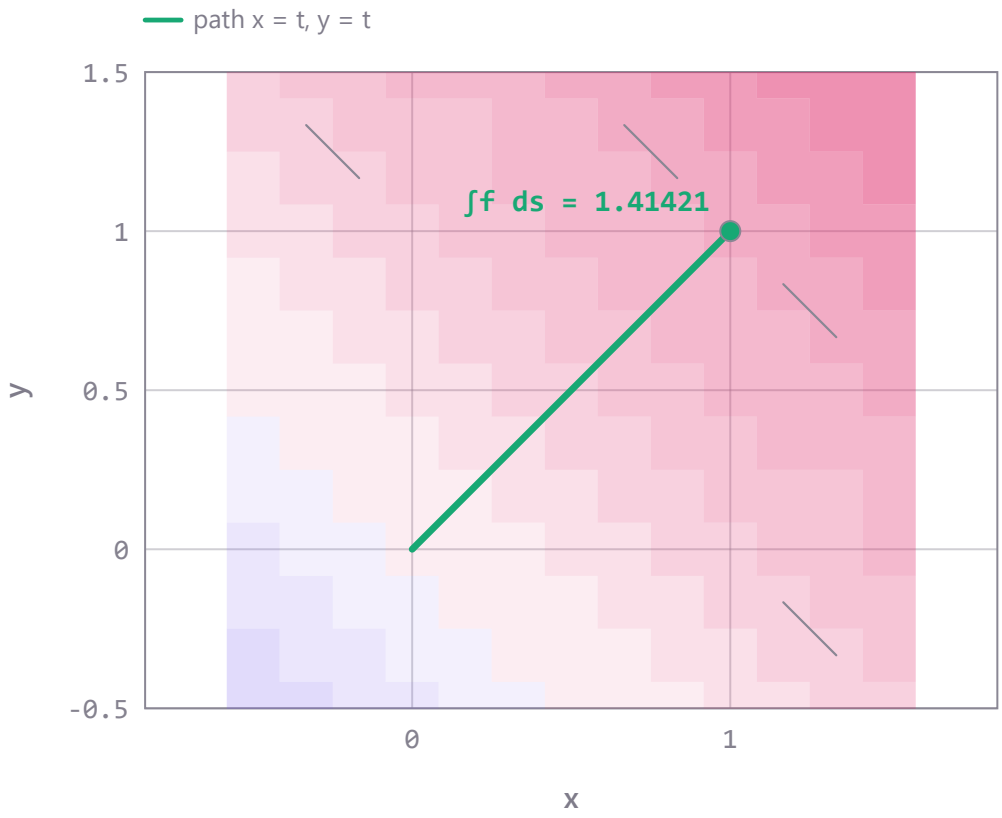
The integral of f(x, y) ds along the path $x = x$(t), $y = y$(t), $t_0 \le t \le t_1$ (mass of a wire, average along a curve).

$$\int_C f\,ds = \int_{t_0}^{t_1} f(x(t), y(t)) \sqrt{x'(t)^2 + y'(t)^2}\,dt$$

input	meaning	condition
`f`	the function f(x, y)	
`xt`	the path x(t)	
`yt`	the path y(t)	
`t0`	start parameter	
`t1`	end parameter	

output	meaning
`line_integral`	the integral of f ds along the path

- **Method:** f(r(t)) |r'(t)| with exact r'(t), then tanh-sinh quadrature
- **Assumes:** $t_0 < t_1$; the integrand continuous on [t0, t1]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Level curves of f(x, y) = x + y (computed by the package) and the path x = t, y = t, t from 0 to 1 (points computed by the package with t written as x): the line integral of f with respect to arc length, 1.41421, sums f along the path (marked at its end).

```
calculus_ai999x.solve('line_integral_scalar', f='x + y', xt='t', yt='t', t0=0, t1=1)
# -> line_integral = 1.41421   (SOLVED)
```

```
calculus_ai999x.solve('line_integral_scalar', f='1', xt='cos(t)', yt='sin(t)', t0=0,
t1=3.141592653589793)
# -> line_integral = 3.14159   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the line integral of x + y ds along x = t, y = t from t = 0 to t = 1.	SOLVED	line_integral = 1.41421
Find the line integral of x + y ds along x = t, y = t from t = 1 to t = 0.	INVALID_INPUT	Line integral with respect to arc length: the parameter must run forward (t0 < t1) for an integral with respect to arc l

Green's theorem on a rectangle, `greens_theorem`

Both sides of Green's theorem for F = (P, Q) on the rectangle [a, b] x [c, d]: the counter-clockwise circulation around the boundary and the double integral of Q_x - P_y.

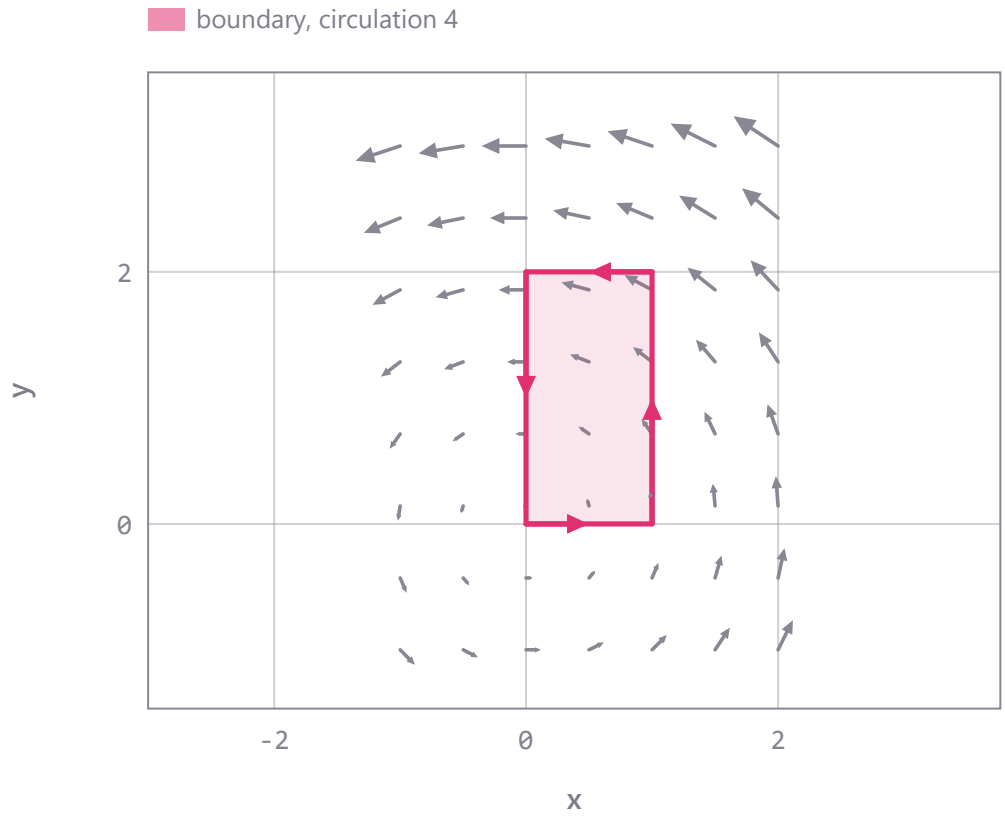
$$\oint_C P\,dx + Q\,dy = \iint_R (Q_x - P_y)\,dA$$

input	meaning	condition
`P`	the x-component P(x, y)	

input	meaning	condition
`Q`	the y-component Q(x, y)	
`a`	x from	
`b`	x to	
`c`	y from	
`d`	y to	

output	meaning
`circulation`	the boundary line integral (counter-clockwise)
`double_integral`	the integral of Q_x - P_y

- **Method:** the four sides by tanh-sinh quadrature; the double integral of the exact Q_x - P_y by Gauss-Legendre product rules; refused unless the two agree
- **Assumes:** $a < b$, $c < d$; P and Q continuously differentiable on the rectangle
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The field F = (-y, x) (arrows of equal scale, components computed by the package) and the rectangle [0, 1] × [0, 2] traversed counter-clockwise: Green's theorem equates the circulation around it, 4, with the double integral of ∂Q/∂x − ∂P/∂y over it, 4, both from the package.

```
calculus_ai999x.solve('greens_theorem', P='-y', Q='x', a=0, b=1, c=0, d=2)
# -> circulation = 4, double_integral = 4   (SOLVED)
```

```
calculus_ai999x.solve('greens_theorem', P='x*y', Q='x^2', a=0, b=1, c=0, d=1)
# -> circulation = 0.5, double_integral = 0.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Verify Green's theorem for F = (-y, x) on the rectangle [0, 1] x [0, 2].	SOLVED	circulation = 4, double_integral = 4
Use Green's theorem for F = (x*y, x^2) on [0, 1] x [0, 1].	SOLVED	circulation = 0.5, double_integral = 0.5

Outward flux through a box (divergence theorem), `divergence_theorem_box`

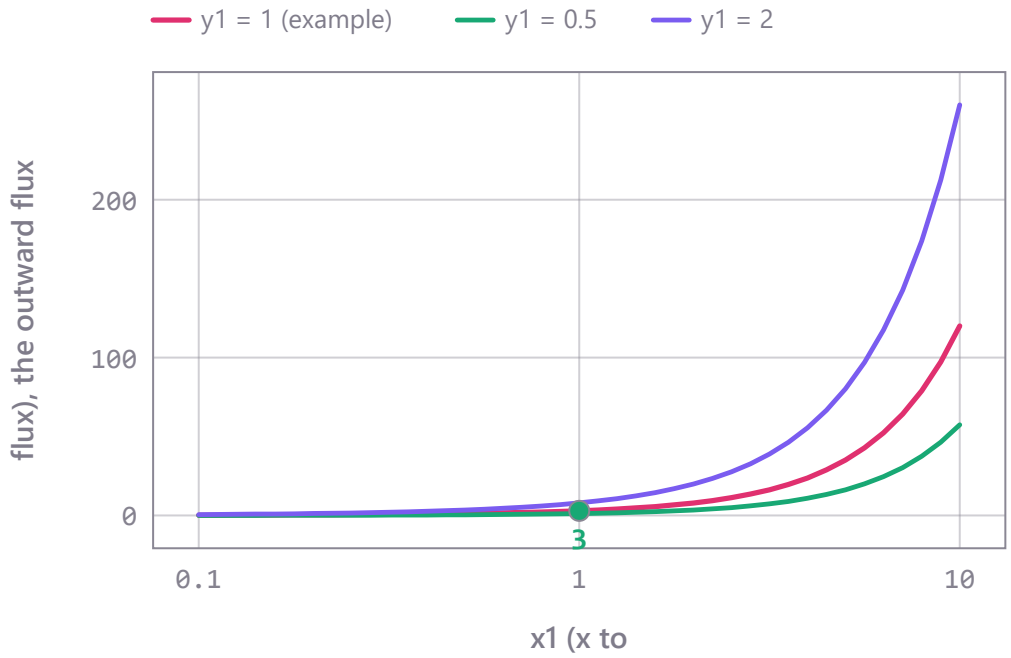
The outward flux of F = (P, Q, R) through the surface of the box [x0, x1] x [y0, y1] x [z0, z1].

$$\oiint_S \mathbf{F} \cdot \mathbf{n}\, dS = \iiint_B \nabla \cdot \mathbf{F}\, dV$$

input	meaning	condition
`P`	the x-component	
`Q`	the y-component	
`R`	the z-component	
`x0`	x from	
`x1`	x to	
`y0`	y from	
`y1`	y to	
`z0`	z from	
`z1`	z to	

output	meaning
`flux`	the outward flux

- **Method:** the divergence theorem: the triple integral of the exact divergence (exact for polynomials, otherwise Gauss-Legendre product rules until two orders agree)
- **Assumes:** the box is non-degenerate; F continuously differentiable on the box
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



flux (the outward flux) against x1 (x to) from 0.1 to 10 on a logarithmic axis. One curve for each value of y1 (y to): 1, 0.5, 2. Other inputs as in the worked example: P = x^2, Q = y^2, R = z^2, x0 = 0, y0 = 0, z0 = 0, z1 = 1. 41 points per curve computed by calculus-ai999x; marked: the worked example, x1 = 1, flux = 3.

```
calculus_ai999x.solve('divergence_theorem_box', P='x^2', Q='y^2', R='z^2', x0=0, x1=1, y0=0, y1=1,
z0=0, z1=1)
# -> flux = 3   (SOLVED)
```

```
calculus_ai999x.solve('divergence_theorem_box', P='x', Q='y', R='z', x0=0, x1=1, y0=0, y1=2, z0=0,
z1=3)
# -> flux = 18   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the outward flux of F = (x^2, y^2, z^2) through the box [0, 1] x [0, 1] x [0, 1].	SOLVED	flux = 3
Find the flux of the field F = (x, y, z) through the box [0, 1] x [0, 2] x [0, 3].	SOLVED	flux = 18

Arc length of a parametric curve, `parametric_arc_length`

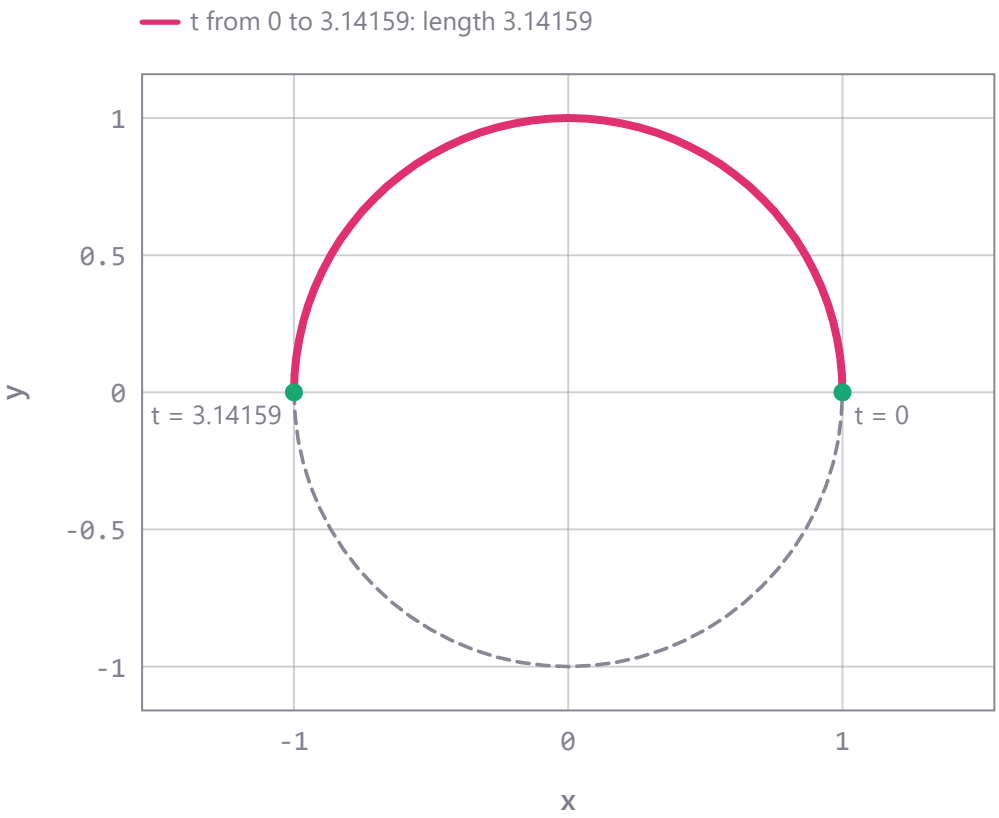
The length of the curve $x = x(t)$, $y = y(t)$ for $t_0 \le t \le t_1$.

$$L = \int_{t_0}^{t_1} \sqrt{x'(t)^2 + y'(t)^2}\, dt$$

input	meaning	condition
`xt`	x(t)	
`yt`	y(t)	
`t0`	start parameter	
`t1`	end parameter	

output	meaning
`arc_length`	the length of the curve

- **Method:** sqrt(x'$(t)^2 + y$'(t)^2) with exact derivatives, tanh-sinh quadrature
- **Assumes:** $t_0 < t_1$; x and y continuously differentiable
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The curve x = cos(t), y = sin(t) (coordinates computed by the package with t written as x), the arc from t = 0 to t = 3.14159 drawn thick and the rest of one turn dashed: arc length 3.14159 from the package.

```
calculus_ai999x.solve('parametric_arc_length', xt='cos(t)', yt='sin(t)', t0=0,
t1=3.141592653589793)
# -> arc_length = 3.14159   (SOLVED)
```

```
calculus_ai999x.solve('parametric_arc_length', xt='3t', yt='4t', t0=0, t1=2)
# -> arc_length = 10   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the arc length of the parametric curve x = cos(t), y = sin(t) from t = 0 to t = pi.	SOLVED	arc_length = 3.14159
Find the length of the curve x = 3t, y = 4t from t = 0 to t = 2.	SOLVED	arc_length = 10

Slope and concavity of a parametric curve, `parametric_slope`

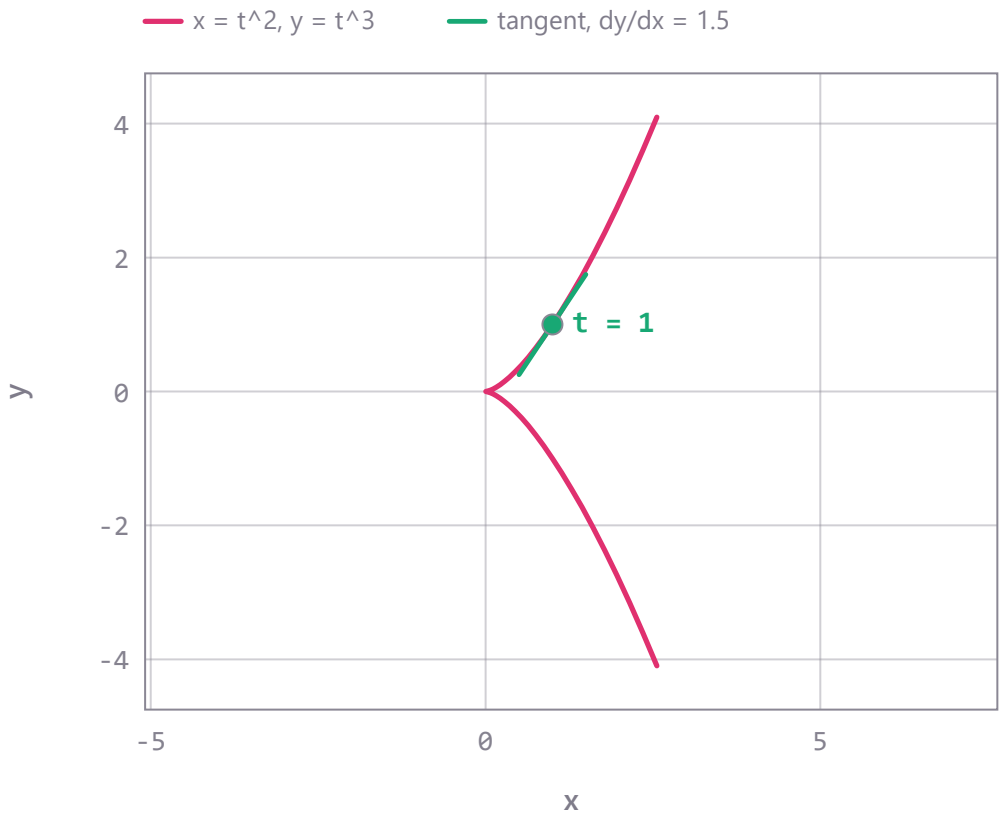
dy/dx and d^{2y}/dx^2 of the curve $x = x$(t), $y = y$(t) at a parameter value t0.

$$\frac{dy}{dx} = \frac{y'(t)}{x'(t)}, \quad \frac{d^2y}{dx^2} = \frac{\frac{d}{dt}\frac{dy}{dx}}{x'(t)}$$

input	meaning	condition
`xt`	x(t)	
`yt`	y(t)	
`t0`	the parameter value	

output	meaning
`dy_dx`	dy/dx at t0
`d2y_dx2`	d^2y/dx^2 at t0

- **Method:** $dy/dx = y'(t)/x'(t)$ and d^2y/dx^2 = (d/dt of dy/dx)/x'(t), exact derivatives
- **Assumes:** $dx/dt \neq 0$ at t0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The parametric curve x = t^2, y = t^3 for t from −1.6 to 1.6 (each coordinate computed by the package with t written as x in the expression) and its tangent at t = 1, point (1, 1): slope dy/dx = 1.5, d^2y/dx^2 = 0.75. Marked: the point.

```
calculus_ai999x.solve('parametric_slope', xt='t^2', yt='t^3', t0=1)
# -> dy_dx = 1.5, d2y_dx2 = 0.75   (SOLVED)
```

```
calculus_ai999x.solve('parametric_slope', xt='cos(t)', yt='sin(t)', t0=0.7853981633974483)
# -> dy_dx = -1, d2y_dx2 = -2.82843   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find dy/dx for x = t^2, y = t^3 at t = 1.	SOLVED	dy_dx = 1.5, d2y_dx2 = 0.75
Find dy/dx for x = t^2, y = t^3 at t = 0.	INVALID_INPUT	Slope and concavity of a parametric curve: dx/dt = 0 at this parameter: the tangent is vertical (dy/dx undefined).

Area in polar coordinates, `polar_area`

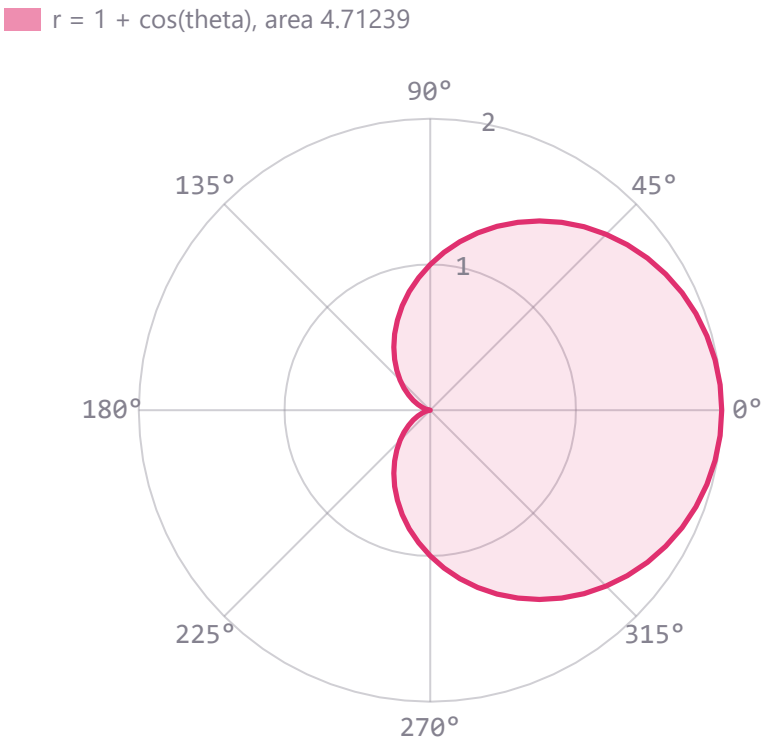
The area swept by the polar curve $r = r$(theta) between $\theta = \alpha$ and $\theta = \beta$.

$$A = \frac{1}{2}\int_{\alpha}^{\beta} r(\theta)^2 \, d\theta$$

input	meaning	condition
`r`	the polar curve r(theta)	
`al`	alpha, the start angle (radians)	
`be`	beta, the end angle (radians)	

output	meaning
`area`	the area

- **Method:** $(1/2)$ integral of r^2 by tanh-sinh quadrature
- **Assumes:** $\alpha < \beta \leq \alpha + 2$ pi; r continuous
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The polar curve r = 1 + cos(theta) for θ from 0 to 6.28319 (r computed by the package at 73 angles, θ written as x in the expression) and the region it encloses: area 4.71239, the integral of $r^2/2\ d\theta$, from the package.

```
calculus_ai999x.solve('polar_area', r='1 + cos(theta)', al=0, be=6.283185307179586)
# -> area = 4.71239   (SOLVED)
```

```
calculus_ai999x.solve('polar_area', r='2', al=0, be=1.5707963267948966)
# -> area = 3.14159   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the area enclosed by the polar curve r = 1 + cos(theta) from theta = 0 to theta = 2pi.	SOLVED	area = 4.71239
Find the area of the polar region r = 2 from theta = 0 to theta = pi/2.	SOLVED	area = 3.14159
Find the area enclosed by the polar curve r = 1 + cos(theta) from theta = 0 to theta = 4pi.	INVALID_INPUT	Area in polar coordinates: the angle interval may span at most one full turn (2 pi).

Gamma function, `gamma_function`

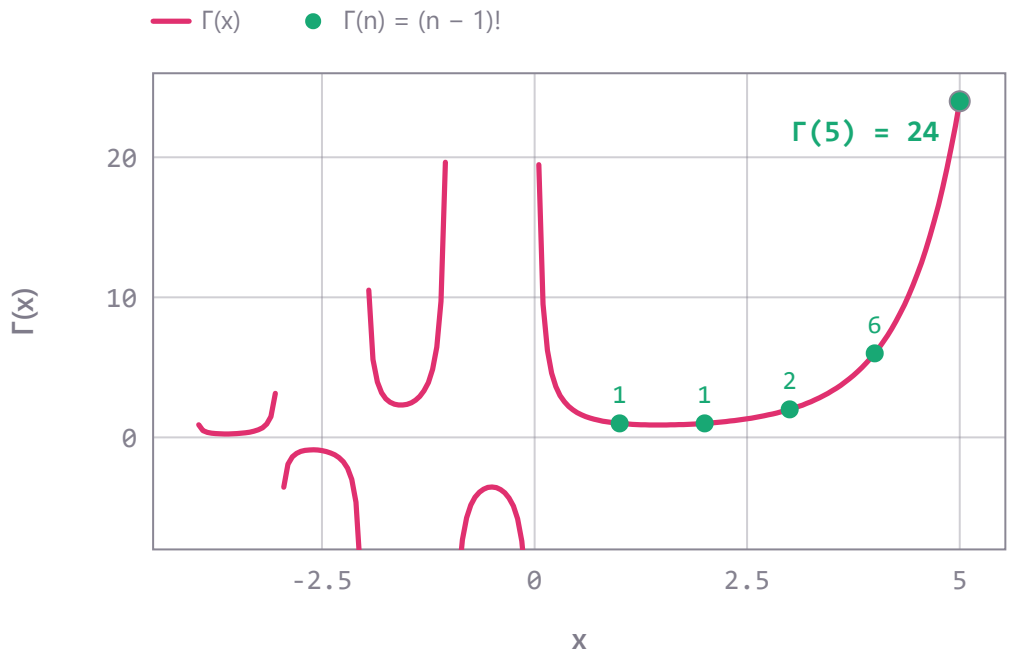
The value of the gamma function (Gamma(n) = (n-1)! for whole n).

$$\Gamma(x) = \int_0^\infty t^{x-1} e^{-t}\,dt$$

input	meaning	condition
`x`	the argument x	

output	meaning
`gamma`	Gamma(x)

- **Method:** the standard library's Lanczos-type approximation (double precision)
- **Assumes:** x is not 0 or a negative whole number; $x < 171.6$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The gamma function for −4 < x ≤ 5, computed by the package at 176 points (poles at 0, −1, −2, −3, where it is not drawn): at the positive integers it is the factorial, Γ(n) = (n − 1)!, dots Γ(1) … Γ(4) = 1, 1, 2, 6. The worked example: Γ(5) = 24 (marked).

```
calculus_ai999x.solve('gamma_function', x=5)
# -> gamma = 24   (SOLVED)
```

```
calculus_ai999x.solve('gamma_function', x=0.5)
# -> gamma = 1.77245   (SOLVED)
```

```
calculus_ai999x.solve('gamma_function', x=4.5)
# -> gamma = 11.6317   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate the gamma function at x = 4.5.	SOLVED	gamma = 11.6317
Find Gamma(5).	SOLVED	gamma = 24
Find Gamma(-2).	INVALID_INPUT	Gamma function: the gamma function is undefined at 0 and negative whole numbers, and overflows beyond x = 171.

Beta function, `beta_function`

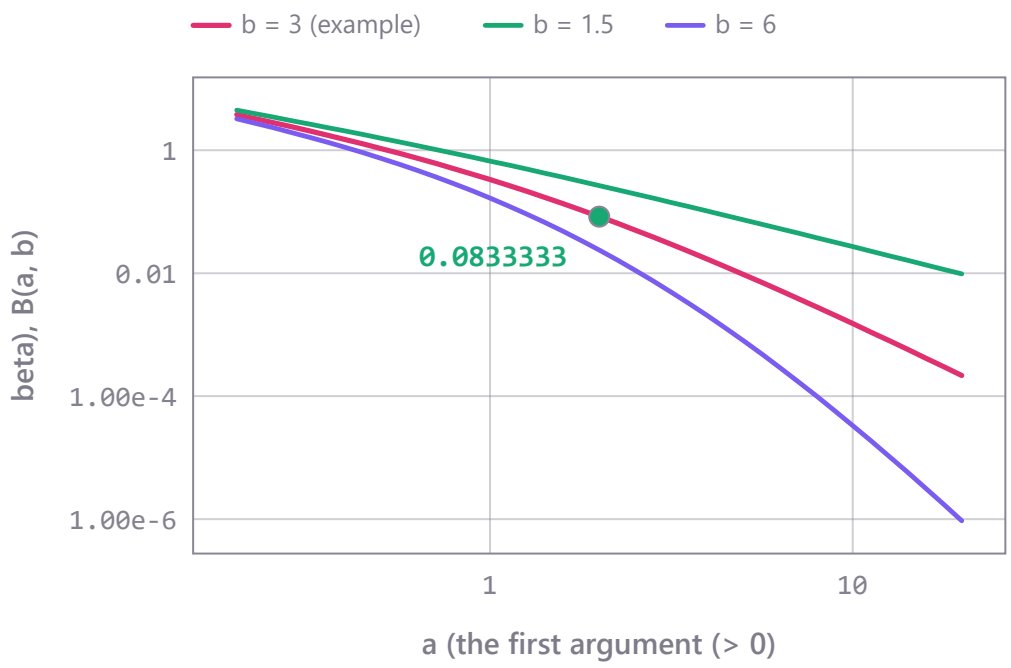
The beta function B(a, b) for positive a and b.

$$B(a,b) = \int_0^1 t^{a-1}(1-t)^{b-1}dt = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)}$$

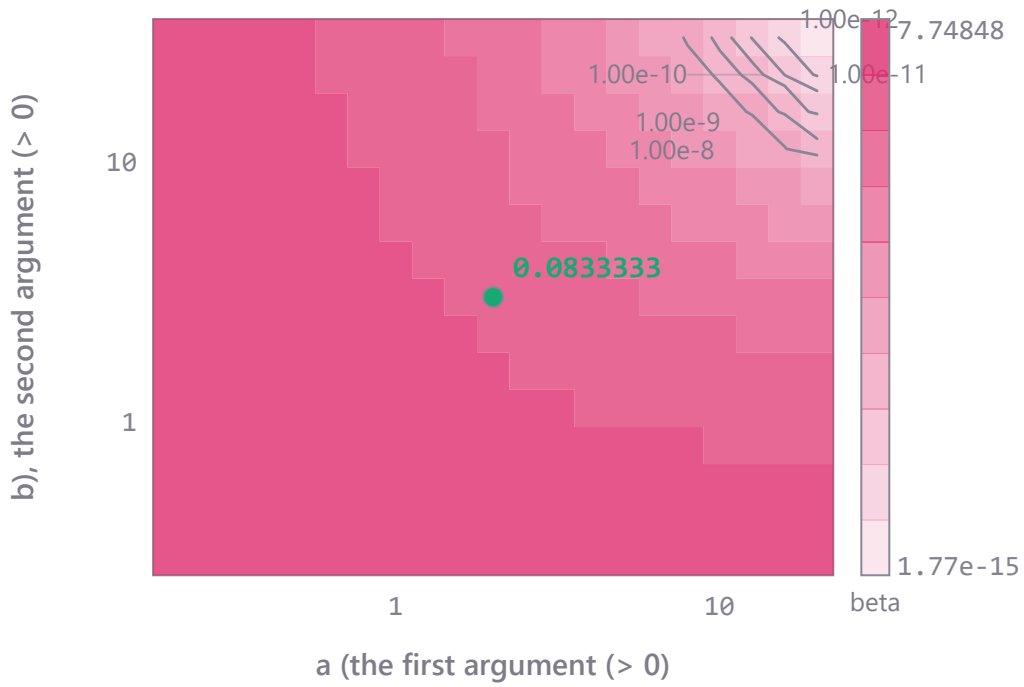
input	meaning	condition
`a`	the first argument (> 0)	
`b`	the second argument (> 0)	

output	meaning
`beta`	B(a, b)

- **Method:** exp(ln Gamma(a) + ln Gamma(b) - ln Gamma(a + b))
- **Assumes:** $a > 0$; $b > 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



beta (B(a, b)) against a (the first argument (> 0)) from 0.2 to 20 on a logarithmic axis, logarithmic output axis. One curve for each value of b (the second argument (> 0)): 3, 1.5, 6. 41 points per curve computed by calculus-ai999x; marked: the worked example, a = 2, beta = 0.0833333.



beta (B(a, b)) over a (the first argument (> 0)) from 0.2 to 20 and b (the second argument (> 0)) from 0.3 to 30 (logarithmic axes): each of the 21 × 15 cells is one run of calculus-ai999x, shaded by its value (light 1.7682e-15 to dark 7.74848, logarithmic shading); iso-lines at beta = 1.00e-12, 1.00e-11, 1.00e-10, 1.00e-9, 1.00e-8. Marked: the worked example, a = 2, b = 3, beta = 0.0833333.

```
calculus_ai999x.solve('beta_function', a=2, b=3)
# -> beta = 0.0833333   (SOLVED)
```

```
calculus_ai999x.solve('beta_function', a=0.5, b=0.5)
# -> beta = 3.14159   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate the beta function B(2, 3).	SOLVED	beta = 0.0833333
Find the beta function B(0.5, 0.5).	SOLVED	beta = 3.14159
Evaluate the beta function B(-1, 3).	INVALID_INPUT	Beta function: this capability needs a > 0 and b > 0.

Error function, `error_function`

The error function erf(x) and the complementary error function erfc(x) = 1 - erf(x).

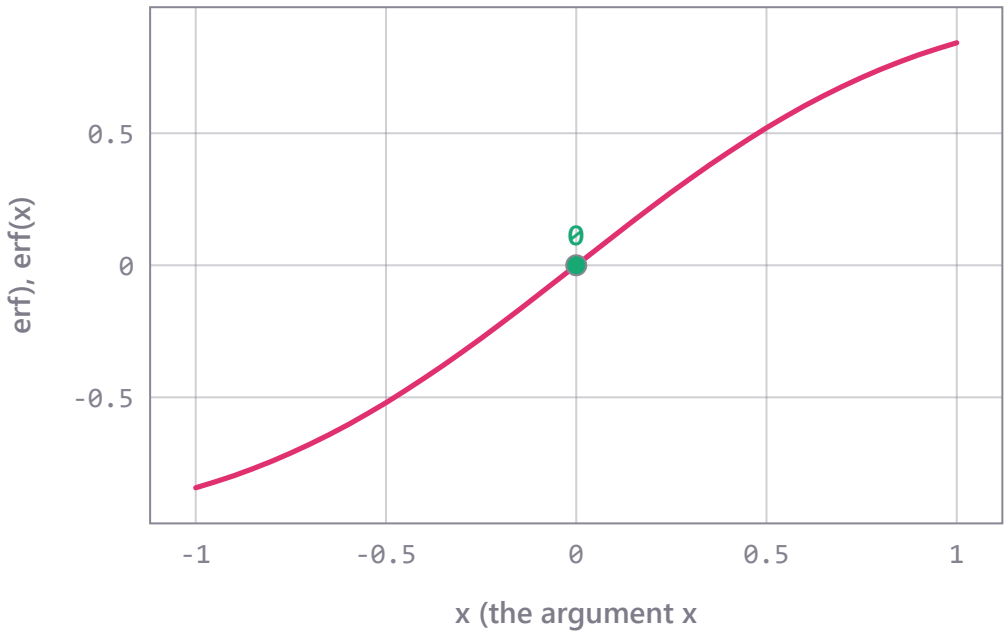
$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt$$

input	meaning	condition
`x`	the argument x	

output	meaning
`erf`	erf(x)

output	meaning
`erfc`	erfc(x)

- **Method:** the standard library's erf and erfc (double precision)
- **Assumes:** x real
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



erf (erf(x)) against x (the argument x) from -1 to 1. 41 points computed by calculus-ai999x; marked: the worked example, x = 0, erf = 0.

```
calculus_ai999x.solve('error_function', x=0)
# -> erf = 0, erfc = 1   (SOLVED)
```

```
calculus_ai999x.solve('error_function', x=0.5)
# -> erf = 0.5205, erfc = 0.4795   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate erf(0.5).	SOLVED	erf = 0.5205, erfc = 0.4795
Find the error function at x = 0.	SOLVED	erf = 0, erfc = 1

Regularized incomplete gamma functions, `regularized_gamma`

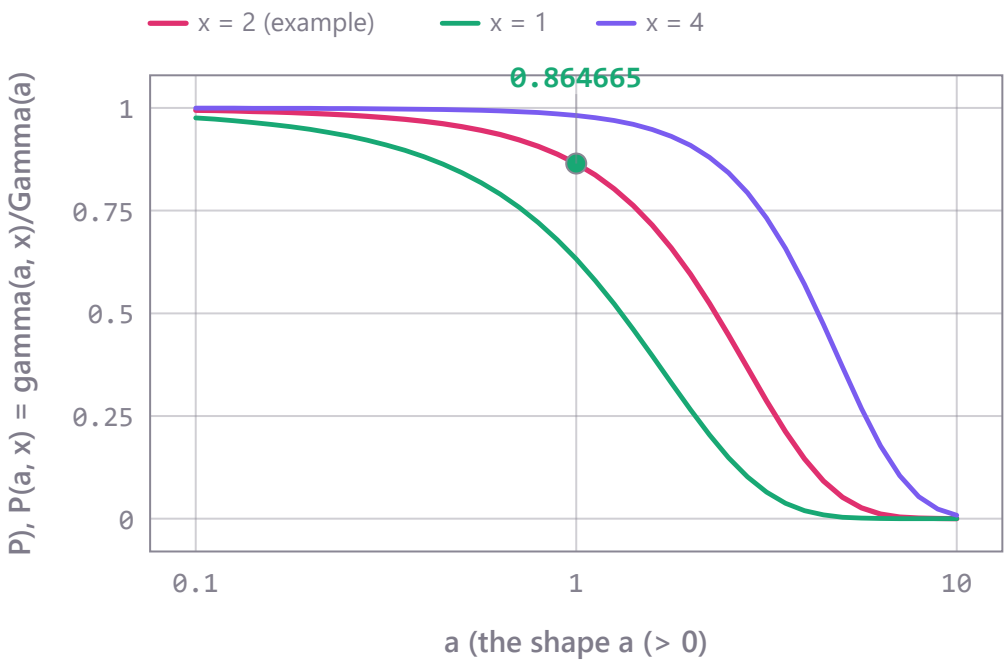
The regularized lower and upper incomplete gamma functions P(a, x) and Q(a, x) = 1 - P(a, x).

$$P(a,x) = \frac{1}{\Gamma(a)} \int_0^x t^{a-1} e^{-t}\, dt$$

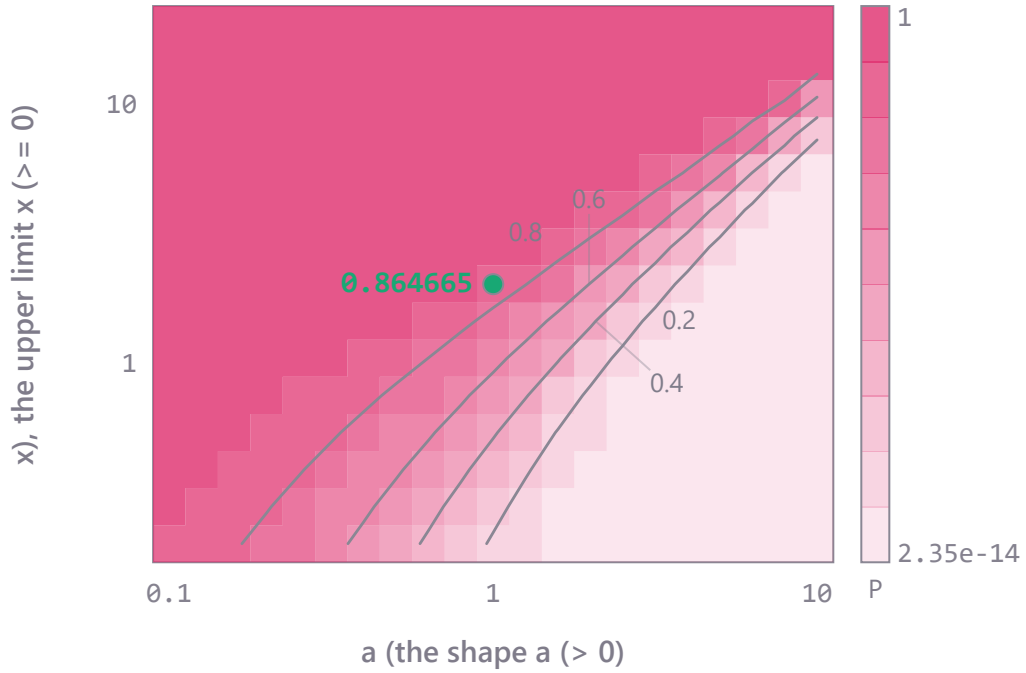
input	meaning	condition
`a`	the shape a (> 0)	
`x`	the upper limit x (>= 0)	

output	meaning
`P`	P(a, x) = gamma(a, x)/Gamma(a)
`Q`	Q(a, x) = Gamma(a, x)/Gamma(a)

- **Method:** power series for $x < a + 1$, Lentz continued fraction otherwise (the smaller of P and Q computed directly)
- **Assumes:** $a > 0$; $x \geq 0$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P (P(a, x) = gamma(a, x)/Gamma(a)) against a (the shape a (> 0)) from 0.1 to 10 on a logarithmic axis. One curve for each value of x (the upper limit x (>= 0)): 2, 1, 4. 41 points per curve computed by calculus-ai999x; marked: the worked example, a = 1, P = 0.864665.



P (P(a, x) = gamma(a, x)/Gamma(a)) over a (the shape a (> 0)) from 0.1 to 10 and x (the upper limit x (>= 0)) from 0.2 to 20 (logarithmic axes): each of the 21 × 15 cells is one run of calculus-ai999x, shaded by its value (light 2.3531e-14 to dark 1); iso-lines at P = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, a = 1, x = 2, P = 0.864665.

```
calculus_ai999x.solve('regularized_gamma', a=1, x=2)
# -> P = 0.864665, Q = 0.135335   (SOLVED)
```

```
calculus_ai999x.solve('regularized_gamma', a=2, x=3)
# -> P = 0.800852, Q = 0.199148   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the regularized incomplete gamma function P(2, 3).	SOLVED	P = 0.800852, Q = 0.199148
Find the regularized incomplete gamma function P(-1, 3).	INVALID_INPUT	Regularized incomplete gamma functions: this capability needs a > 0 and x >= 0.

Bessel function of the first kind, `bessel_j`

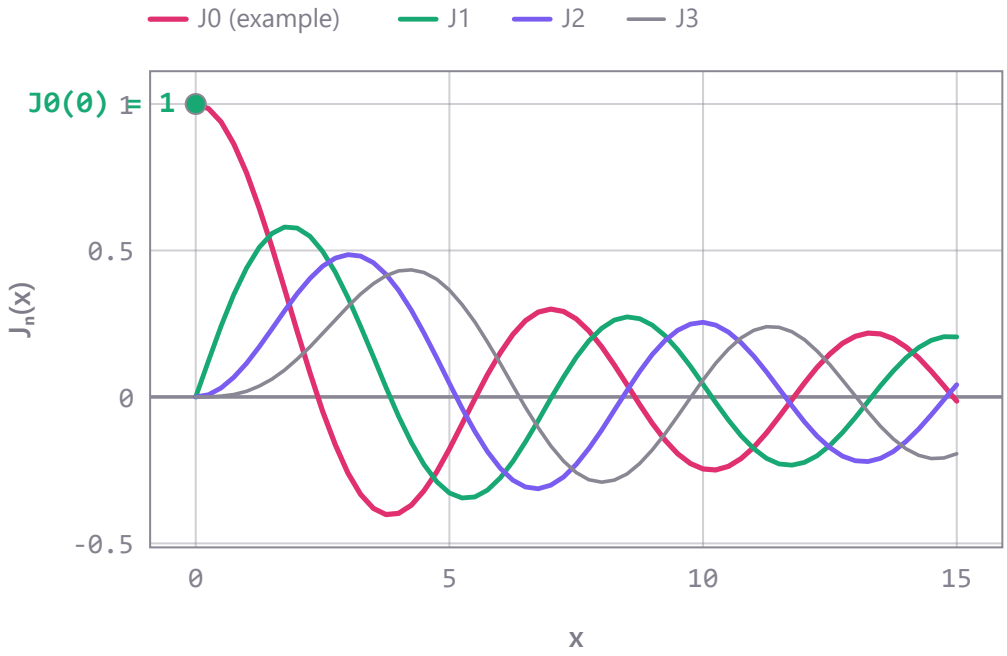
The Bessel function J_n(x) of whole-number order n.

$$J_n(x) = \sum_{m=0}^{\infty} \frac{(-1)^m}{m!\,(m+n)!} \left(\frac{x}{2}\right)^{2m+n}$$

input	meaning	condition
`n`	the order n	
`x`	the argument x	

output	meaning
`J`	J_n(x)

- **Method:** hypergeometric series at 30 digits
- **Assumes:** $0 \leq n \leq 50$; |x| <= 1000
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Bessel functions of the first kind J0, J1, J2, J3 for 0 ≤ x ≤ 15, each computed by the package at 61 points: decaying oscillations, J_n starting at 0 for n ≥ 1. Marked: the worked example, J0(0) = 1.

```
calculus_ai999x.solve('bessel_j', n=0, x=0)
# -> J = 1   (SOLVED)
```

```
calculus_ai999x.solve('bessel_j', n=0, x=1)
# -> J = 0.765198   (SOLVED)
```

```
calculus_ai999x.solve('bessel_j', n=1, x=1)
# -> J = 0.440051   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate the Bessel function J_0(1).	SOLVED	J = 0.765198
Evaluate the Bessel function of the first kind of order 1 at x = 1.	SOLVED	J = 0.440051

Riemann zeta function, `riemann_zeta`

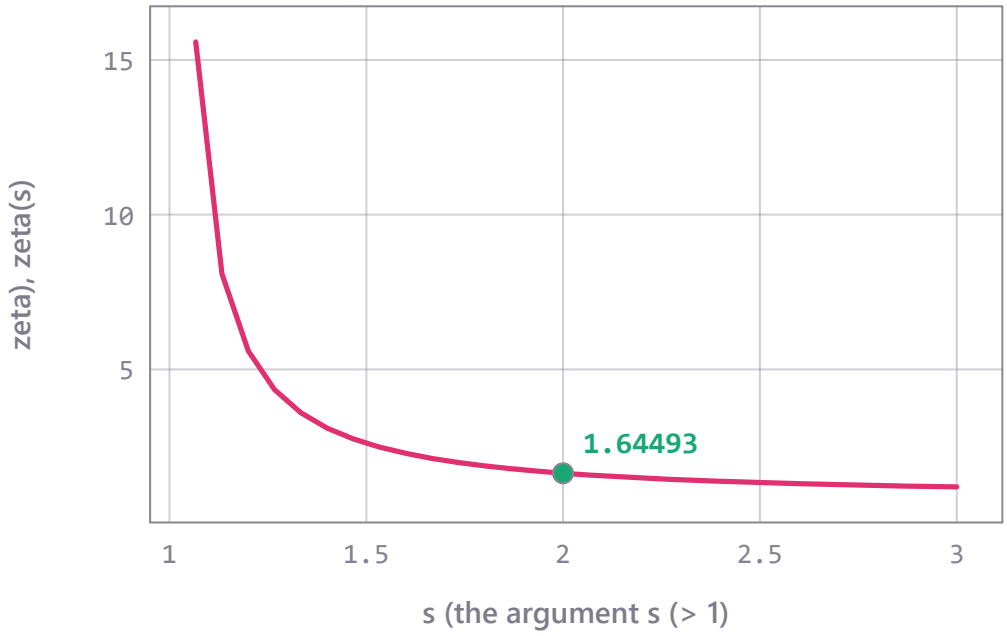
The Riemann zeta function zeta(s) = sum $1/n^s$ for $s > 1$ (the p-series sum).

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s}$$

input	meaning	condition
`s`	the argument s (> 1)	

output	meaning
`zeta`	zeta(s)

- **Method:** Euler-Maclaurin / Borwein evaluation at 30 digits
- **Assumes:** $s > 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



zeta (zeta(s)) against s (the argument s (> 1)) from 1 to 3. 30 points computed by calculus-ai999x; marked: the worked example, s = 2, zeta = 1.64493.

```
calculus_ai999x.solve('riemann_zeta', s=2)
# -> zeta = 1.64493   (SOLVED)
```

```
calculus_ai999x.solve('riemann_zeta', s=4)
# -> zeta = 1.08232   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find zeta(2).	SOLVED	zeta = 1.64493
Evaluate the Riemann zeta function at s = 4.	SOLVED	zeta = 1.08232
Find zeta(1).	INVALID_INPUT	Riemann zeta function: this capability covers s > 1 (where the defining series converges).

Legendre polynomial, `legendre_polynomial`

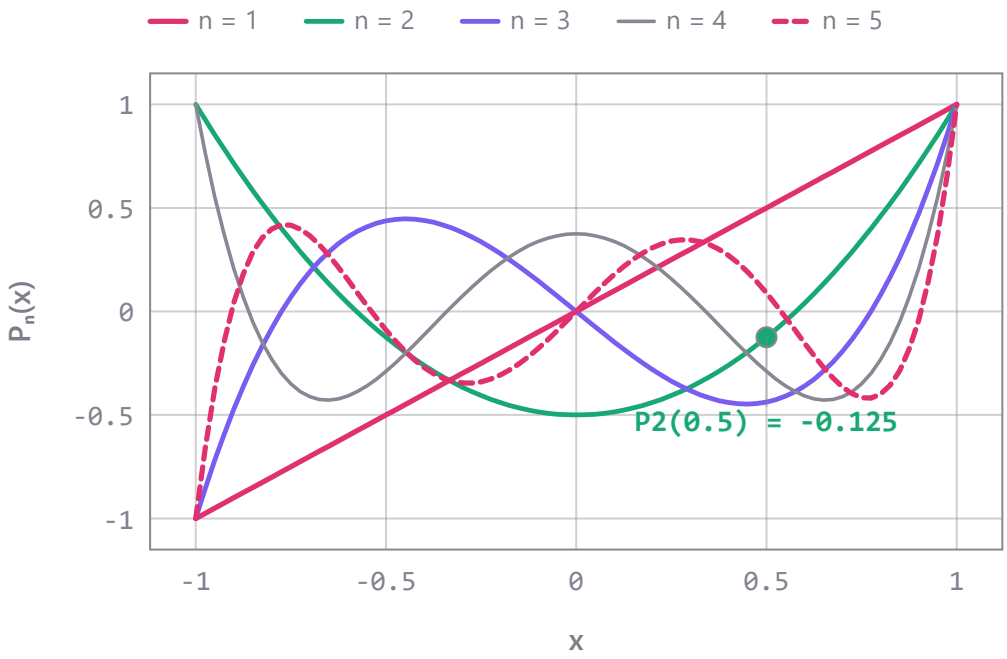
The value of the Legendre polynomial P_n(x).

$$(k+1)P_{k+1}(x) = (2k+1)\,x\,P_k(x) - k\,P_{k-1}(x)$$

input	meaning	condition
n	the degree n	
x	the argument x	

output	meaning
P	P_n(x)

- **Method:** Bonnet's three-term recurrence $(k+1)$ P_(k+1) = (2k+1) x P_k - k P_(k-1)
- **Assumes:** $0 \le n \le 200$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Legendre polynomials of degree 1 to 5 on [−1, 1], each computed by the package at 81 points: degree n has n zeros in the interval. Marked: the worked example P2(0.5) = -0.125.

```
calculus_ai999x.solve('legendre_polynomial', n=2, x=0.5)
# -> P = -0.125   (SOLVED)
```

```
calculus_ai999x.solve('legendre_polynomial', n=3, x=0.5)
# -> P = -0.4375   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate the Legendre polynomial P_3(0.5).	SOLVED	P = -0.4375

question	outcome	result
Find the Legendre polynomial of degree 2 at x = 0.5.	SOLVED	P = -0.125

Chebyshev polynomial, `chebyshev_polynomial`

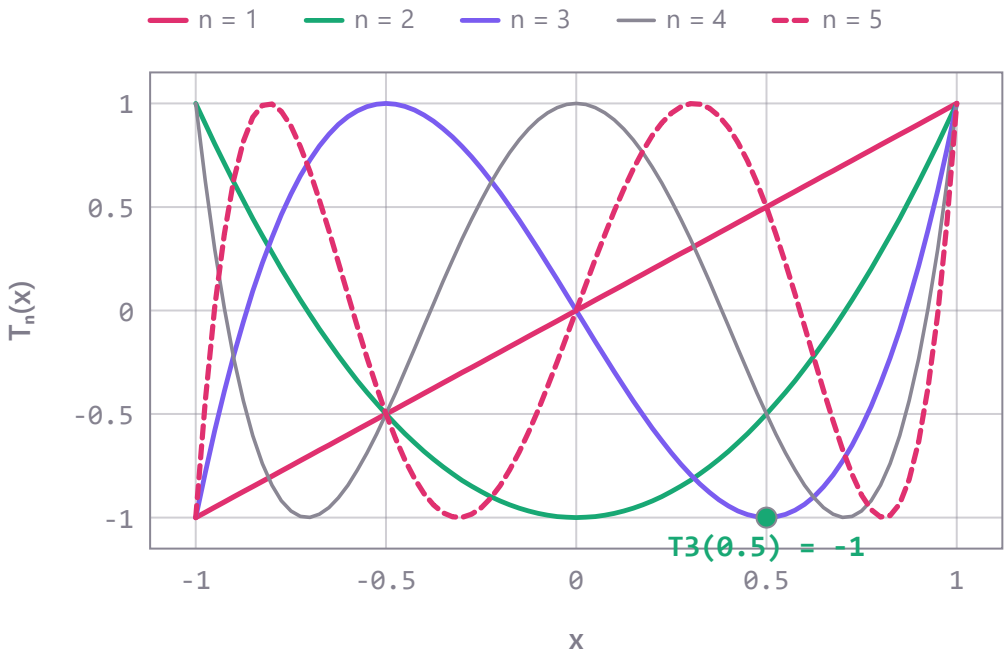
The value of the Chebyshev polynomial of the first kind T_n(x) (T_n(cos t) = cos(n t)).

$$T_{k+1}(x) = 2x\,T_k(x) - T_{k-1}(x)$$

input	meaning	condition
n	the degree n	
x	the argument x	

output	meaning
T	T_n(x)

- **Method:** the recurrence T_(k+1) = 2x T_k - T_(k-1)
- **Assumes:** $0 \le n \le 500$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Chebyshev polynomials of degree 1 to 5 on [−1, 1], each computed by the package at 81 points: degree n has n zeros in the interval. Marked: the worked example T3(0.5) = -1.

```
calculus_ai999x.solve('chebyshev_polynomial', n=3, x=0.5)
# -> T = -1   (SOLVED)
```

```
calculus_ai999x.solve('chebyshev_polynomial', n=2, x=0.3)
# -> T = -0.82   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Evaluate the Chebyshev polynomial T_3(0.5).	SOLVED	T = -1
Find the Chebyshev polynomial of degree 2 at x = 0.3.	SOLVED	T = -0.82

Fourier coefficients, `fourier_coefficients`

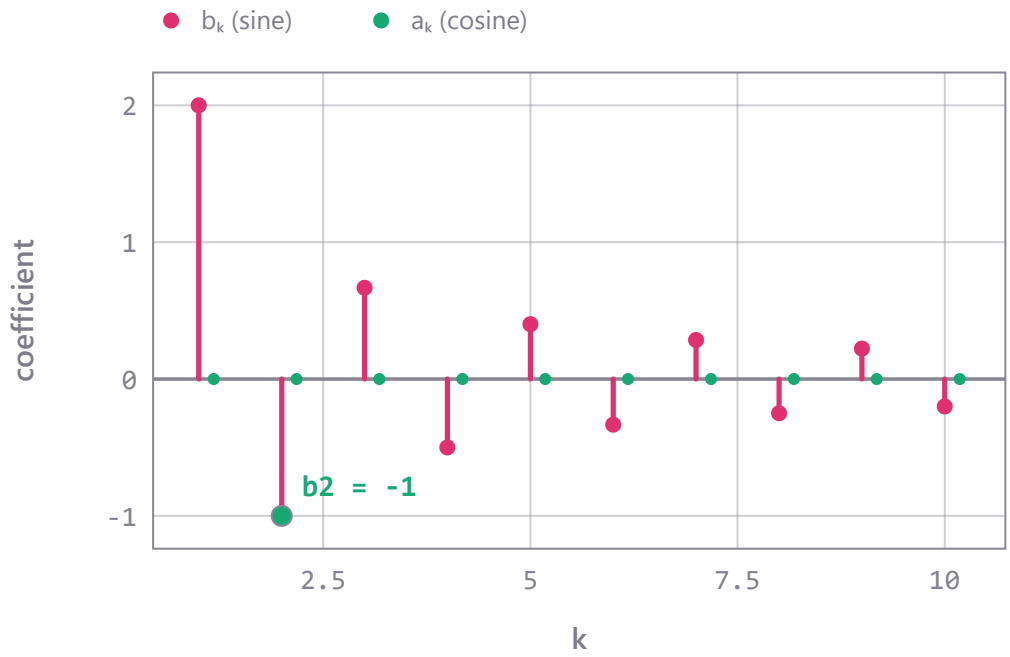
The Fourier coefficients a_0, a_k and b_k of f on [-L, L] (series a_0/2 + sum a_k cos(k pi x/L) + b_k sin(k pi x/L)).

$$a_k = \frac{1}{L}\int_{-L}^{L} f(x)\cos\frac{k\pi x}{L}dx, \quad b_k = \frac{1}{L}\int_{-L}^{L} f(x)\sin\frac{k\pi x}{L}dx$$

input	meaning	condition
`f`	the function f(x)	
`L`	the half-period L (> 0)	
`k`	the order k	

output	meaning
`a0`	$a_0 = (1/L)$ integral of f
`a_k`	a_k
`b_k`	b_k

- **Method:** tanh-sinh quadrature on the k half-wave subintervals at 20 digits
- **Assumes:** $L > 0$; f continuous on [-L, L]; $1 \le k \le 50$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Fourier coefficients of f(x) = x on [−L, L] with L = 3.14159, for k = 1 … 10, each computed by the package: an odd function has only sine terms (a_k = 0, a0 = 0), and b_k alternates in sign while shrinking like 1/k. Marked: the worked example's b2.

```
calculus_ai999x.solve('fourier_coefficients', f='x', L=3.141592653589793, k=2)
# -> a0 = 0, a_k = 0, b_k = -1   (SOLVED)
```

```
calculus_ai999x.solve('fourier_coefficients', f='x^2', L=3.141592653589793, k=1)
# -> a0 = 6.57974, a_k = -4, b_k = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Find the Fourier coefficients of order 2 of f(x) = x on [-pi, pi].	SOLVED	a0 = 0, a_k = 0, b_k = -1
Find the Fourier coefficients of order 1 of x^2 on the interval [-pi, pi].	SOLVED	a0 = 6.57974, a_k = -4, b_k = 0
Find the Fourier coefficients of order 1 of 1/x on [-1, 1].	INVALID_INPUT	Fourier coefficients: the function is not continuous (or not real) on the whole interval [-1, 1].